



可在线教学平台 www.cnxpress.com 开源电子网 /

论坛: www.cncnedy.com 正点原子官方网站

www.aliyuntek.com 正点原子淘宝店铺

<https://openedv.taobao.com>

正占原子 B 站视频: <https://space.bilibili.com/394620890>

请下载原子哥 APP，数千讲视频免费学习，更快更流畅。请关注正点原子公众号，资料发布更新我们会通知。



扫码下载“原子哥”APP



扫码关注正点原子公众号

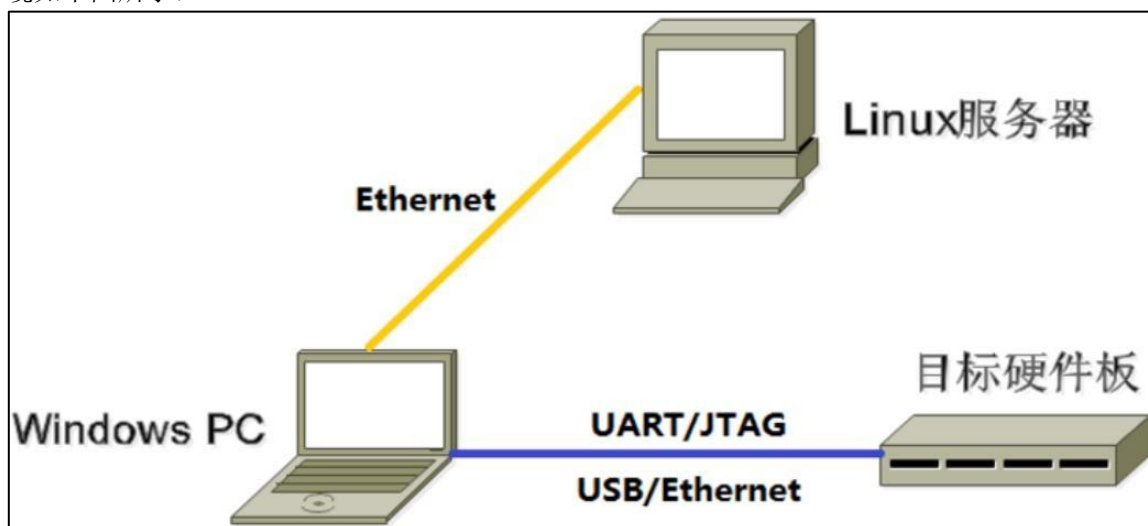
前言

本文档主要记录了正点原子 ATK-DLRK3588 开发平台 Linux 系统开发相关内容，包括以下几个章节：

- 1, 安装 Ubuntu 操作系统；
- 2, 搭建开发环境；
- 4, RK3588 Linux SDK 的安装与使用；
- 5, 镜像烧录；

嵌入式开发环境概述

一个典型的嵌入式开发环境通常包括 Linux 服务器、Windows PC 和目标硬件板，典型开发环境如下图所示：



- Linux 服务器上建立交叉编译环境，为软件开发提供代码更新下载、代码交叉编译服务。
- Windows PC 安装远程终端软件，譬如 Putty、SecureCRT 或 MobaXterm 等，通过网络远程登录（ssh 远程登录）到 Linux 服务器，进行交叉编译（将编译后生成的镜像文件通过网络拷贝到 Windows PC），及代码的开发调试。
- Windows PC 通过串口和 USB 与目标硬件板连接，可将编译后生成的镜像文件烧写到目标硬件板（一般都是通过 USB 或者 JTAG 烧录器进行烧录），并调试系统或应用程序。

虚拟机 Linux 系统+Windows PC+目标硬件板

对于很多开发者来说，可能并没有 Linux 服务器或者公司并未提供专用于代码交叉编译的 Linux 服务器，你只有一台 Windows 电脑；那么这个时候可以在 Windows PC 上安装虚拟机软件（譬如 VMware、Virtual Box 或 Virtual PC），使用虚拟机软件在 Windows PC 上创建虚拟机，虚拟机安装 Linux 操作系统（譬如 Ubuntu）；使用虚拟机 Linux 系统来代替 Linux 服务器。

第一章 安装 Ubuntu 系统

嵌入式 linux 开发一般都是基于 Windows+Ubuntu 双系统开发环境, 本章向大家介绍如何在虚拟机上安装 Ubuntu 操作系统。

本章将分为如下几个小节:

1.1 安装 VMware 虚拟机软件;

1.2 安装 Ubuntu 系统。

1.1 安装 VMware 虚拟机软件

VMware (Vmware Workstation) 是一款功能非常强大的虚拟机软件, 通过该软件我们可以在 Windows PC 上创建一个或多个虚拟机。所谓虚拟机 (Virtual Machine), 它是指通过软件模拟出来的具有实体计算机完整硬件系统功能 (譬如内存、硬盘、处理器等) 的计算机系统。可以在虚拟机中安装实体机所支持的各种操作系统, 譬如 Windows 操作系统、DOS、Linux 操作系统 (Ubuntu、Debian、CentOS 等), 在实体计算机中能够完成的工作在虚拟机中也能够完成, 可以像使用实体机一样去使用虚拟机。

目前流行的虚拟机软件有 VMware (Vmware Workstation)、Virtual Box 和 Virtual PC 等, 本文档将向用户介绍如何通过 VMware 软件创建虚拟机并安装 Ubuntu 操作系统。

1.1.1 下载 VMware 软件

开发板资料盘中已经给大家提供了 VMware 虚拟机软件的安装包文件, 路径为: **开发板光盘 A 盘-基础资料 04、软件 04-VMware-workstation-full-16.2.1-18811642.exe**

对 VMware 虚拟机软件的版本没有什么要求, 最新版本也行、旧的版本 (譬如 15.X.X 或 14.X.X) 也可以, 本文档将以 16.2.1 版本为例, 其对应的安装包文件名为 VMware-workstationfull-16.2.1-18811642.exe。



图 1.1.1.5 16.2.1 版本的 VMware 虚拟机软件安装包文件

1.1.2 安装 VMware 软件

Windows 下安装 VMware 软件非常简单, 直接双击运行安装包文件 VMware-workstation-full-16.2.1-18811642.exe, 按照图 1.1.2.1~1.1.2.8 所示操作步骤安装 VMware 虚拟机软件:

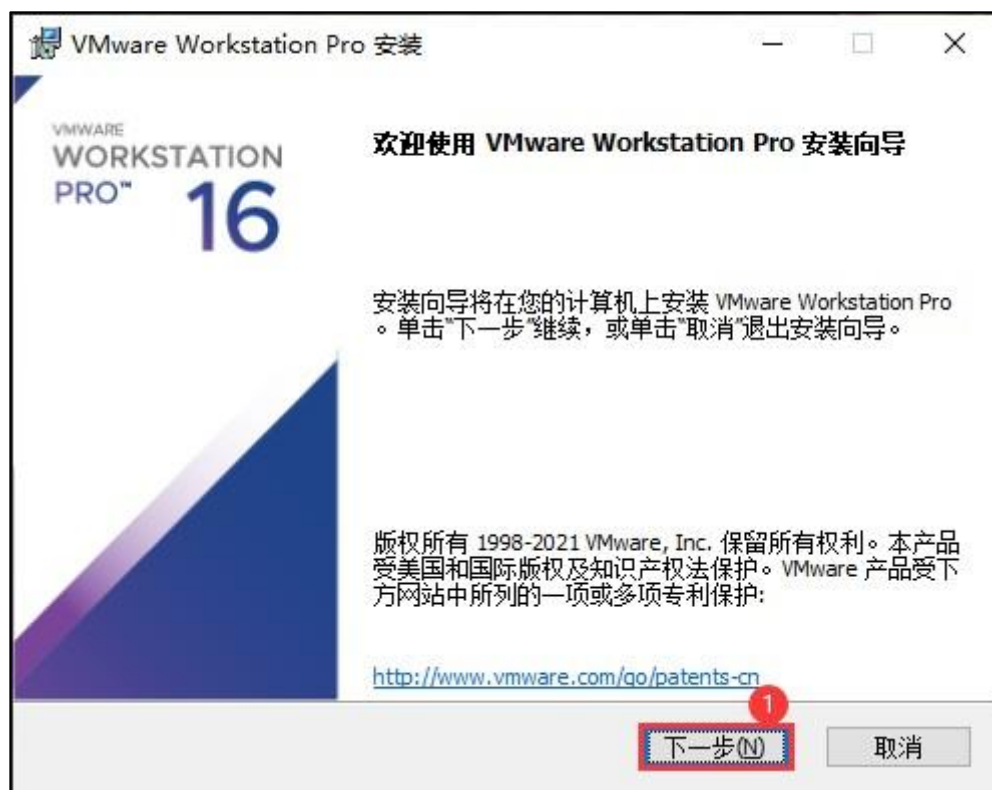


图 1.1.2.1 安装 VMware 软件 1

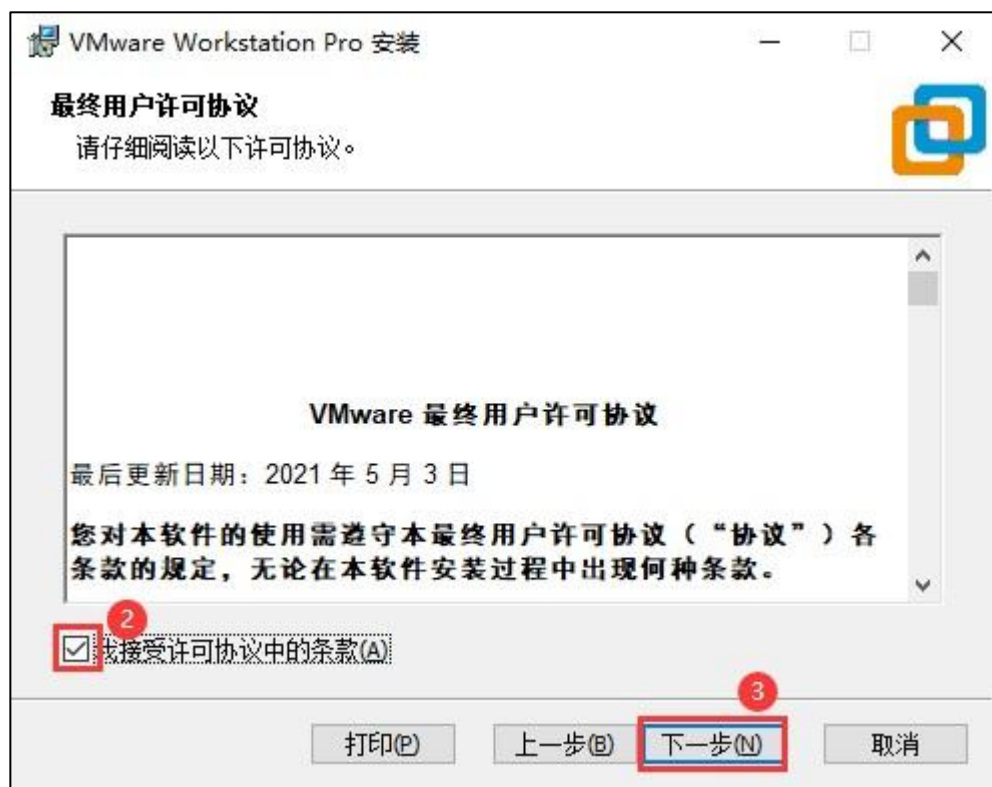


图 1.1.2.2 安装 VMware 软件 2



1.2.3

3

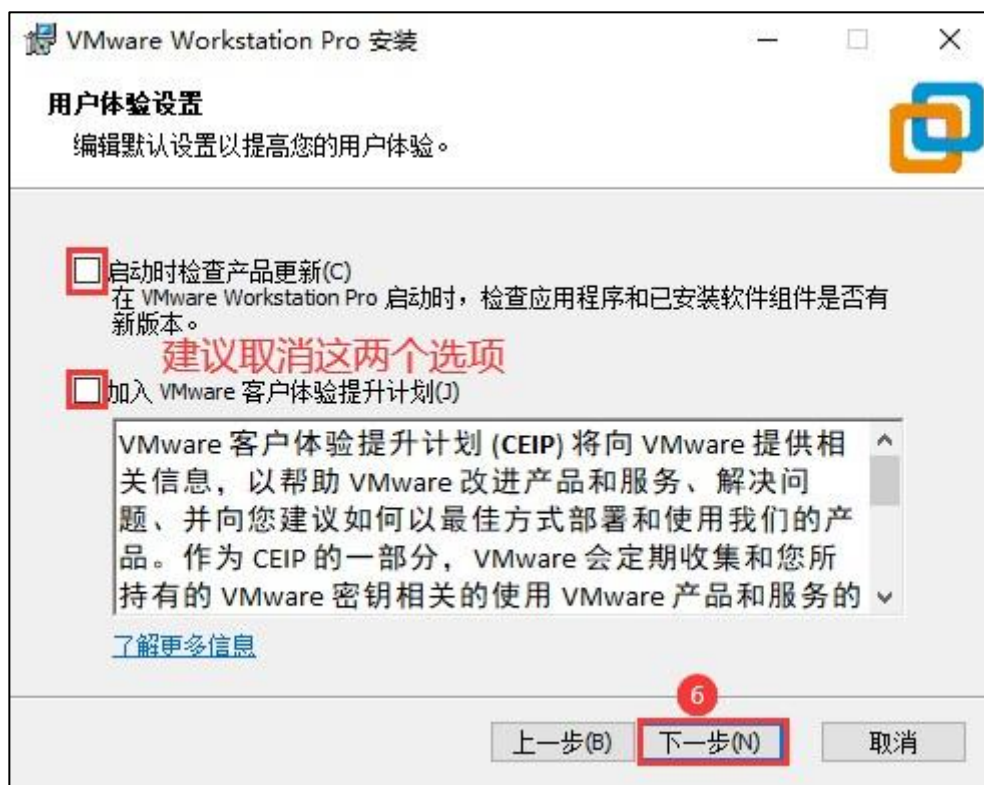
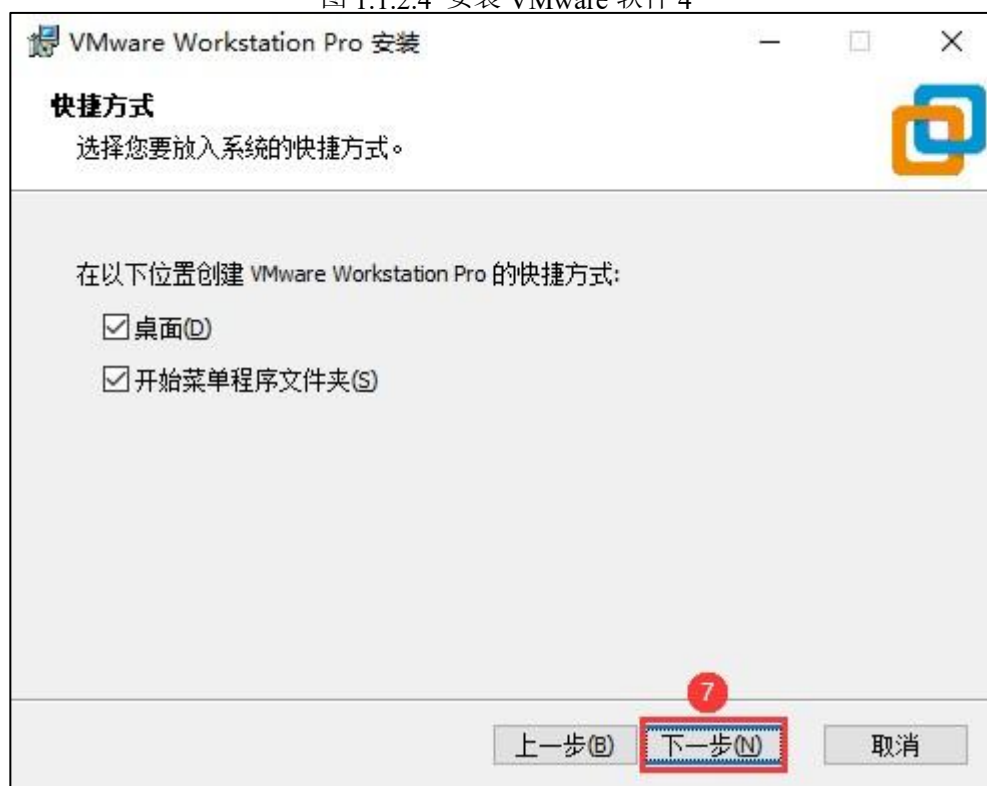


图 1.1.2.4 安装 VMware 软件 4



1.1.2.5

5

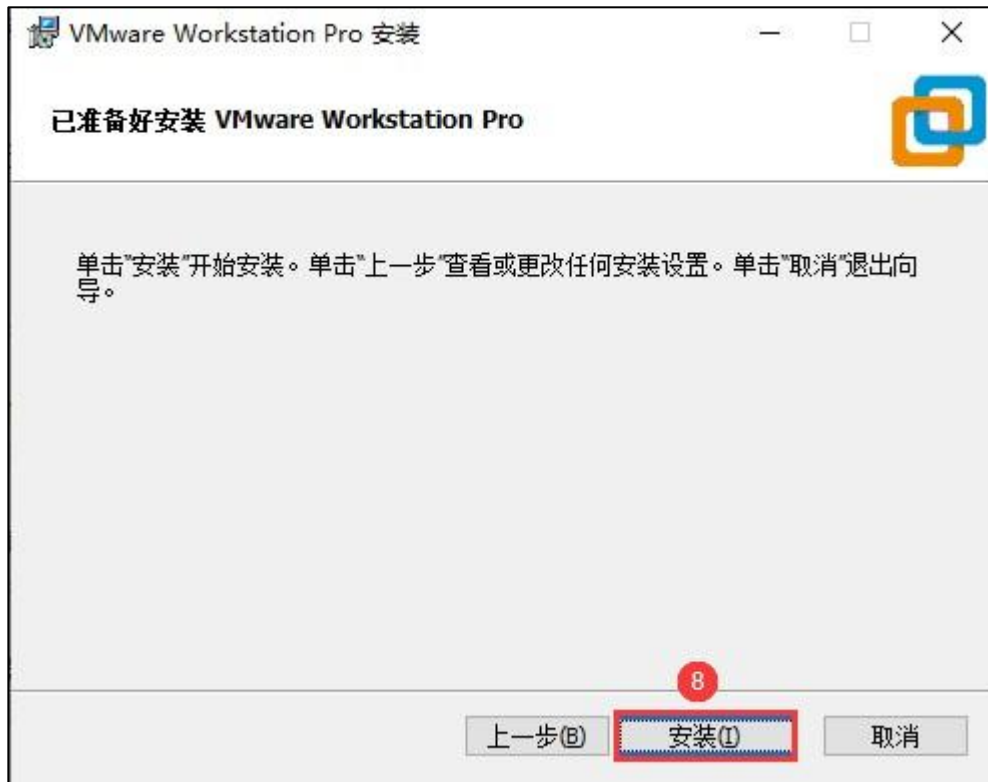
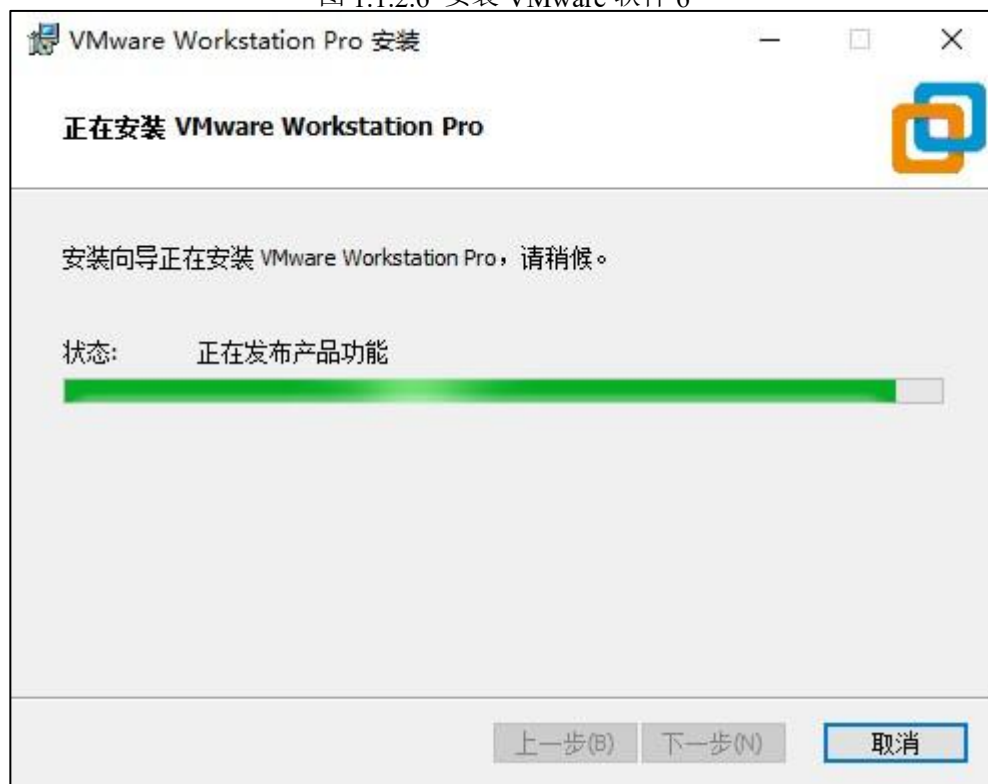


图 1.1.2.6 安装 VMware 软件 6



.1.2.7

7



图 1.1.2.8 安装 VMware 软件 8

点击“完成”按钮，至此，VMware 虚拟机软件安装完成。

安装完成后，会在 Windows 系统桌面生成 VMware 虚拟机软件快捷方式图标，如图 1.1.2.9 所示：



图 1.1.2.9 VMware 软件桌面快捷方式图标

双击图标打开 VMware 虚拟机软件，首次打开软件会提示用户输入许可证密钥，如图 1.1.2.10 所示：



图 1.1.2.10 输入许可证密钥

VMware 虚拟机软件是付费软件，需要用户购买才能使用；如果用户购买了 VMware 软件使用许可，那么将会得到一串许可证密钥，将许可证密钥填写至“我有 **VMware Workstation 16** 的许可证密钥(H)”所对应的输入框中进行激活，激活成功后便可以正常使用 VMware 软件了。如果用户没有购买软件使用许可，那么可以选择“我希望使用 **VMware Workstation 16 30 天(W)**”选项，这样便可获得 VMware 软件的 30 天试用期，点击“继续”按钮：



图 1.1.2.11 欢迎使用 VMware 软件

点击“完成”按钮，接着进入到 VMware 虚拟机软件主界面，如图 1.1.2.12 所示：

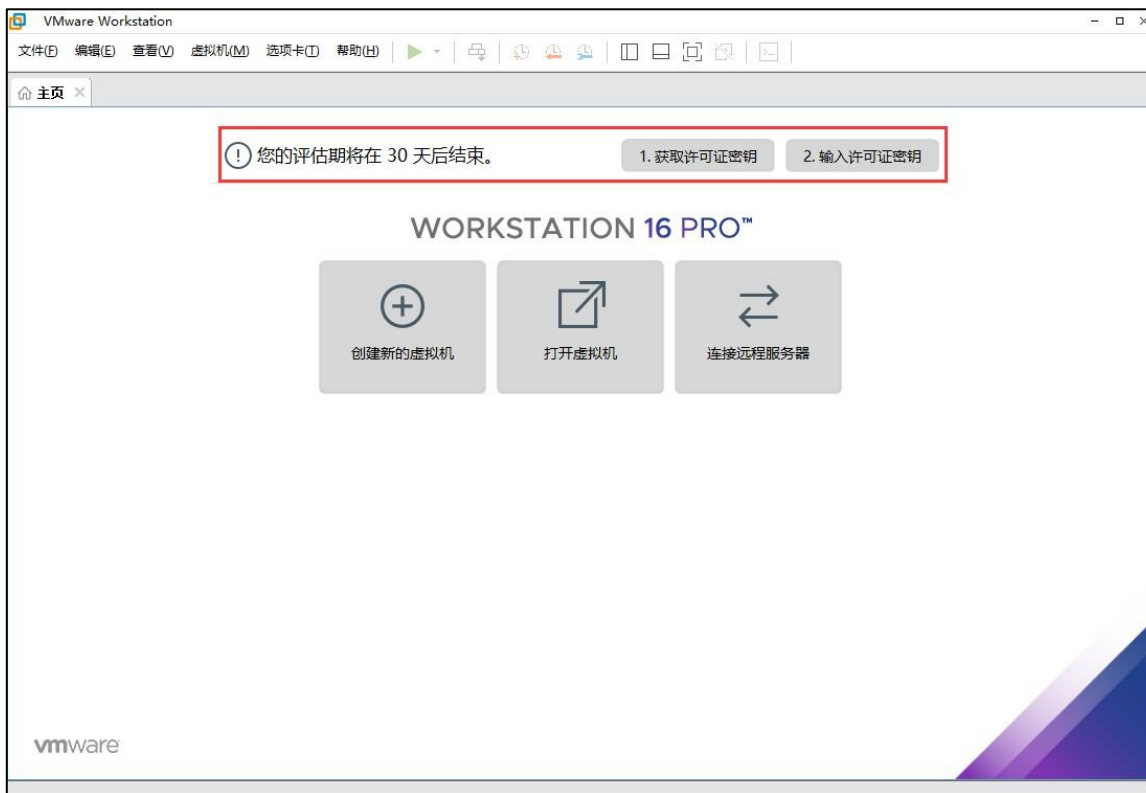
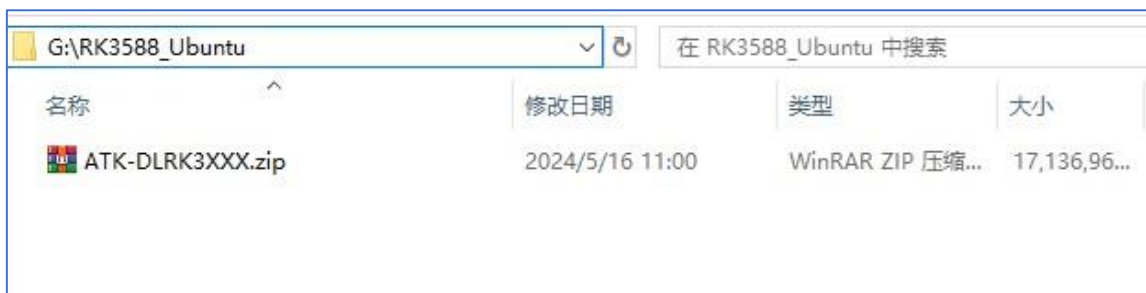


图 1.1.2.12 VMware 虚拟机软件主界面

2.2 解压 ATK-DLRK3XXX.zip

将开发板光盘 B 盘-开发环境及 SDK⑦01、Ubuntu 开发环境⑦ATK-DLRK3XXX.zip 压缩文件拷贝到某个目录下（考虑到后续需要编译 SDK，建议预留出 300~500GB 磁盘空间），譬如 **G:\RK3588_Ubuntu**，如下所示：



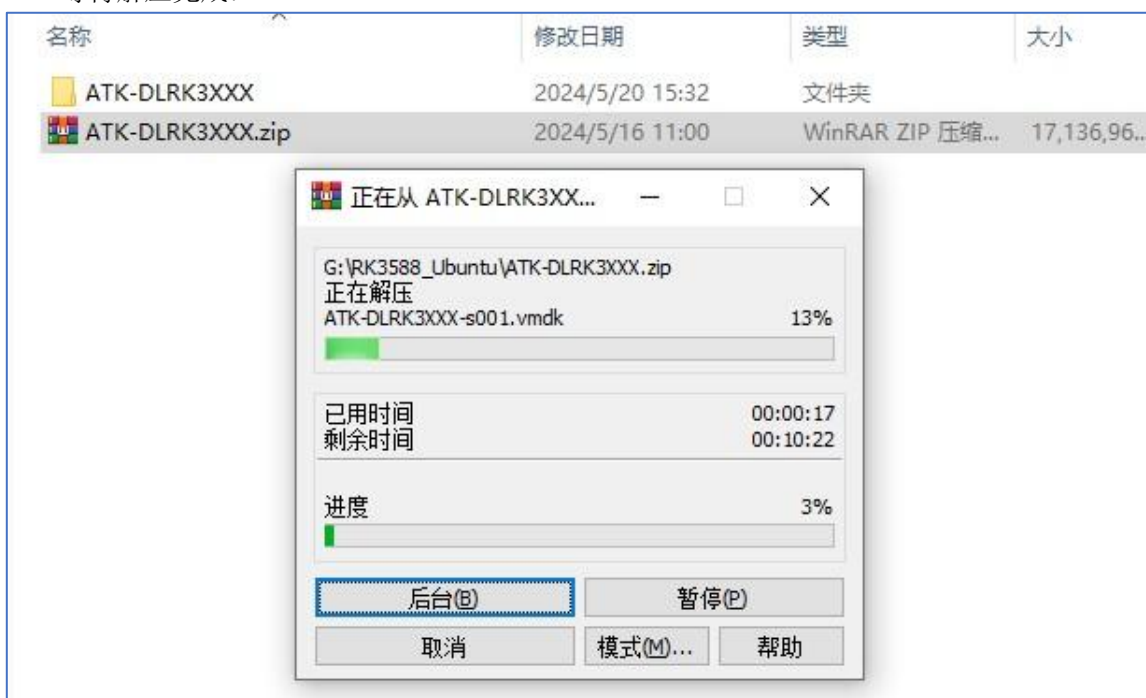
将 ATK-DLRK3XXX.zip 拷贝到某个目录

接着将其解压到当前目录：



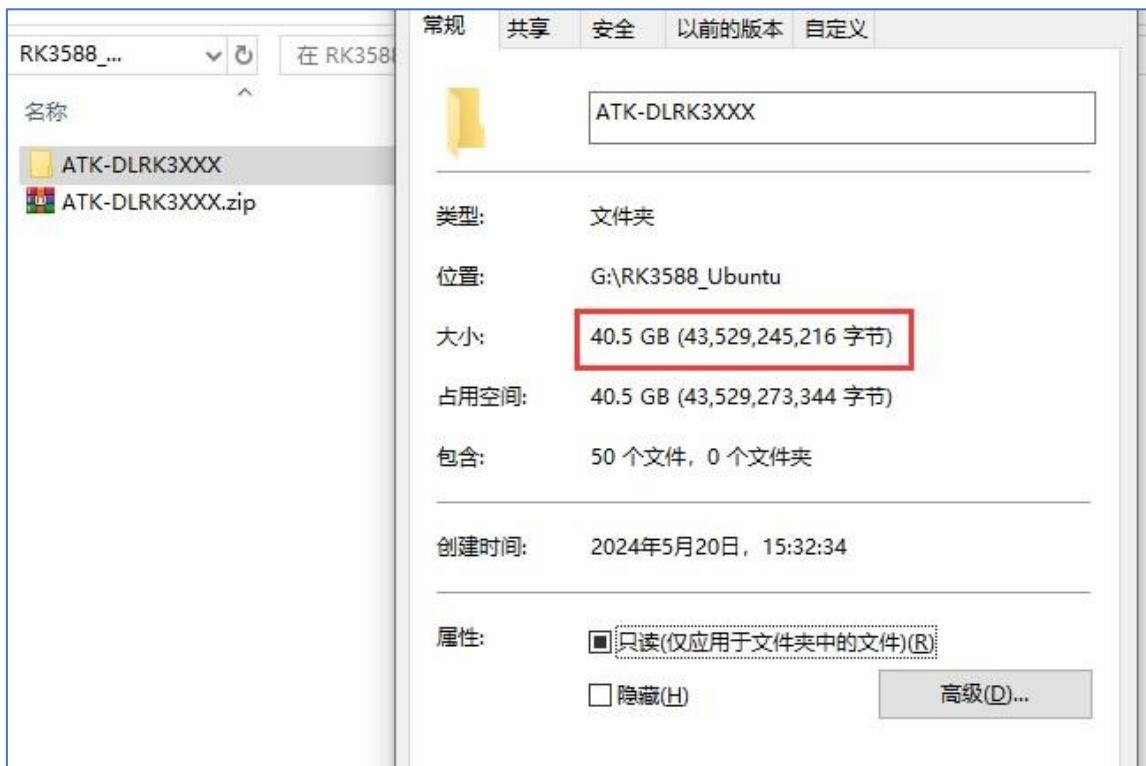
将 ATK-DLRK3XXX.zip 解压到当前目录

等待解压完成:



等待解压完成

解压完成后得到 Ubuntu 虚拟机文件夹 **ATK-DLRK3XXX**, 该文件夹的大小为 40.5GB, 如下所示 (解压完成后将 ATK-DLRK3XXX.zip 压缩文件删除, 避免占用磁盘空间):



ATK-DLRK3XXX 文件夹中的内容如下所示:

ATK-DLRK3XXX.nvram	2023/7/15 14:28	VMware 虚拟机...
ATK-DLRK3XXX.vmdk	2024/5/16 10:18	VMware 虚拟磁...
ATK-DLRK3XXX.vmsd	2023/7/15 11:50	VMware 快照元...
ATK-DLRK3XXX.vmx	2024/5/16 10:33	VMware 虚拟机...
ATK-DLRK3XXX.vmx	2023/7/15 11:50	VMware 组成...
ATK-DLRK3XXX-s001.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s002.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s003.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s004.vmdk	2024/3/25 16:17	VMware 虚拟磁...
ATK-DLRK3XXX-s005.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s006.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s007.vmdk	2024/5/16 10:24	VMware 虚拟磁...
ATK-DLRK3XXX-s008.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s009.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s010.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s011.vmdk	2024/3/25 16:17	VMware 虚拟磁...
ATK-DLRK3XXX-s012.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s013.vmdk	2024/3/25 16:17	VMware 虚拟磁...
ATK-DLRK3XXX-s014.vmdk	2024/3/25 16:17	VMware 虚拟磁...
ATK-DLRK3XXX-s015.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s016.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s017.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s018.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s019.vmdk	2024/5/16 10:32	VMware 虚拟磁...
ATK-DLRK3XXX-s020.vmdk	2024/5/16 10:33	VMware 虚拟磁...
ATK-DLRK3XXX-s021.vmdk	2024/5/16 10:32	VMware 虚拟磁...
ATK-DLRK3XXX-s022.vmdk	2024/3/25 16:17	VMware 虚拟磁...
ATK-DLRK3XXX-s023.vmdk	2024/3/25 16:17	VMware 虚拟磁...

2.3 启动虚拟机

注意：您需要提前安装好 VMware 虚拟机软件、方可进行接下来的操作！VMware 软件的安装方法可以参考<开发板光盘 A 盘-基础资料⑦10、用户手册⑦02、开发文档⑦【正点原子】ATK-DLRK3588 嵌入式 Linux 系统开发手册 V1.0.pdf>文档中的 [1.1 小节](#)。

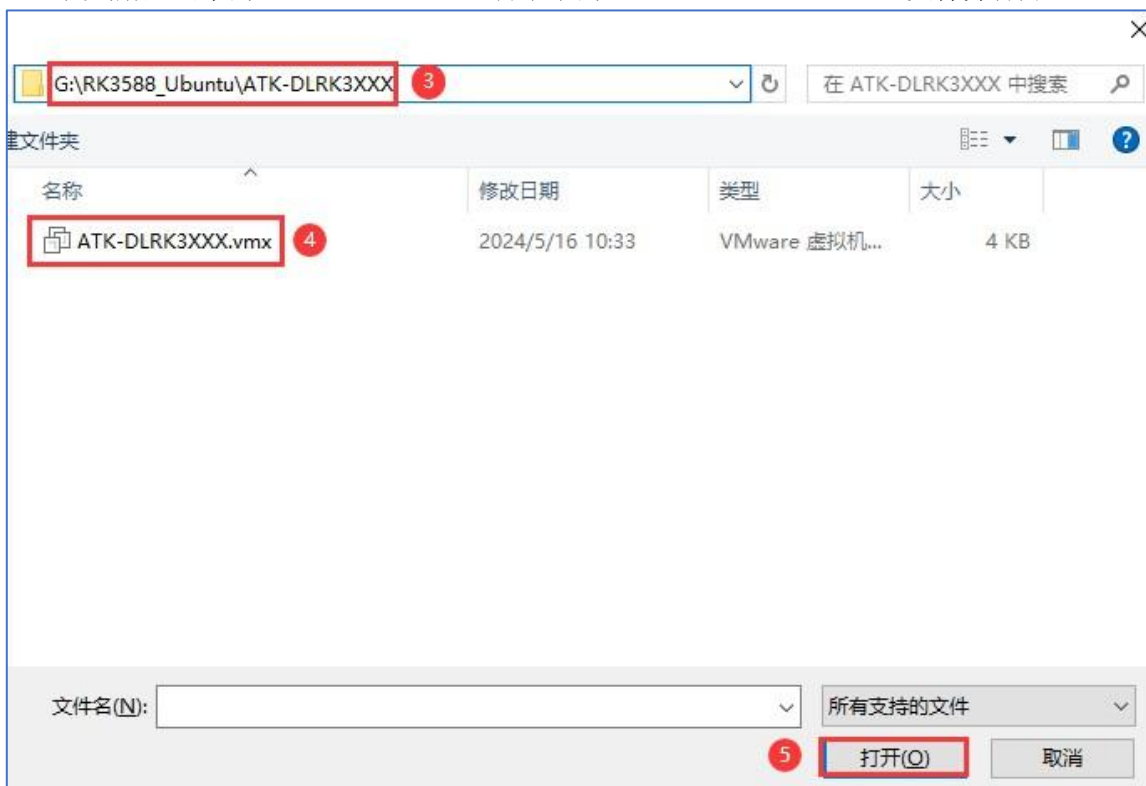
2.3.1 打开虚拟机

VMware 软件安装成功后，打开 VMware 软件，点击左上角菜单：**文件⑦打开：**



打开虚拟机

找到解压出来的 ATK-DLRK3XXX 目录下的 **ATK-DLRK3XXX.vmx** 文件并打开。



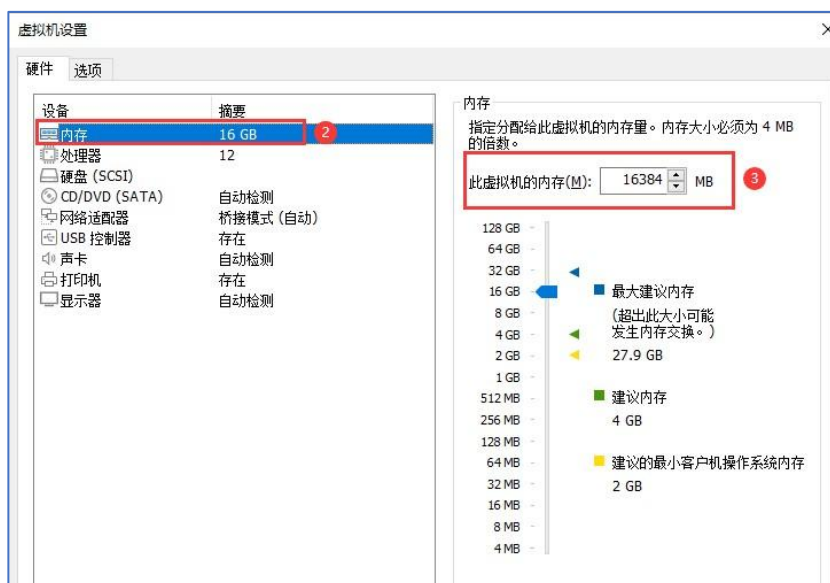
打开.vmx 文件

打开成功后，如下图所示：

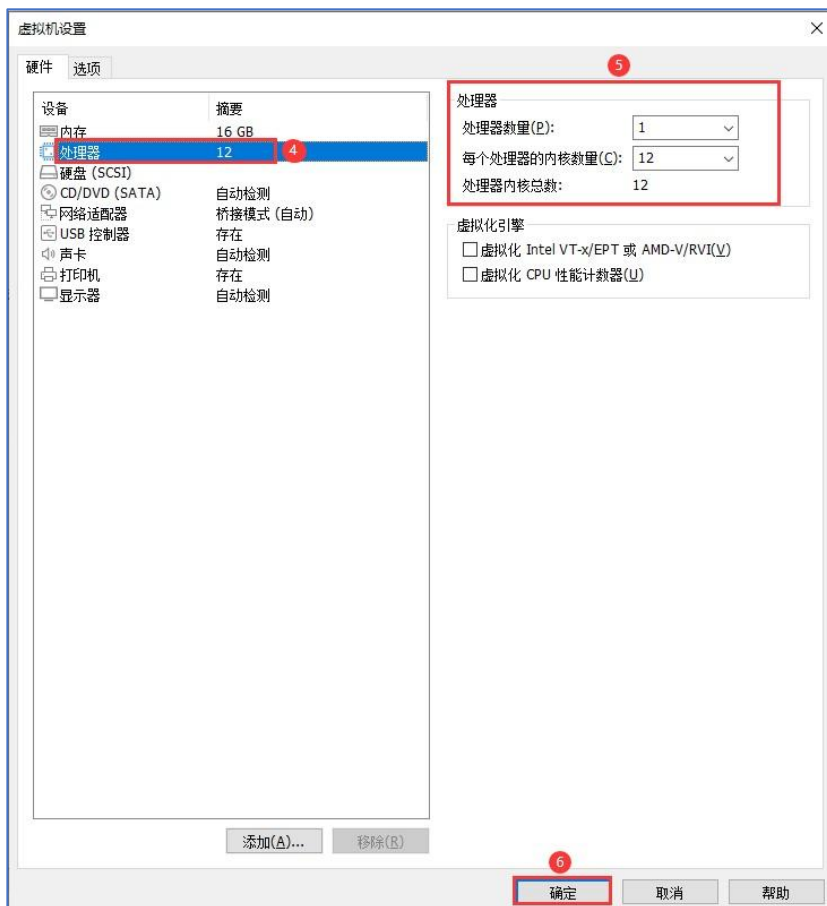


2.3.2 配置虚拟机

此时先别着急开启虚拟机，我们需要检查虚拟机的配置信息，尤其是处理器数量和内存大小，如果与自己电脑的实际配置情况不符，则需对虚拟机进行设置：**必须要保证虚拟机的处理器数量小于电脑的逻辑处理器数量（也就是 CPU 实际线程数）、虚拟机的内存容量小于电脑的内存容量，否则将无法打开虚拟机！切记！** 点击“编辑虚拟机设置”按钮可对虚拟机进行设置，如下所示：



设置虚拟机的内存大小



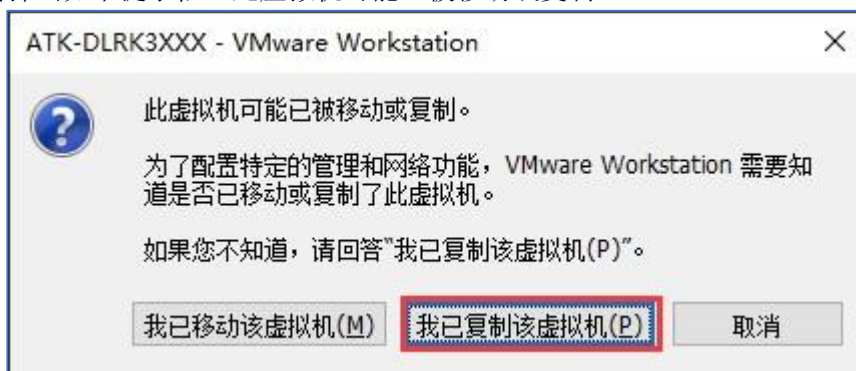
设置虚拟机的处理器数量

用户需根据自己电脑的实际配置情况来设置虚拟机的内存大小以及处理器数量。虚拟机的处理器数量将会影响 SDK 的编译速度，一般而言，处理器数量越多编译速度自然也就越快；此外，虚拟机的内存不能太小，否则编译 SDK 必然会报错，应尽量大于或等于 **16GB**！**2.3.3 启动虚拟机**虚拟机配置完成后，点击“**开启此虚拟机**”按钮开启 Ubuntu 虚拟机：



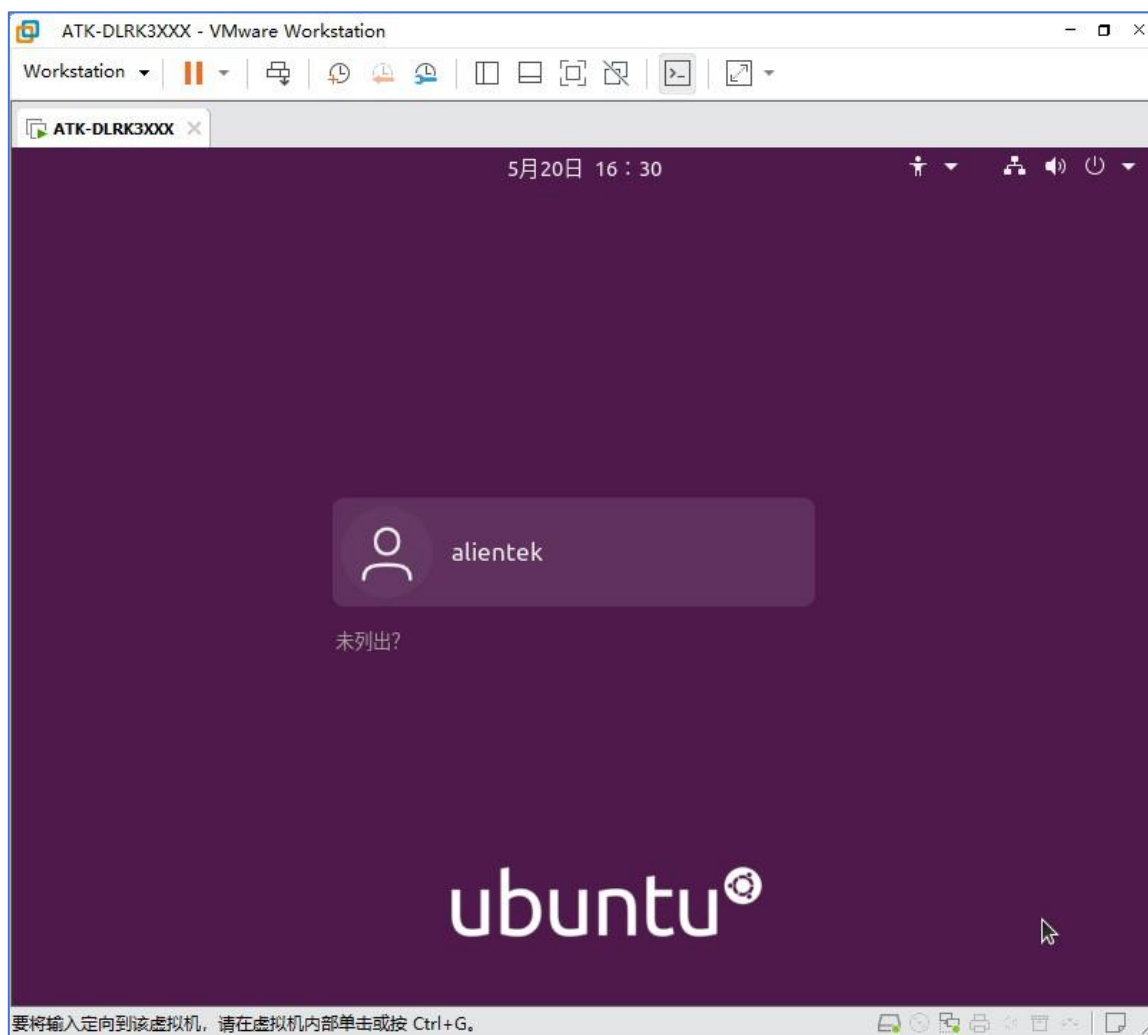
开启 Ubuntu 虚拟机

接下来将会弹出如下提示框（此虚拟机可能已被移动或复制）：



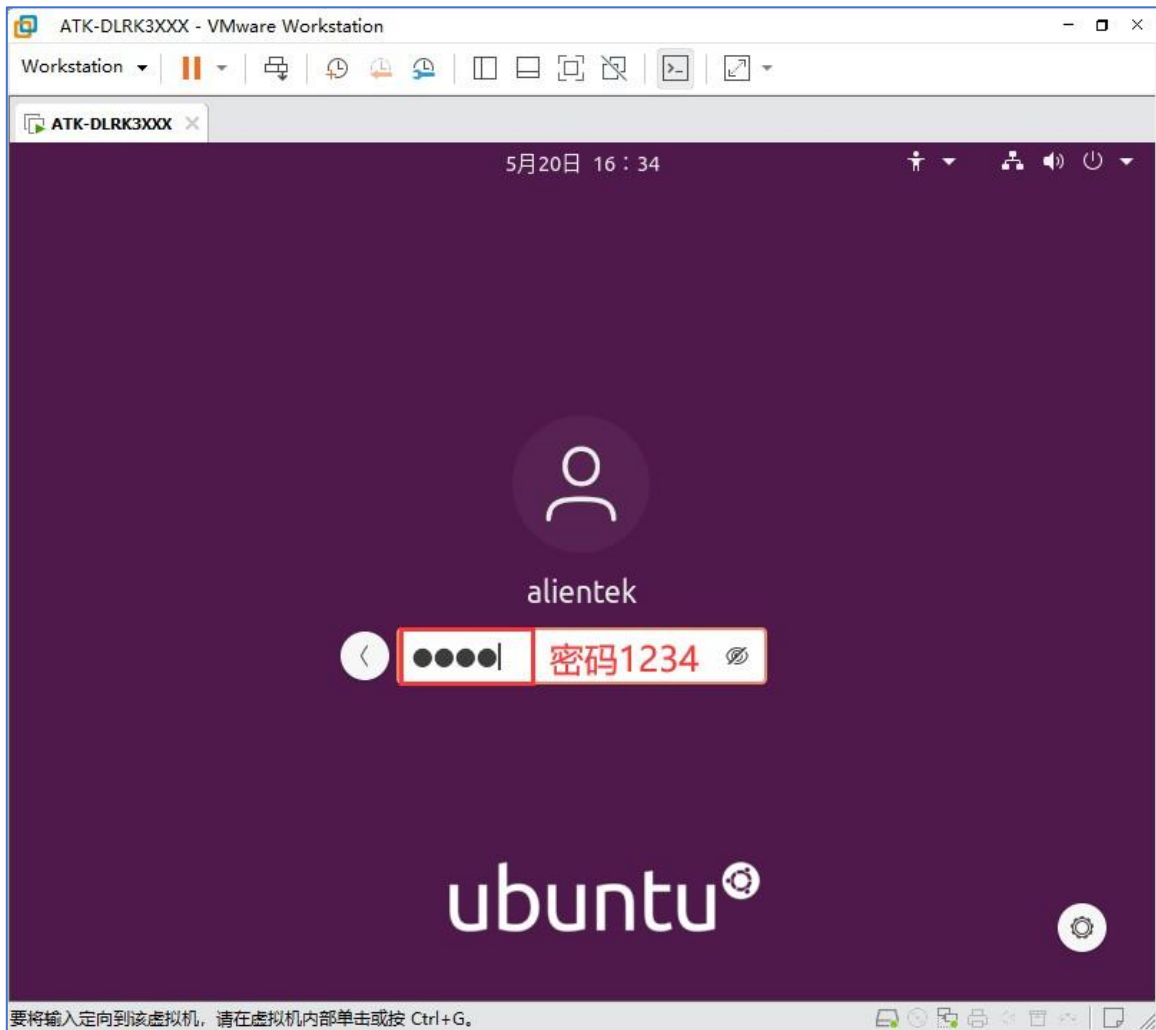
这里点击“我已复制该虚拟机(P)”按钮。

接下来不出现意外的话，将会成功启动虚拟机，如下所示：



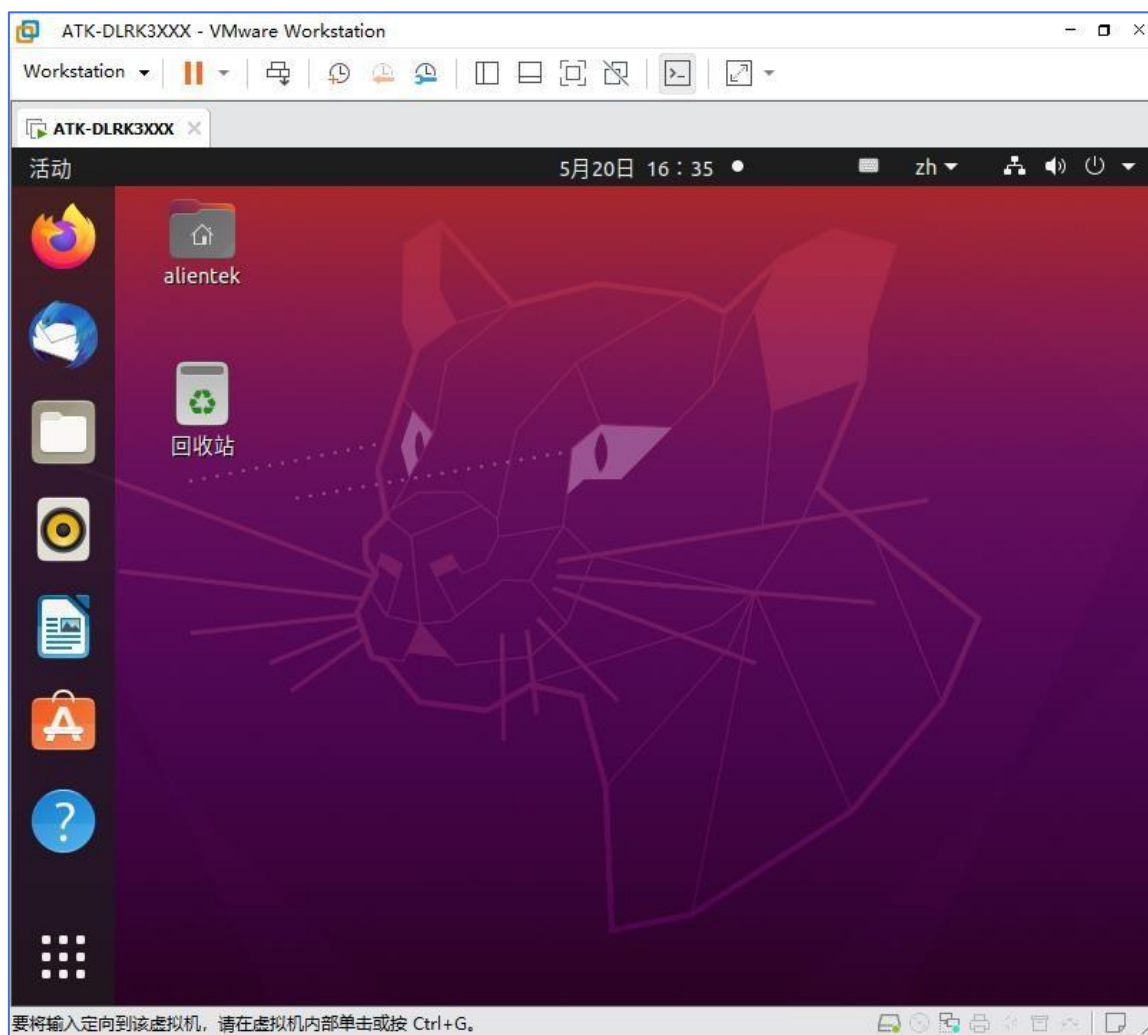
成功启动虚拟机 接着输入密码登录

Ubuntu 系统（默认密码为 1234）:



登录 Ubuntu 系统

登录成功进入 Ubuntu 系统主界面:



Ubuntu 系统主界面

第二章 开发环境搭建

在做 RK3588 嵌入式 Linux 开发之前，我们需要先搭建好开发环境；包括 Windows 和 Ubuntu 这两种操作系统下的环境搭建，因为嵌入式 linux 开发一般都是基于 Windows+Ubuntu 双系统开发环境。

本章将分为如下几个小节：

- 2.1 Ubuntu 系统设置；
- 2.2 Ubuntu 与 Windows 之间文件互传；
- 2.6 CH343 串口驱动安装；
- 2.7 MobaXterm 软件安装；
- 2.8 Rockchip USB 驱动安装；
- 2.9 镜像烧录工具的使用；
- 2.10 adb 工具安装。

2.2 Ubuntu 与 Windows 之间文件互传

我们在开发过程中，经常需要在 Windows 系统与 Ubuntu 系统之间进行文件传输，文件互传的方式比较多，譬如共享文件夹、Samba、FTP、scp 等等，本小节向用户介绍通过 FTP 方式进行文件传输。

通过 FTP（File Transfer Protocol，文件传输协议）在 Windows 与 Ubuntu 之间进行文件传输，需要完成以下两件事情。

2.2.1 Ubuntu 系统下搭建 FTP 服务器

在 Ubuntu 系统下打开终端，执行如下命令安装 FTP 服务：

```
sudo apt-get update
sudo apt-get install vsftpd
```

vsftpd 安装完成后，使用 vi 命令打开/etc/vsftpd.conf 配置文件，如果没有安装 vim 软件，则需要先通过如下命令安装 vim 编辑器：

```
sudo apt-get install vim
```

然后再执行如下命令打开/etc/vsftpd.conf 配置文件：

```
sudo vi /etc/vsftpd.conf
```

打开配置文件后，找到如下两行，确保其前面没有“#”（“#”号表示注释，我们要取消注释）：

```
26 #
27 # Uncomment this to allow local users to log in.
28 local_enable=YES
29 #
30 # Uncomment this to enable any form of FTP write command.
31 write_enable=YES
32 #
```

图 2.2.1.1 修改/etc/vsftpd.conf 配置文件

默认情况下，“write_enable=YES”前面有一个“#”号，我们需要将其去掉，使能该配置。修改完成后保存退出，然后执行如下命令重启 FTP 服务：

```
sudo /etc/init.d/vsftpd restart
```

可通过如下命令确认 FTP 服务是否开启:

```
ps -aux | grep vsftpd | grep -v grep
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ ps -aux | grep vsftpd | grep -v grep
root      5469  0.0  0.0  6816  3076 ?        Ss   11:15   0:00 /usr/sbin/vsftpd /etc/vsftpd.conf
tgg@tgg-virtual-machine:~$
```

图 2.2.1.2 确认 FTP 服务是否开启

2.2.2 Windows 下安装 FTP 客户端

我们需要在 Windows 系统下安装一个 FTP 客户端软件, 这里选择 FileZilla 作为 FTP 客户端软件, 这是一个免费的 FTP 客户端软件。

ATK-DLRK3588 开发板资料盘中已经给用户提供了 FileZilla 软件安装包, 路径为: 开发板光盘 A 盘-基础资料⑦04、软件⑦FileZilla_3.61.0_win64-setup.exe,



图 2.2.2.2 FileZilla 软件安装包

接着双击安装包文件 FileZilla_3.61.0_win64-setup.exe, 按照图 2.2.2.3~2.2.2.8 所示步骤进行安装:

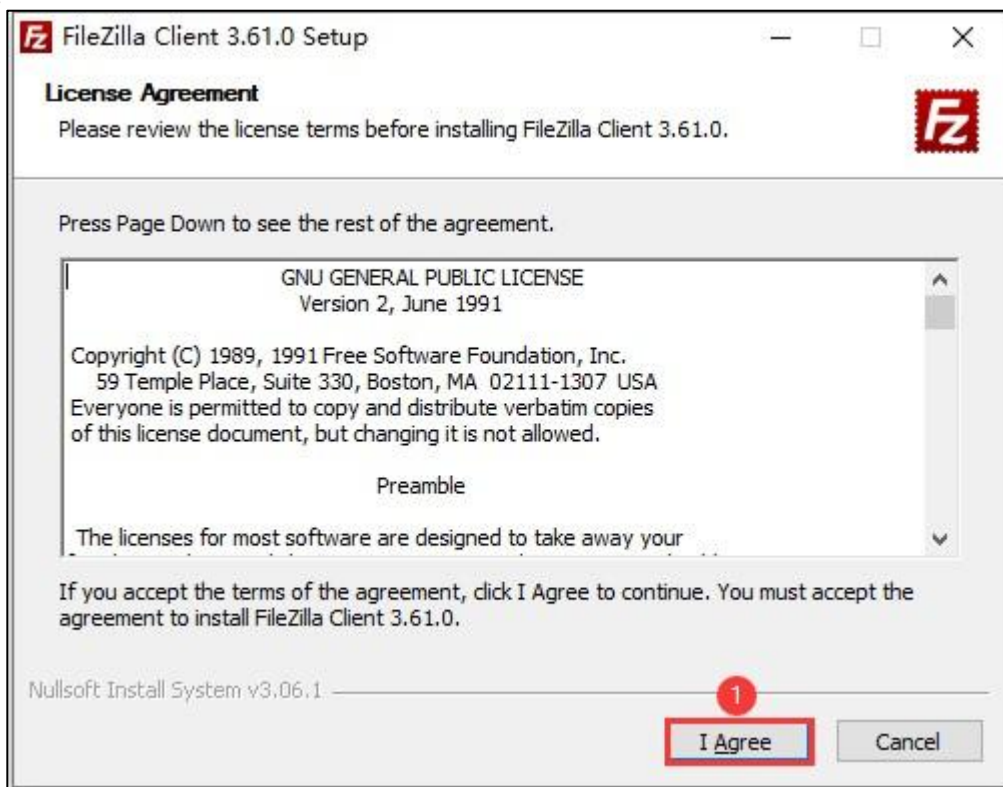


图 2.2.2.3 安装 FileZilla 软件(1)

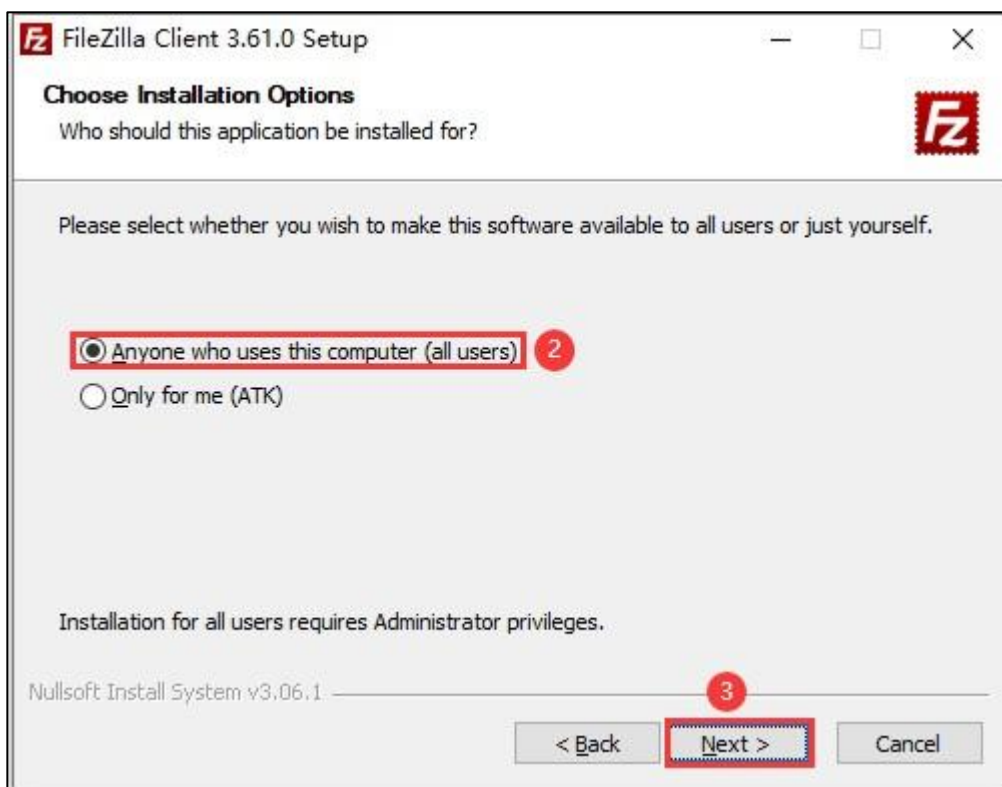


图 2.2.2.4 安装 FileZilla 软件(2)

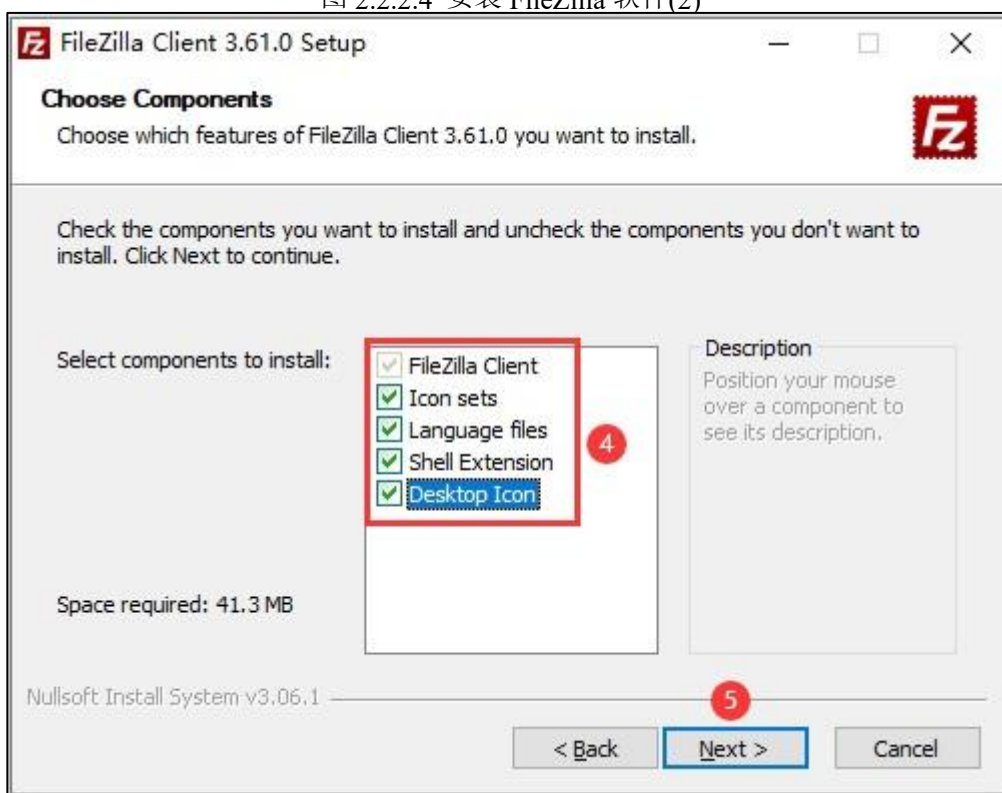


图 2.2.2.5 安装 FileZilla 软件(3)

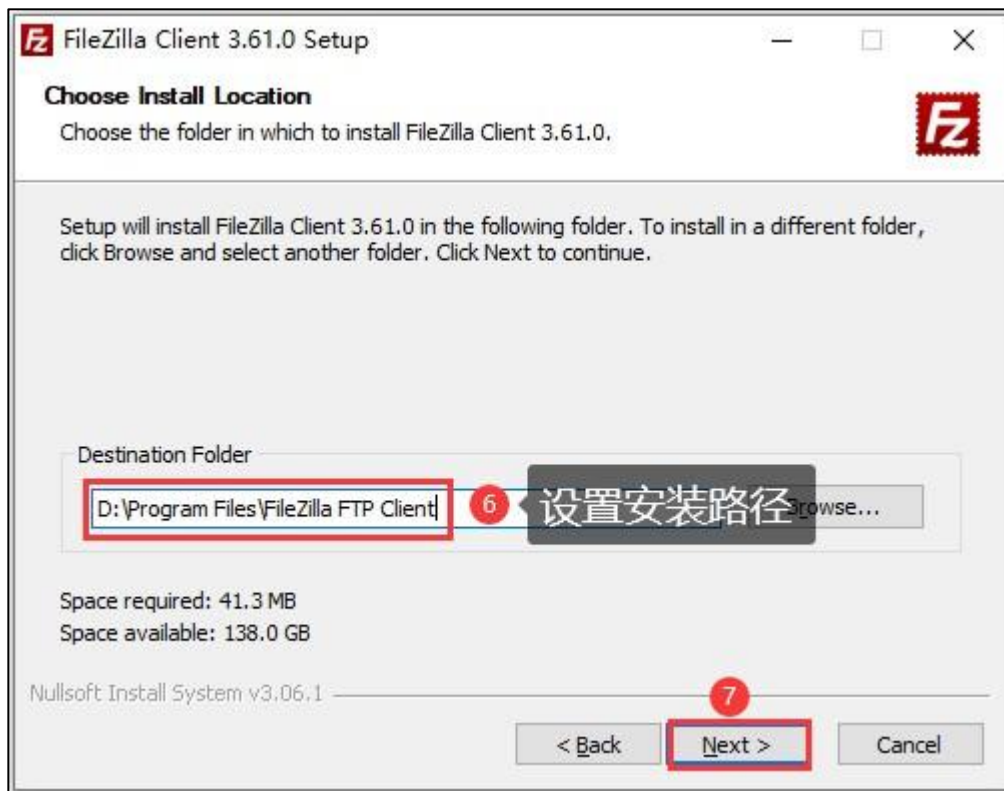


图 2.2.2.6 安装 FileZilla 软件(4)

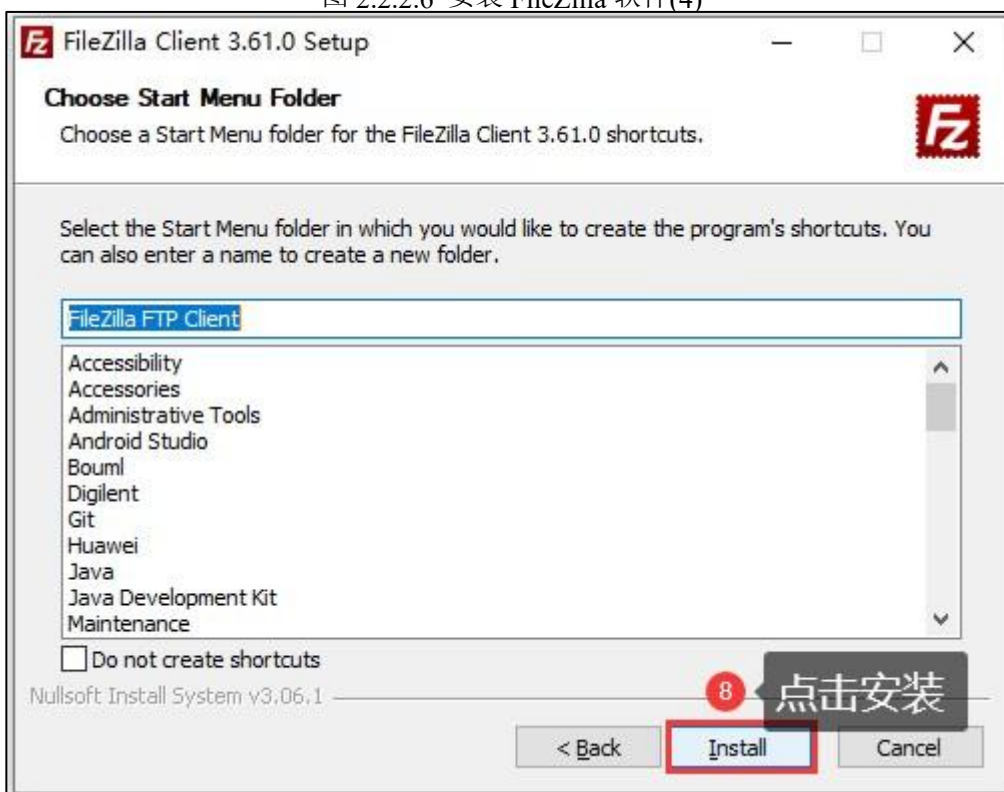


图 2.2.2.7 安装 FileZilla 软件(5)



图 2.2.2.8 安装 FileZilla 软件(6)

至此，软件安装完成，桌面会生成对应的快捷方式：



图 2.2.2.9 FileZilla 桌面图标

2.2.3 FileZilla 使用方法

接下来介绍一下 FileZilla 的使用方法，首先双击 FileZilla 桌面图标打开该软件：

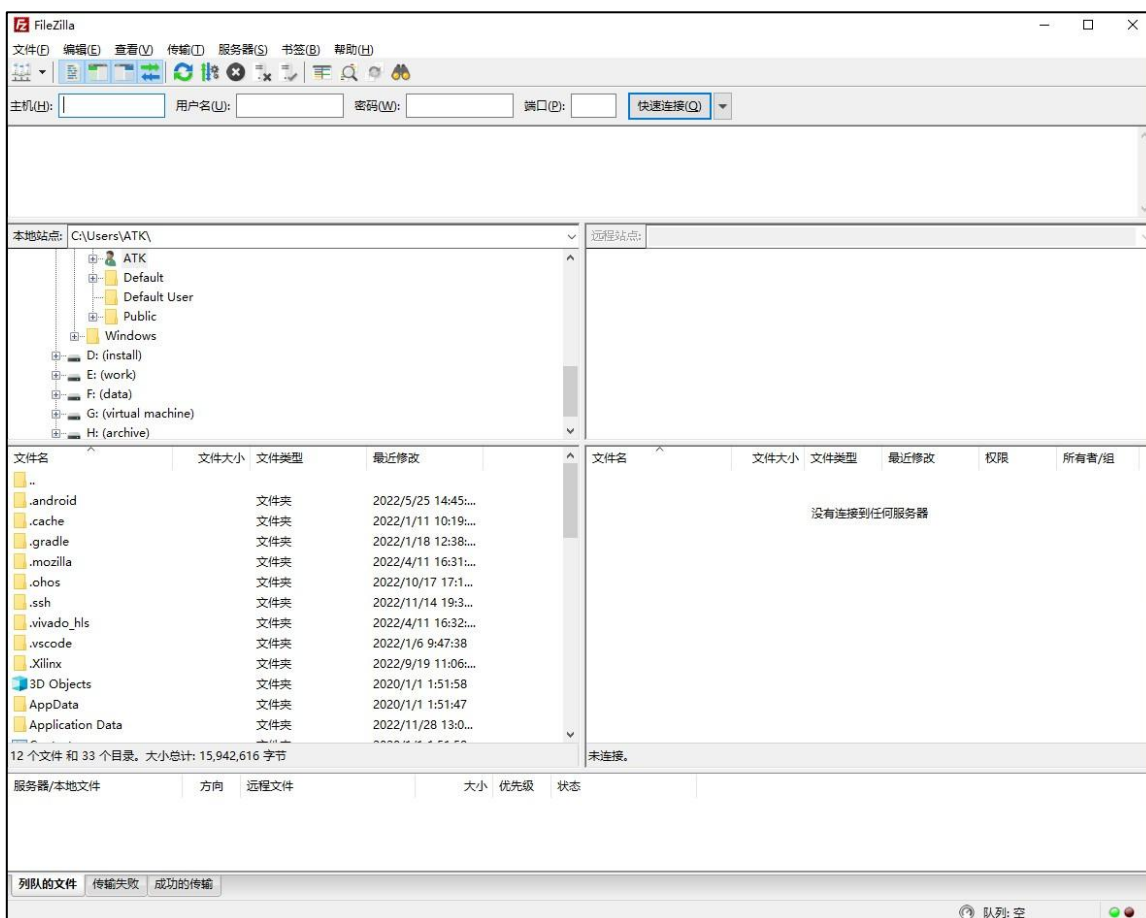


图 2.2.3.1 FileZilla 软件界面

Ubuntu 作为 FTP 服务器，Windows 作为 FTP 客户端，在进行文件传输之前，客户端需要先连接到服务器。按照图 2.2.3.2~2.2.3.6 所示步骤连接 FTP 服务器：



图 2.2.3.2 FTP 客户端连接 FTP 服务器(1)



图 2.2.3.3 FTP 客户端连接 FTP 服务器(2)



图 2.2.3.4 FTP 客户端连接 FTP 服务器(3)

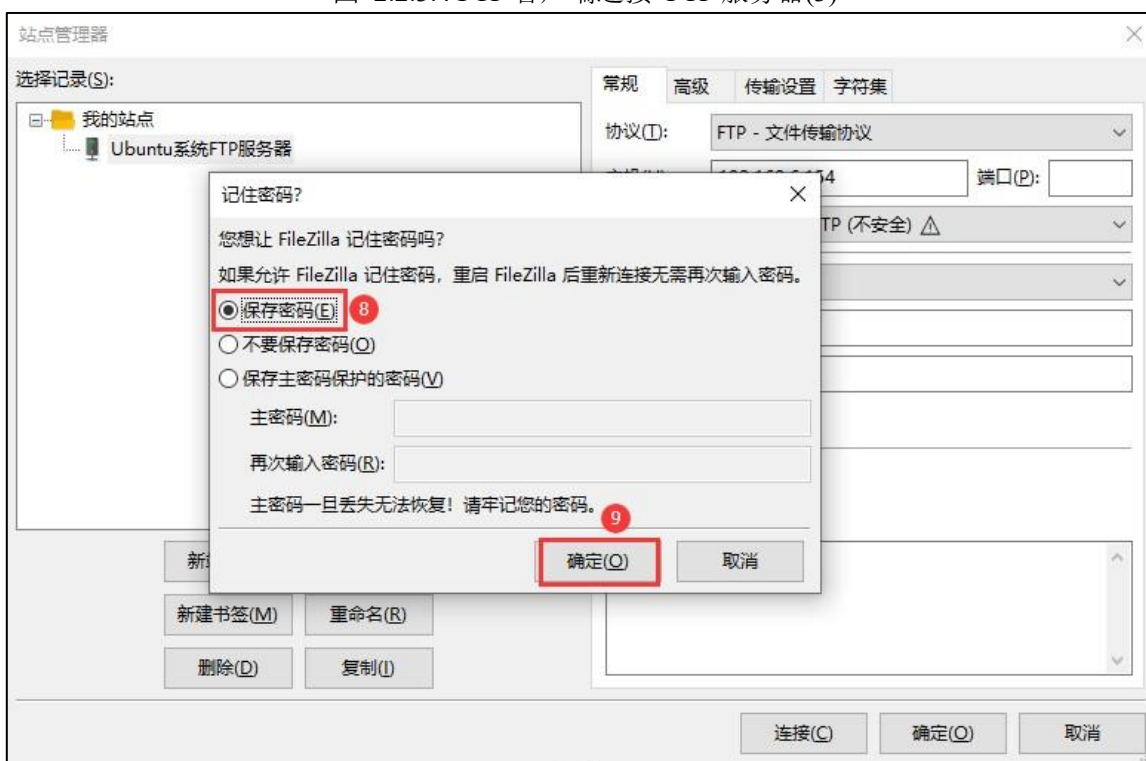


图 2.2.3.5 FTP 客户端连接 FTP 服务器(4)

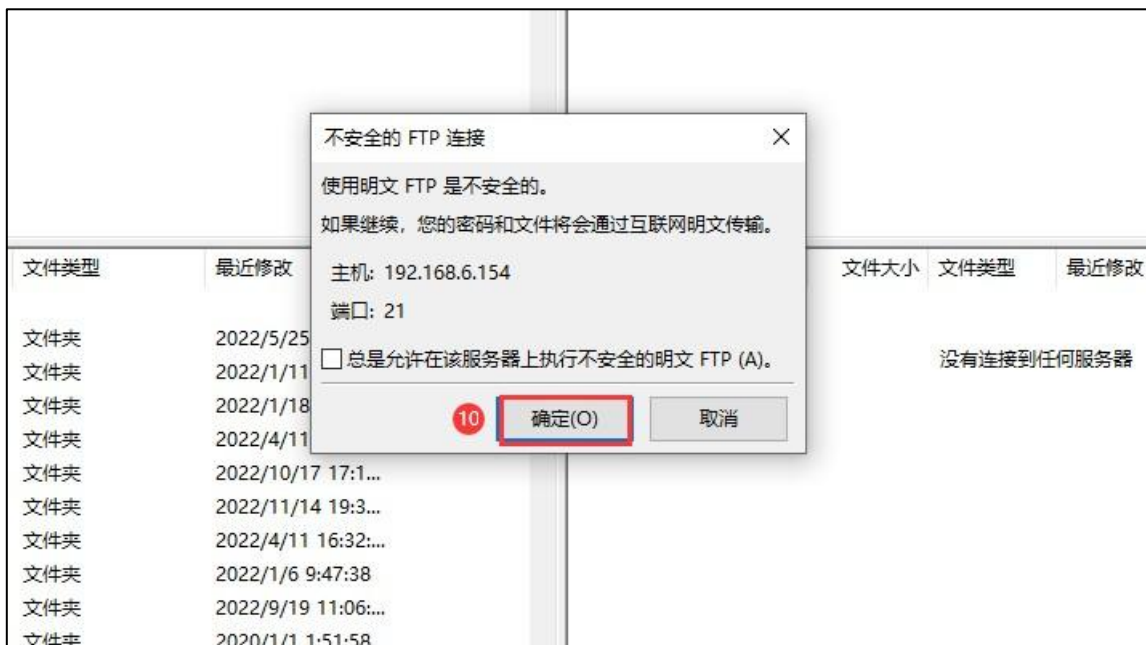


图 2.2.3.6 FTP 客户端连接 FTP 服务器(5)

连接成功如图 2.2.3.7 所示:

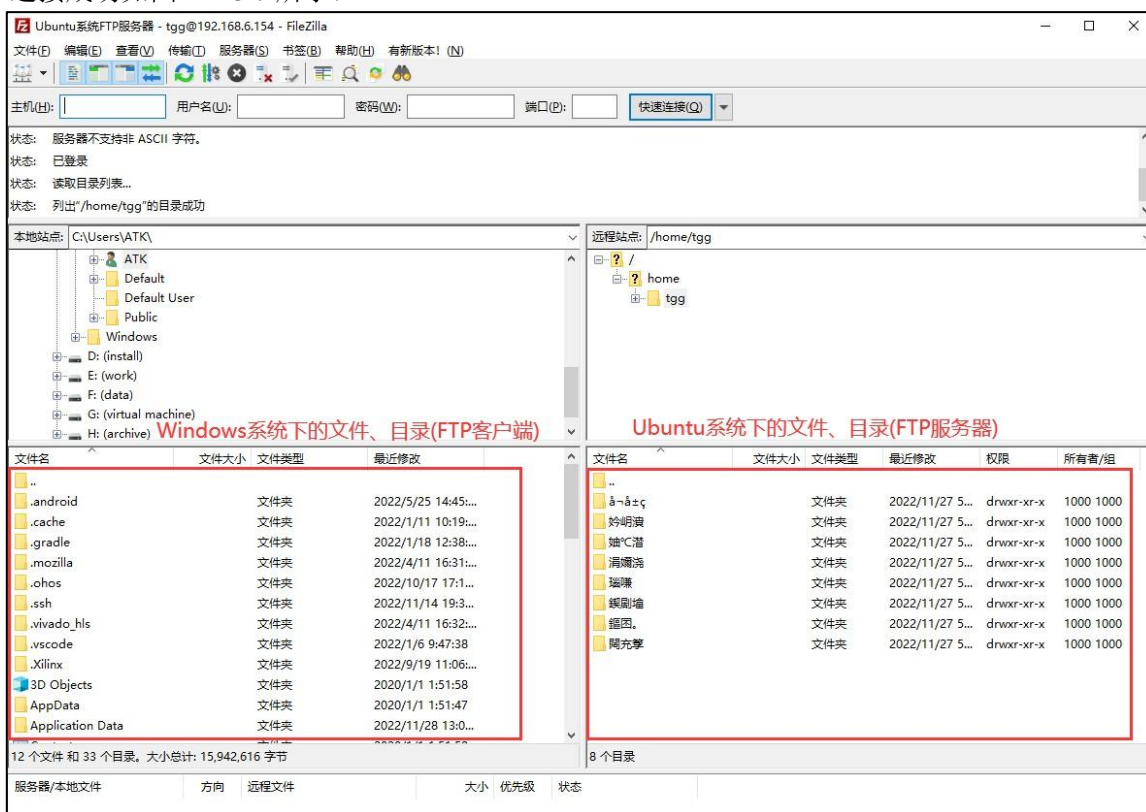


图 2.2.3.7 连接成功

其中左边是 Windows 系统下的文件、目录 (FTP 客户端), 右边是 Ubuntu 系统下的文件、目录 (FTP 服务器, 默认会进入到用户家目录, 譬如 “/home/tgg”)。从上图可知, Ubuntu 系统下的文件列表名称全是乱码, 这是因为编码的问题导致的, 我们需要修改编码方式, 再次打开 “站点管理器”, 按照图 2.2.3.8 所示步骤设置编码方式:



图 2.2.3.8 设置字符编码方式(1)



图 2.2.3.9 设置字符编码方式(2)

再次连接之后, 会发现文件名已经显示正常了, 如图 2.2.3.10 所示:

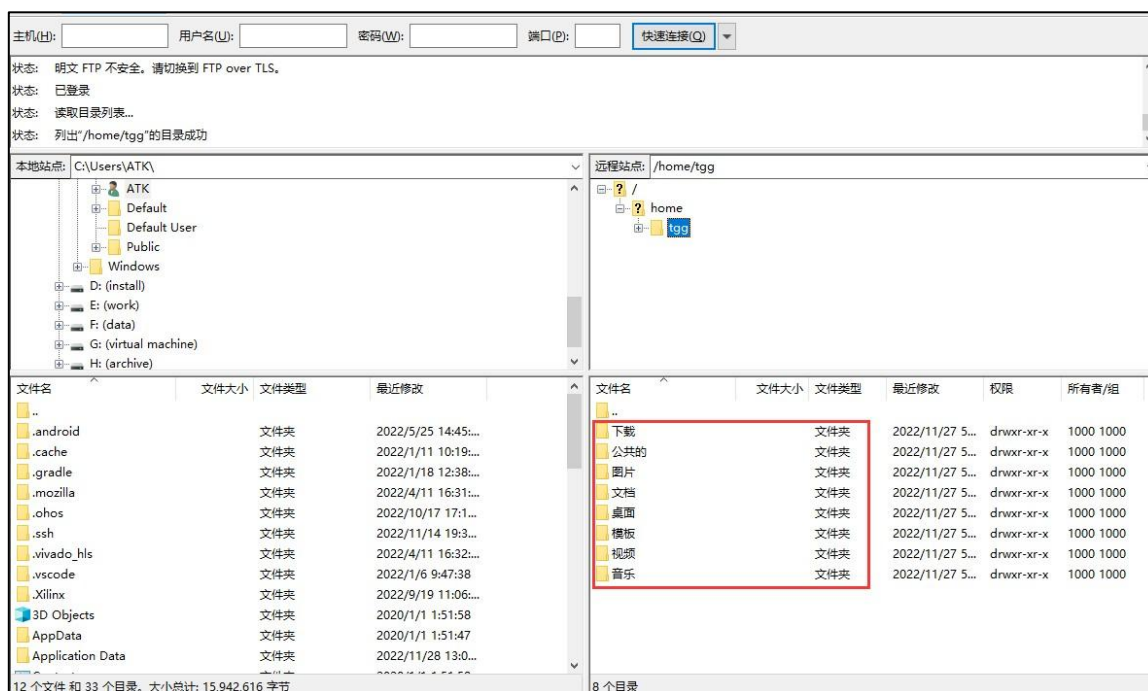


图 2.2.3.10 文件名显示正常

客户端成功连接上服务器后，便可以进行文件传输了。传输的方式也非常简单，选择要传输的文件，直接使用鼠标左键将其拖动到 Windows 目录区域或者 Ubuntu 系统目录区域即可！比如，将 Windows 系统下的 test.txt 文件拷贝到 Ubuntu 系统，首先在左侧 Windows 目录区域找到该文件 test.txt，然后使用鼠标左键将其拖动至右边 Ubuntu 目录区域释放即可；同理，Ubuntu 系统下的文件拷贝到 Windows 系统也是如此！

2.6 CH343 串口驱动安装

正点原子 ATK-DLRK3588 开发板使用了国产芯片 CH343 来实现 USB 转串口功能，可以直接通过 USB 线将开发板的调试串口连接到电脑、而无需使用 USB 转串口线，方便用户使用；但是需要在 Windows 下安装 CH343 驱动才能识别到开发板的调试串口。

开发板资料盘中已经给用户提供了 CH343 驱动安装文件，路径为：**开发板光盘 A 盘-基础资料⑦04、软件⑦CH343SER.EXE**，双击 CH343SER.EXE 可执行文件，按照图 2.6.1~2.6.2 所示步骤安装 CH343 驱动（**安装之前先不要连接开发板调试串口**）：

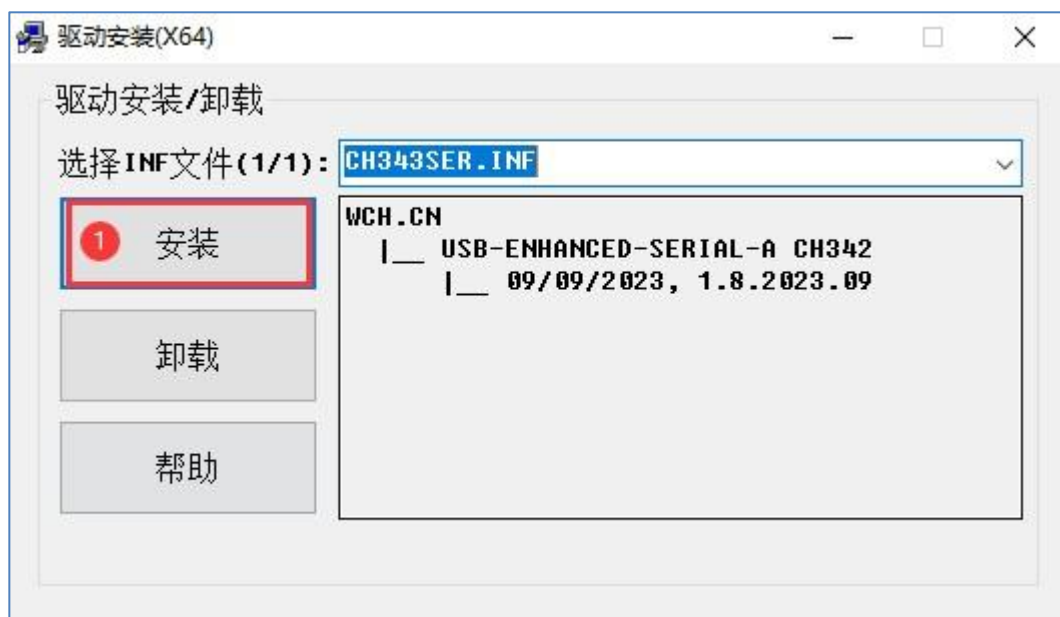


图 2.6.1 安装 CH343 驱动(1)

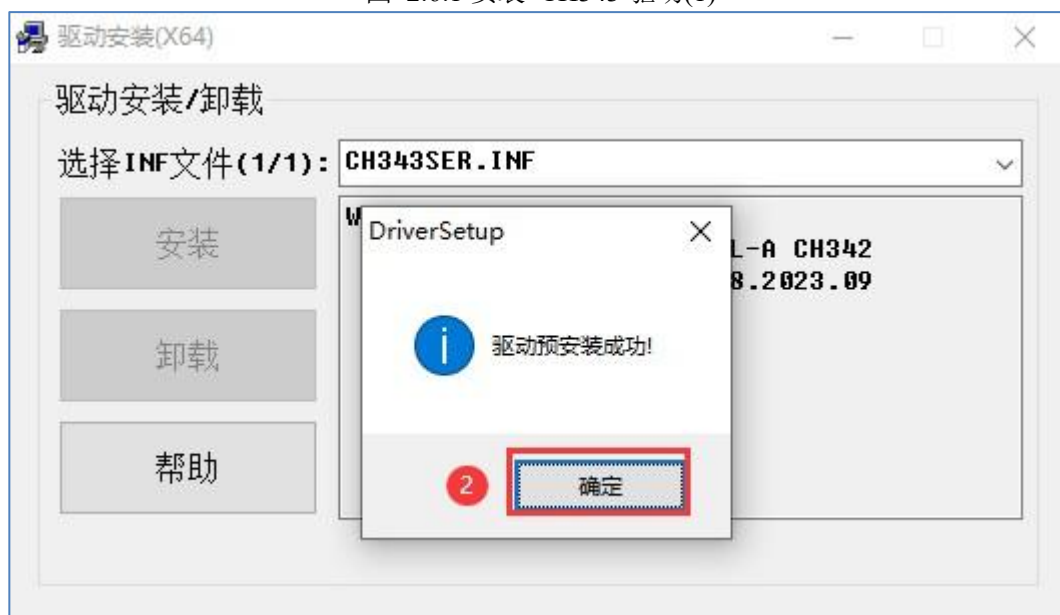


图 2.6.2 安装 CH343 驱动(2)

安装成功后，通过 USB 线将开发板调试串口与电脑相连，连接方式如图 2.6.3 所示：

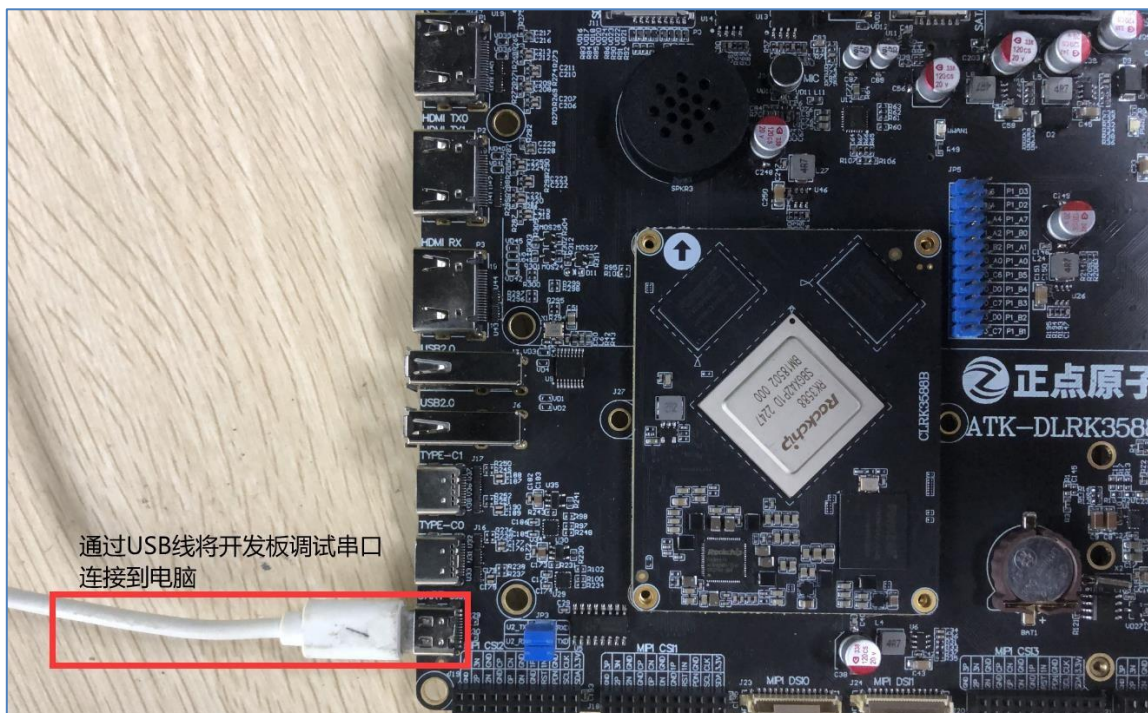


图 2.6.3 开发板调试串口与电脑相连连接成功后, 此时电脑会检测到一个 USB 设备, 打开 Windows 设备管理器:

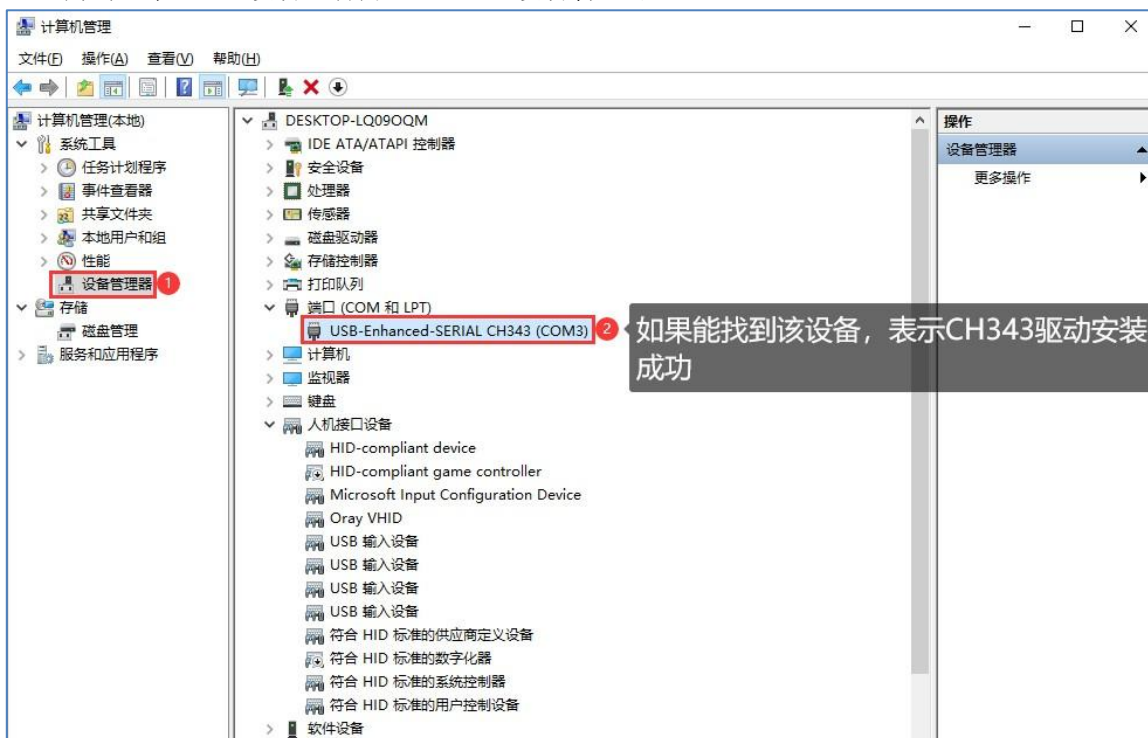


图 2.6.4 Windows 设备管理器

如果在“设备管理器①端口（COM 和 LPT）”下能找到一个名为“USB-SERIAL CH343”的设备, 则表示 CH343 驱动安装成功! 如果找不到名为“USB-SERIAL CH343”的设备, 请用户自行检查硬件连接是否有误! 可拔掉 USB 线、重新连接, 或者连接电脑的其它 USB 口试试。

2.7 MobaXterm 软件安装

MobaXterm 是一款多功能远程终端软件，功能强大、而且免费（也有收费版本），支持创建 SSH、Telnet、Rsh、Xdmc、RDP、VNC、FTP、SFTP、串口(Serial COM)等超多远程连接功能。MobaXterm 提供了人性化的操作界面，功能十分强大，所以推荐用户使用 MobaXterm 这款终端软件。

2.7.1 MobaXterm 软件下载

开发板资料盘中已经给用户提供了 MobaXterm 软件安装包，路径为：**开发板光盘 A 盘-基础资料⑦04、软件⑦MobaXterm_Installer_v12.3.zip**；

资料盘中给用户提供的安装包对应的版本为 12.3，用新的版本也行，旧的版本也可以，这个都没什么关系，这里我们以 12.3 版本为例。

2.7.2 MobaXterm 软件安装

将 MobaXterm_Installer_v12.3.zip 压缩包文件解压，解压之后如图所示：



图 2.7.2.1 MobaXterm_Installer_v12.3.zip 解压后的文件

接着双击运行 MobaXterm_Installer_v12.3.msi 文件，按照图 2.7.2.2~2.7.2.6 所示步骤安装 MobaXterm 软件：

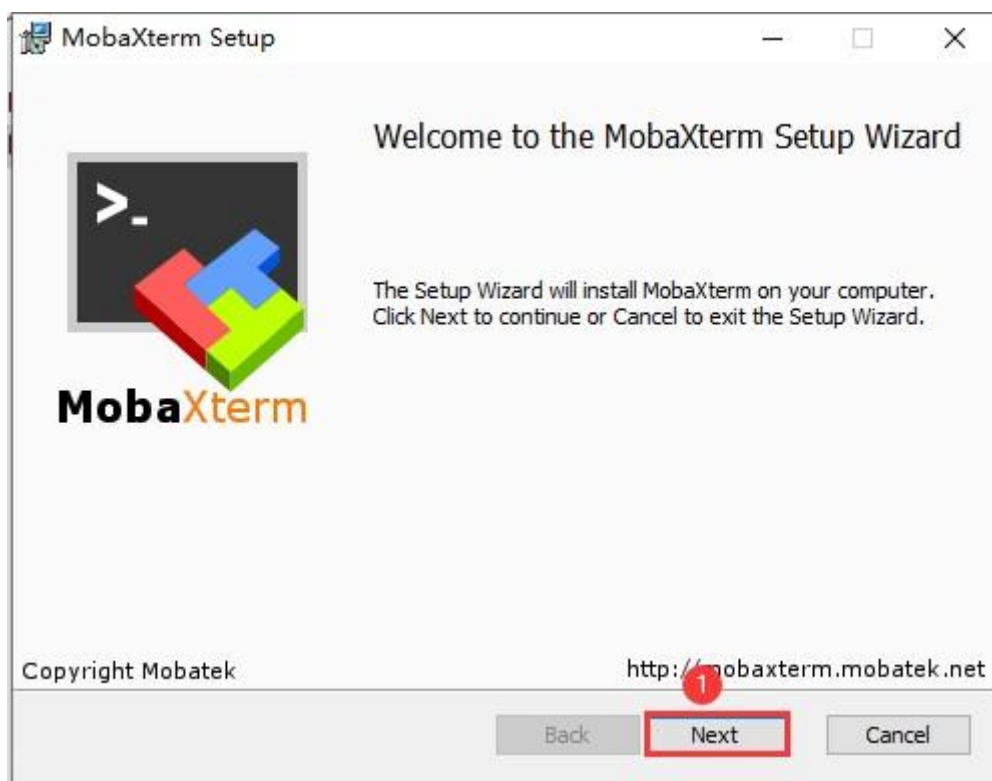


图 2.7.2.2 安装 MobaXterm 软件(1)

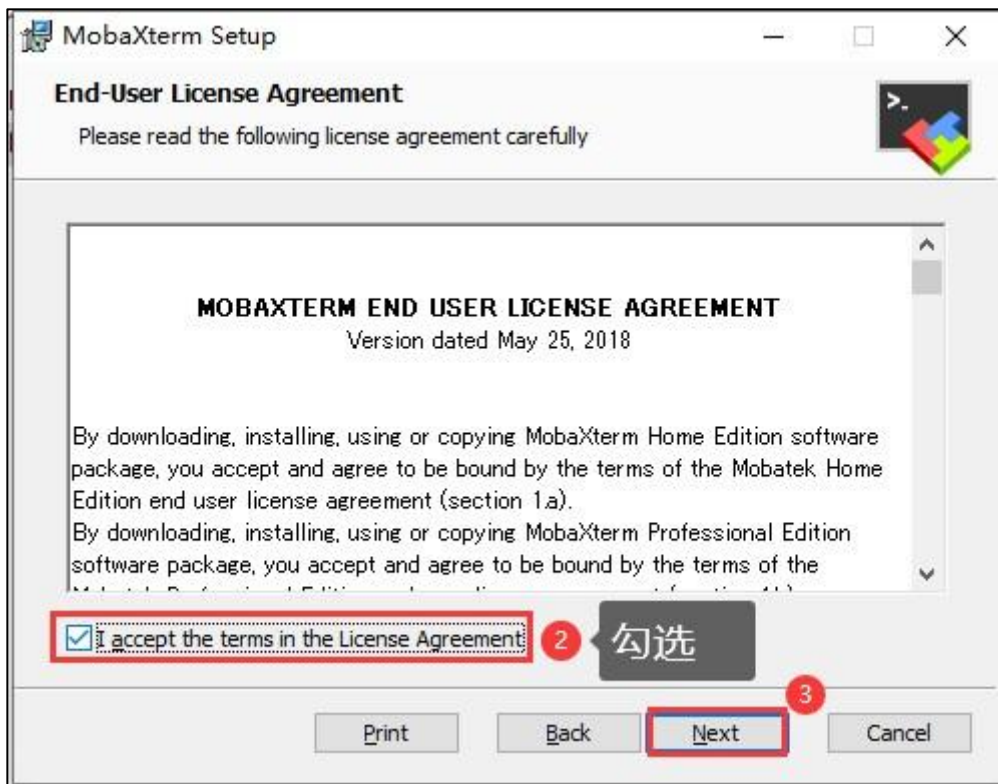


图 2.7.2.3 安装 MobaXterm 软件(2)

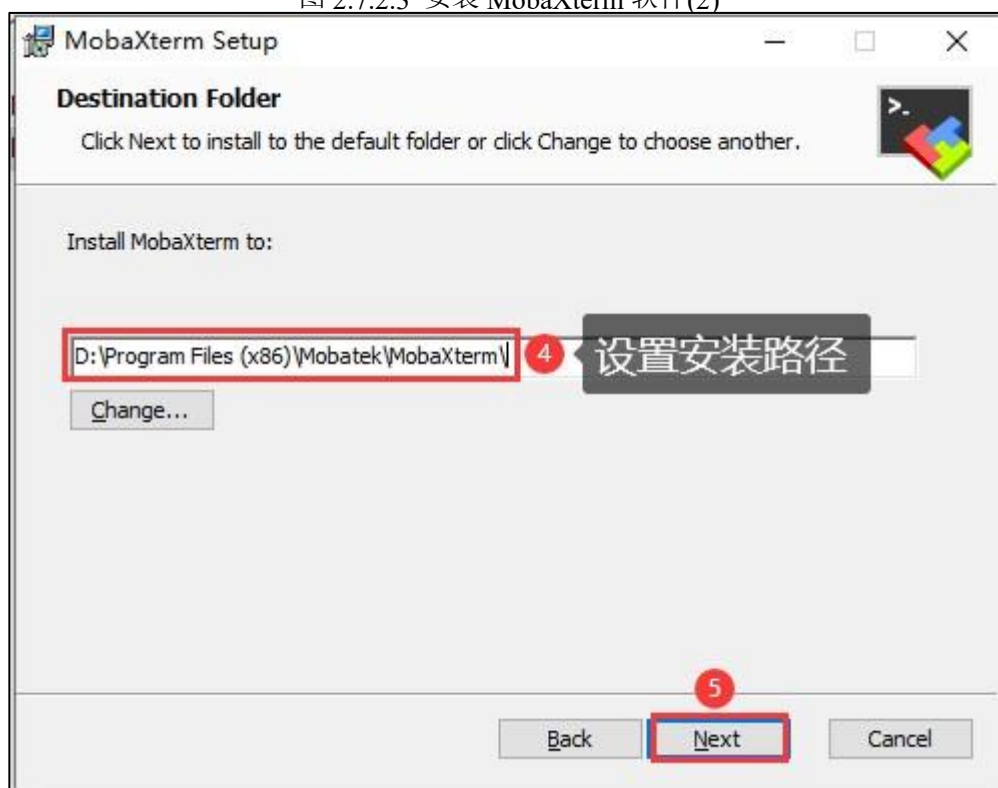


图 2.7.2.4 安装 MobaXterm 软件(3)

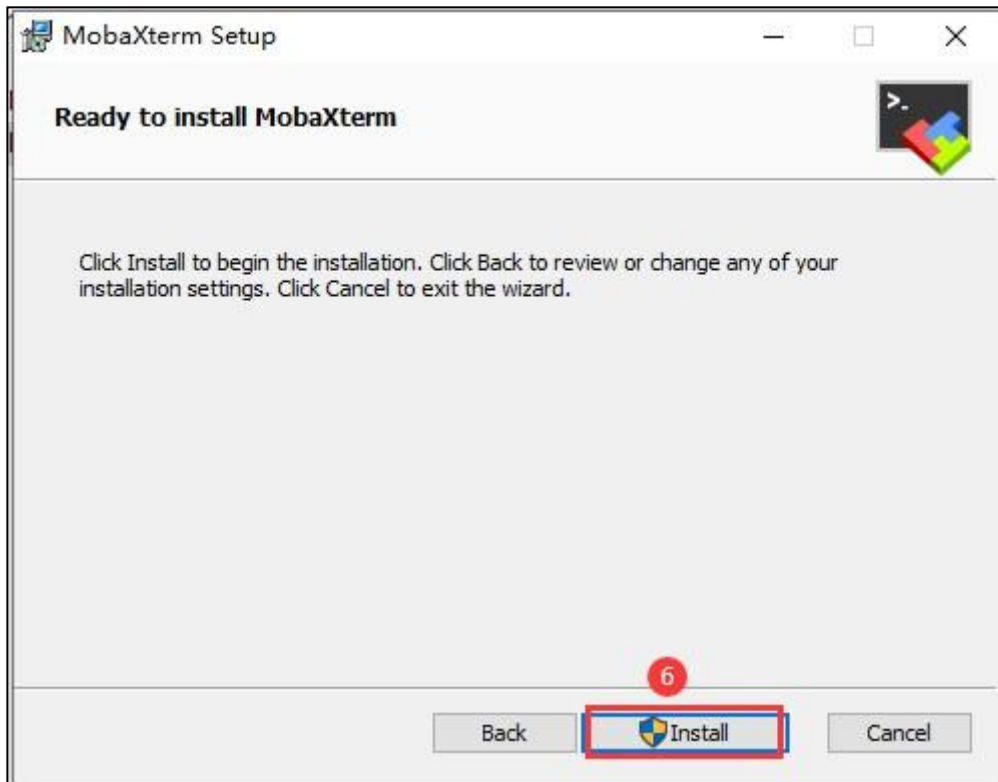


图 2.7.2.5 安装 MobaXterm 软件(4)

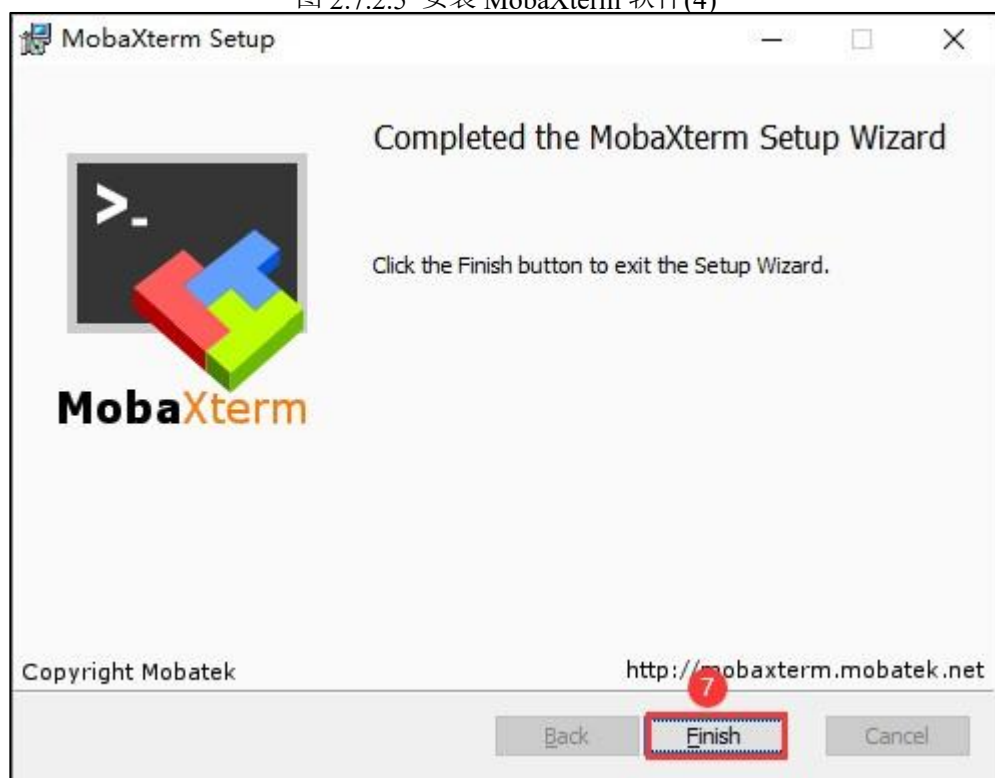


图 2.7.2.6 安装 MobaXterm 软件(5) 至此，软件安装完成，桌面会自动生成 MobaXterm 软件快捷方式图标：



图 2.7.2.7 MobaXterm 软件桌面图标

2.7.3 MobaXterm 软件的使用

双击 MobaXterm 桌面图标打开该软件，如图 2.7.3.1 所示：

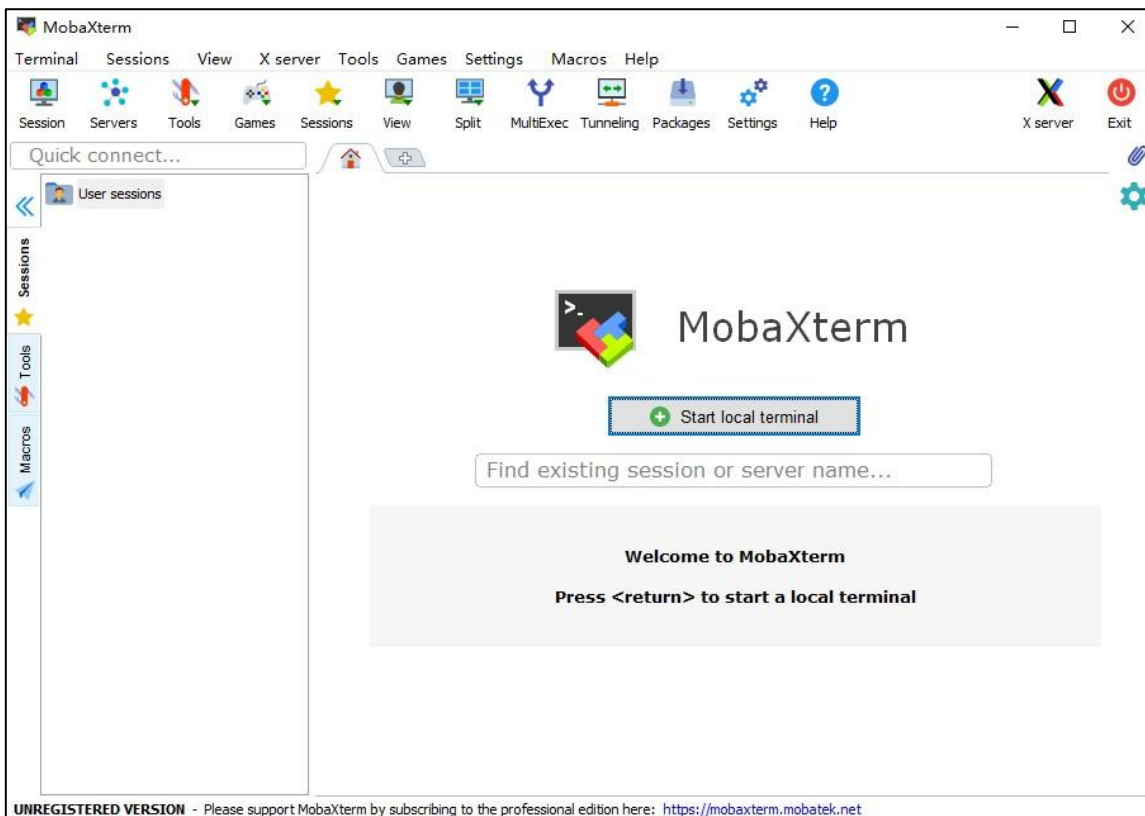


图 2.7.3.1 MobaXterm 软件主界面

接下来向大家介绍如何建立 Serial（串口）连接以及 ssh 远程连接。

1. 串口连接

ATK-DLRK3588 开发板上有一个调试串口，可直接通过 USB 线将其连接到电脑。串口作为嵌入式设备最为常见的通信接口之一，不但能实现计算机与嵌入式设备之间的数据传输，而且还能实现计算机对嵌入式设备的控制，嵌入式开发过程中，通常将其作为调试接口（串口调试），用于调试嵌入式设备。

按照图 2.7.3.2~2.7.3.3 所示操作步骤建立一个 Serial（串口）连接（在建立连接之前，需要通过 USB 线将开发板的调试串口与电脑相连、并且已经安装了 CH343 驱动）：

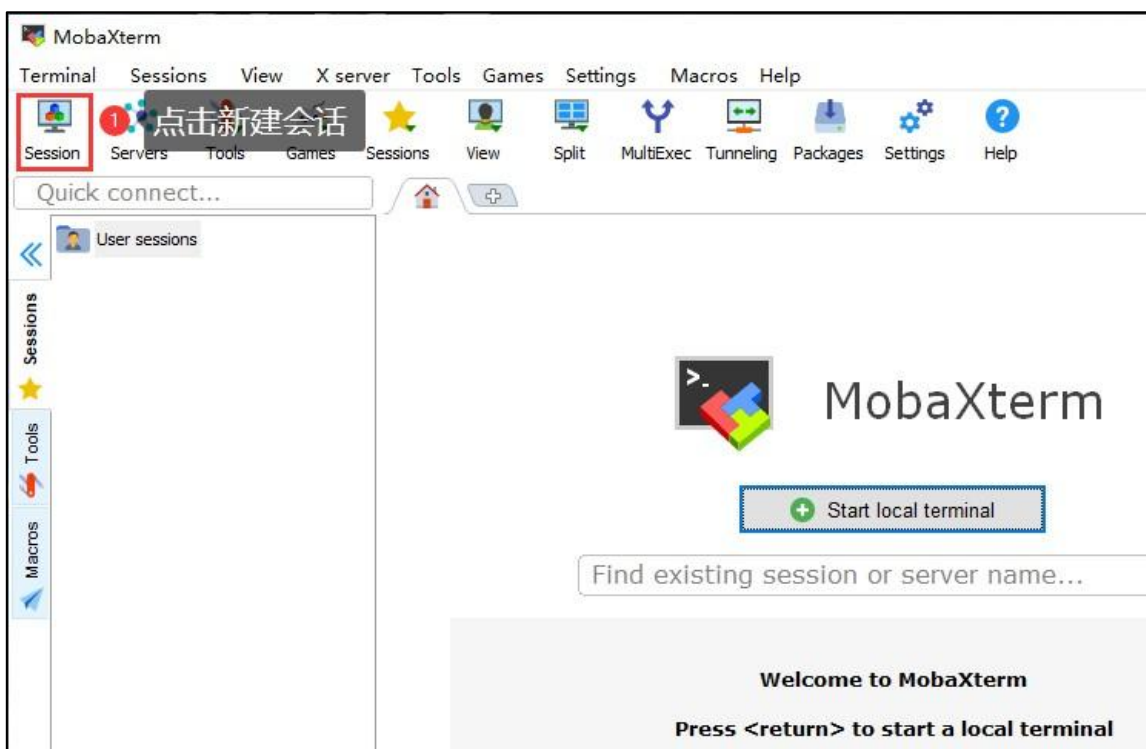


图 2.7.3.2 建立串口连接(1)

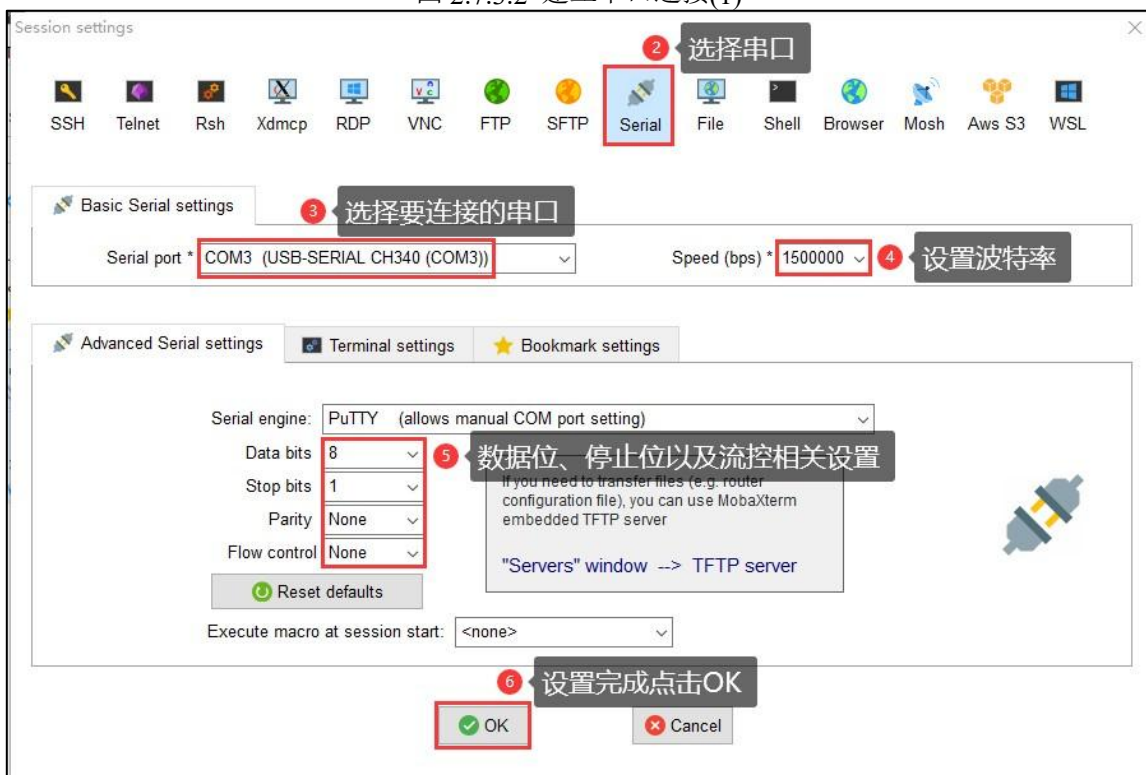


图 2.7.3.3 建立串口连接(2)

首先选择需要进行连接的串口，确保开发板的调试串口与电脑已经通过 USB 线相连、并且 CH343 驱动已经安装成功（“设备管理器”端口（COM 和 LPT）”下能找到一个名为“USB SERIAL CH343”的设备），那么 MobaXterm 软件才能识别到开发板的调试串口，我们

便可以在“Serial port”下拉列表中找到开发板对应的串口（**USB-SERIAL CH343**），然后选择它即可！接着设置串口通信波特率，根据实际情况进行设置，RK3588 平台的调试串口，其默认波特率为 1500000；然后设置数据位、停止位以及流控等，设置完成后点击“OK”按钮。

Serial 连接就建立成功了，如图 2.7.3.4 所示：

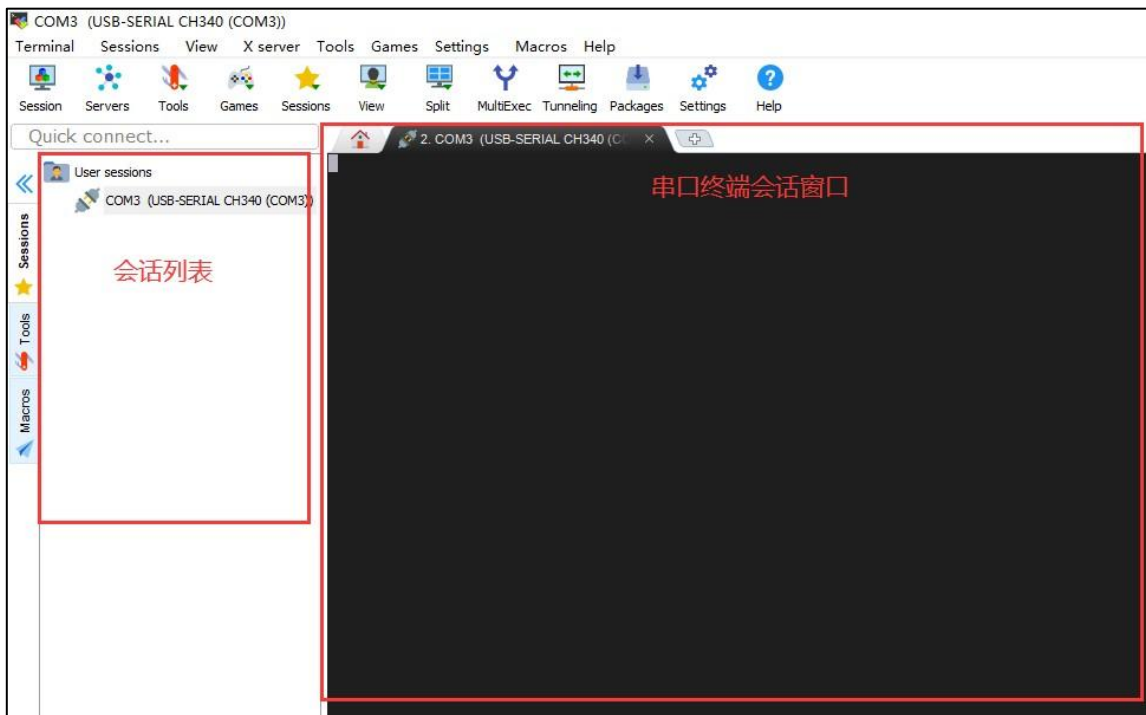


图 2.7.3.4 建立串口连接(3)

开发板上电启动，运行 Linux 系统或 Android 系统，系统启动过程中的 log 信息将会在串口终端会话窗口显示出来，如图 2.7.3.5 所示：


```

vpl adjust cursor plane from 0 to 1
vp0, plane_mask:0x2a, primary-id:5, curser-id:1
vpl adjust cursor plane from 1 to 0
vp1, plane_mask:0x15, primary-id:4, curser-id:0
vp2, plane_mask:0x0, primary-id:0, curser-id:-1
Adding bank: 0x00200000 - 0x08400000 (size: 0x08200000)
Adding bank: 0x00400000 - 0x80000000 (size: 0x76c00000)
Total: 1255.933 ms

Starting kernel ...

[ 0.000000] Booting Linux on physical CPU 0x0000000000 [0x412fd050]
[ 0.000000] Linux version 4.19.232 (alientek@android) (gcc version 6.3.1 20170404 (Linaro GCC 6.3-2017.05), GNU ld (Linaro_Binutils-201
[ 0.000000] Machine model: Rockchip RK3568 ATK EVB1 DDR4 V10 Board
[ 0.000000] earlycon: uart8250 at MMIO32 0x00000000fe60000 (options '')
[ 0.000000] bootconsole [uart8250] enabled
[ 0.000000] cma: Reserved 16 MiB at 0x000000007ec00000
[ 0.000000] psci: probing for conduit method from DT.
[ 0.000000] psci: PSCIv1.1 detected in firmware.
[ 0.000000] psci: Using standard PSCI v0.2 function IDs
[ 0.000000] psci: Trusted OS migration not required
[ 0.000000] psci: SMC Calling Convention v1.2
[ 0.000000] percpu: Embedded 23 pages/cpu s54184 r8192 d31832 u94208
[ 0.000000] Detected VIPT I-cache on CPU0
[ 0.000000] CPU features: detected: Virtualization Host Extensions
[ 0.000000] CPU features: detected: Speculative Store Bypassing Safe (SSBS)
[ 0.000000] Built 1 zonelists, mobility grouping on. Total pages: 511496
[ 0.000000] Kernel command line: storagemedia=emmc androidboot.storagemedia=emmc androidboot.mode=normal androidboot.verifiedbootstate
000
[ 0.000000] Dentry cache hash table entries: 262144 (order: 9, 2097152 bytes)
[ 0.000000] Inode-cache hash table entries: 131072 (order: 8, 1048576 bytes)
[ 0.000000] mem auto-init: stack:off, heap alloc:off, heap free:off
[ 0.000000] Memory: 1993376K/2078720K available (13566K kernel code, 1970K rwdata, 4840K rodata, 1536K init, 514K bss, 68960K reserved,
[ 0.000000] SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=4, Nodes=1
[ 0.000000] ftrace: allocating 49744 entries in 195 pages
[ 0.000000] rcu: Hierarchical RCU implementation.
[ 0.000000] rcu: RCU event tracing is enabled.
[ 0.000000] rcu: RCU restricting CPUs from NR_CPUS=8 to nr_cpu_ids=4.
[ 0.000000] rcu: Adjusting geometry for rcu_fanout_leaf=16, nr_cpu_ids=4
[ 0.000000] NR_IRQS: 64, nr_irqs: 64, preallocated irqs: 0
[ 0.000000] GICv3: GIC: Using split EOI/Deactivate mode
[ 0.000000] GICv3: Distributor has no Range Selector support
[ 0.000000] GICv3: no VPI support, no direct LPI support
[ 0.000000] ITS [mem 0xfd40000-0xfd4ffff]
[ 0.000000] ITS@0x0000000fd440000: allocated 8192 Devices @7e4f0000 (indirect, esz 8, psz 64K, shr 0)
[ 0.000000] ITS@0x0000000fd440000: allocated 32768 Interrupt Collections @7e500000 (flat, esz 2, psz 64K, shr 0)
[ 0.000000] ITS: using cache flushing for cmd queue
[ 0.000000] GIC: using LPI property table @0x000000007e510000

```

图 2.7.3.5 系统启动 log 信息

系统启动成功后，用户可以通过串口终端执行命令、命令执行结果也会通过串口终端显示出来。

2.8 Rockchip USB 驱动安装

在开发调试阶段，经常需要将设备切换至 **Loader** 模式或者 **Maskrom** 模式，譬如使用瑞芯微开发工具（RKDevTool）烧录镜像时（需连接 USB 线），设备需要工作在 Loader 模式或 Maskrom 模式，此时才可进行烧写；需要在 Windows 下安装 Rockchip USB 驱动才能识别到设备。开发板资料盘中已经给用户提供了 Rockchip USB 驱动安装包，路径为：**开发板光盘 A 盘基础资料 005、开发工具 RKTools\windows\DriverAssitant_v5.12.zip**，支持 xp、win7_32、win7_64、win10_32、win10_64 等操作系统；将 DriverAssitant_v5.12.zip 压缩文件解压，解压之

后得到如图 2.8.1 所示文件、目录列表：

名称	修改日期	类型	大小
ADBDriver	2020/11/10 14:13	文件夹	
bin	2020/11/10 14:14	文件夹	
Driver	2022/2/28 14:14	文件夹	
config.ini	2014/6/3 15:38	配置设置	1 KB
DriverInstall.exe	2022/2/28 14:11	应用程序	491 KB
Readme.txt	2018/1/31 17:44	文本文档	1 KB
revision.log	2022/2/28 14:14	文本文档	1 KB

图 2.8.1 DriverAssitant_v5.12.zip 解压

直接双击 DriverInstall.exe 文件，按照图 2.8.2~2.8.4 所示步骤安装 Rockchip USB 驱动：

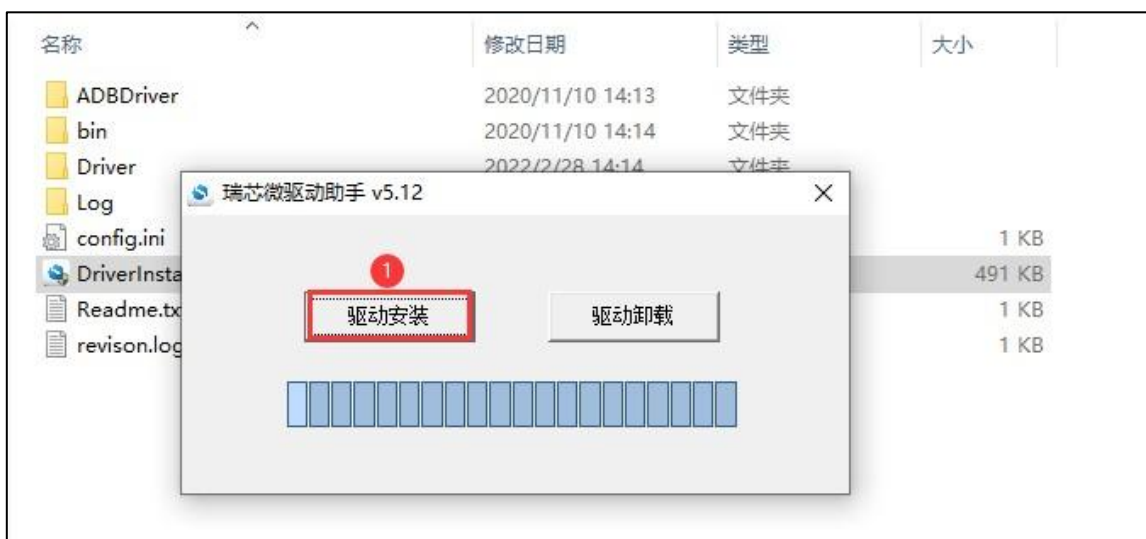


图 2.8.2 安装 Rockchip USB 驱动(1)

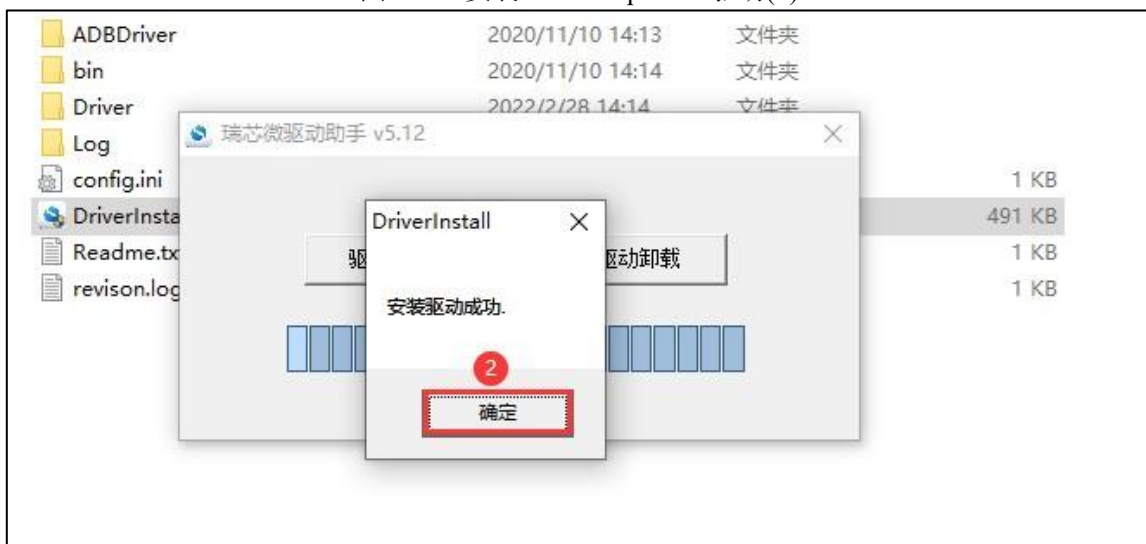


图 2.8.3 安装 Rockchip USB 驱动(2)

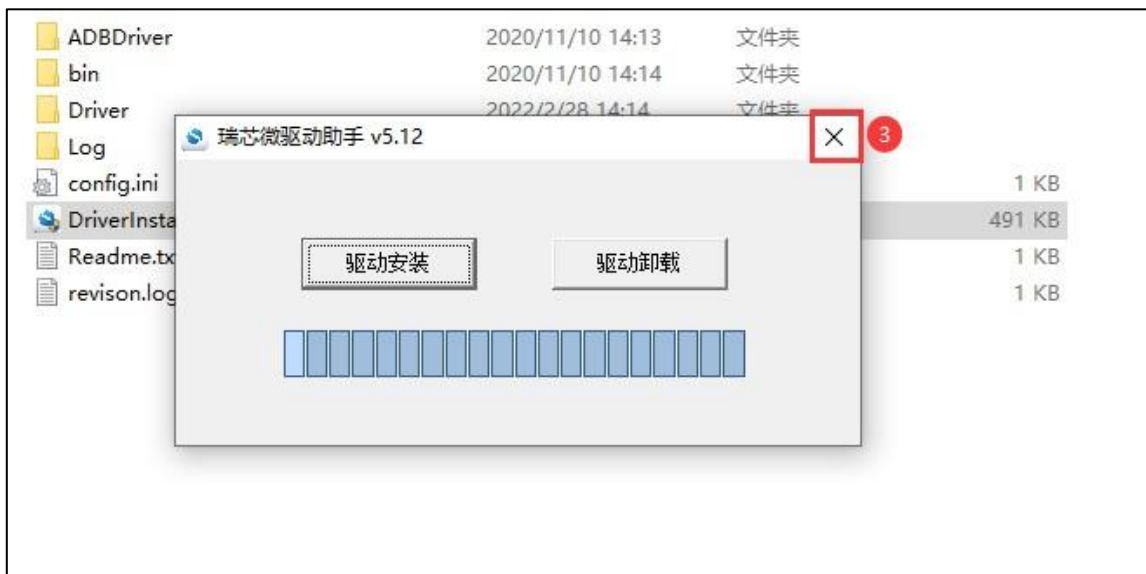


图 2.8.4 安装 Rockchip USB 驱动(3)

2.9 镜像烧录工具的使用

Rockchip 平台提供了多种镜像烧写方式，譬如在 **Windows** 下通过 **RKDevTool**（瑞芯微开发工具）工具烧写、通过 **SD 卡** 方式烧写、通过 **FactoryTool** 工具批量烧写（量产烧写工具、支持 **USB 一拖多** 烧写）以及在 **Ubuntu** 系统下通过 **Linux_Upgrade_Tool** 工具烧写等等，总之，烧写镜像的方式有很多种，用户可以选择合适的烧写方式进行烧写，将镜像文件烧写至开发板。本小节简单介绍下 **Windows** 下的烧写工具 **RKDevTool** 以及 **Linux** 下的烧写工具 **Linux_Upgrade_Tool**，在我们开发调试过程中，这两个应该是使用最多的烧写工具。

2.9.1 烧写模式介绍

Rockchip 平台硬件设备运行的几种模式如下表所示，只有当设备处于 **Maskrom** 以及 **Loader** 模式时（必须要通过 **USB 线** 将开发板的 **TYPE-C0** 口连接到电脑），才能够烧写镜像，或对板上镜像进行更新操作。

模式	是否支持烧录	描述
Maskrom	支持	Flash 在未烧录镜像时，芯片会引导进入 Maskrom 模式，可以进行初次镜像的烧写；开发调试过程中若遇到无法
		正常进入 Loader 模式的情况，也可进入 Maskrom 模式烧写镜像。
Loader	支持	Loader 模式下可以进行镜像烧写、更新升级，可以通过烧写工具单独烧写某一个分区镜像文件，方便调试。
Recovery	不支持	系统引导 recovery 启动，主要作用是升级、恢复出厂设置类操作。
Normal Boot	不支持	系统正常启动后进入的模式，通过引导 rootfs 启动，加载 rootfs ，大多数的开发都是在这个模式下调式的。

表 2.9.1.1 Rockchip 平台运行模式说明

进入 **Maskrom** 烧写模式的方法：（先连接电源适配器和 **TYPE-C0**）

- 开发板未烧录过镜像，上电之后就会进入到 Maskrom 模式；
- 开发板烧录过镜像，按住开发板上的 **UPDATE 按键**，然后开发板上电或复位，系统将会进入到 Maskrom 模式；
- 开发板烧录过镜像，在 U-Boot 命令行下，执行“**rbrom**”命令进入到 Maskrom 模式。

进入 Loader 烧写模式的方法：（先连接电源适配器和 TYPE-C0）

- 开发板烧录过镜像，按住 **V+按键**（音量+）按键，然后开发板上电或复位，系统将会进入到 Loader 模式；
- 开发板烧录过镜像，在 U-Boot 命令行下，执行“**download**”命令进入到 Loader 模式；
- 开发板烧录过镜像，在 Linux 系统下，通过串口或 ADB 执行命令“**reboot loader**”重启进入到 Loader 模式；
- 开发板烧录过镜像，上电或复位后开发板正常启动进入到系统，瑞芯微开发工具（也就是 RKDevTool 工具）会显示“发现一个 ADB 设备”，然后点击“**切换**”按钮，进入到 Loader 模式，如下图。

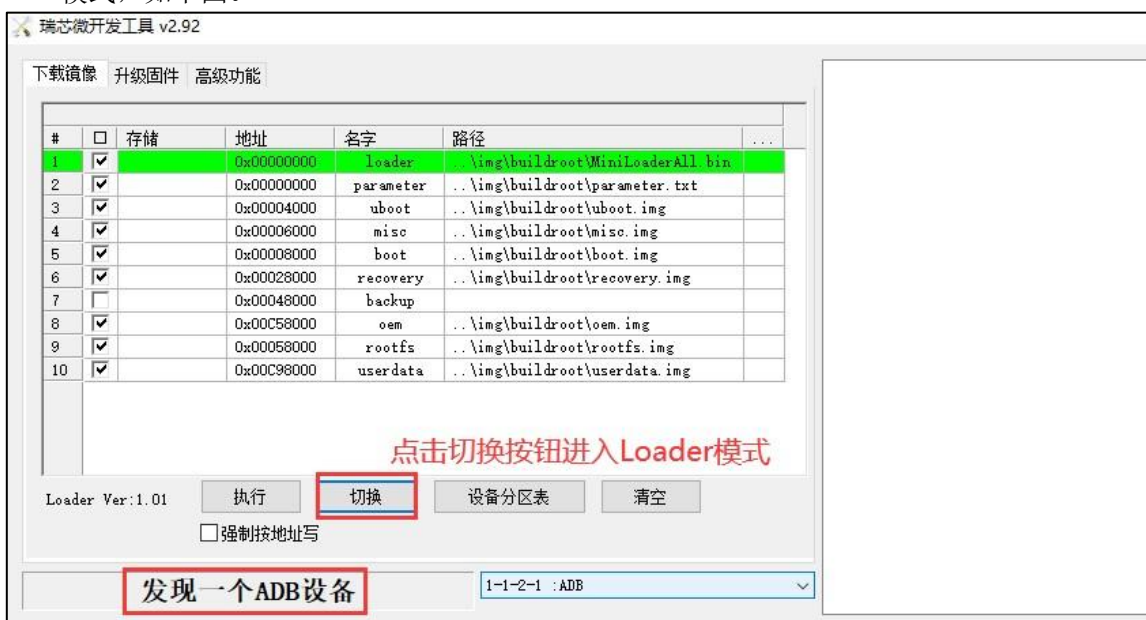


图 2.9.1.1 通过瑞芯微开发工具进入 Loader 模式 以上给大家介绍了开发板如何进入 Maskrom 模式或 Loader 模式，只有进入 Maskrom 模式或 Loader 模式后才可以进行烧录。

2.9.2 Windows 下 RKDevTool 工具的使用

在 Windows 下烧写镜像使用 RKDevTool 工具，RK 的文档中一般也将其称为瑞芯微开发工具，该工具除了用于烧录镜像之外，还有其它的一些功能，是我们开发、调试过程中最常用的工具。

正点原子 ATK-DLRK3588 开发板资料盘中已经给用户提供了该工具，路径为：**开发板光盘 A 盘-基础资料⑦05、开发工具⑦RKTools⑦windows⑦RKDevTool_Release_v2.92.zip**，

首先将 RKDevTool_Release_v2.92.zip 压缩文件解压开来，解压之后的内容如下所示：

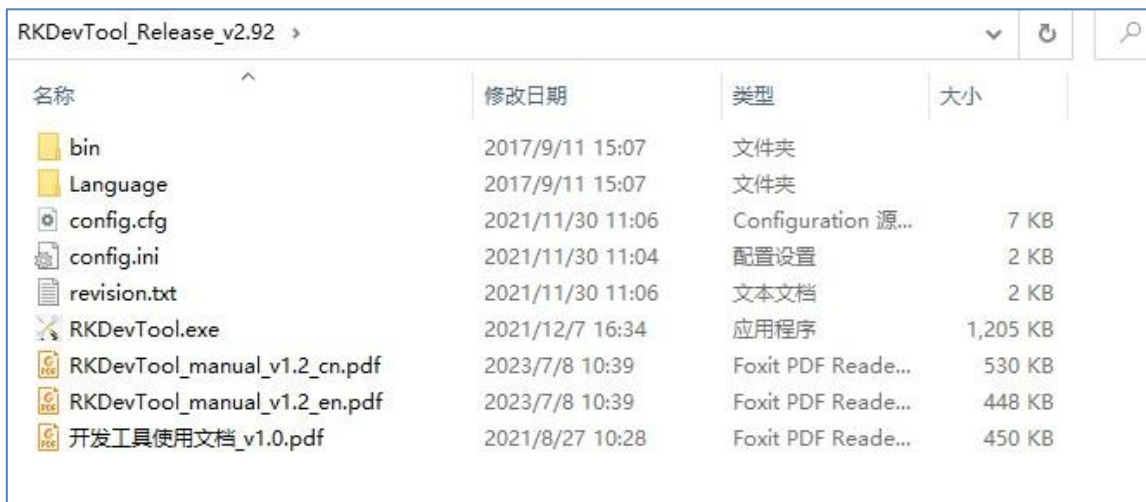


图 2.9.2.1 RKDevTool 目录

该目录下有两份瑞芯微官方提供的文档:《**开发工具使用文档_v1.0.pdf**》、

《**RKDevTool_manual_v1.2_cn.pdf**》, 向用户详细介绍了如何使用 RKDevTool 工具(瑞芯微开发工具), 该工具的详细使用方法大家可以参考这两份文档, 我们这里只是做一个简单的介绍。双击 **RKDevTool.exe** 可执行文件打开瑞星微开发工具, 如下图所示:



图 2.9.2.4 瑞星微开发工具界面

上图便是瑞芯微开发工具的一个主界面,

烧录完整 **update.img** 固件。update.img 并非编译得到, 它其实是由多个镜像打包而成,

包括 boot.img、MiniLoaderAll.bin、uboot.img、misc.img、rootfs.img、recovery.img 等, 是这些分立镜像的集合体; 通过 RK 提供的工具可将各个分立镜像打包成一个 update.img。

执行烧写操作之前, 开发板必须处于 **Maskrom** 模式或 **Loader** 模式, 客户拿到手的开发板默认都是烧录过镜像的, 可以进入到 **Loader** 模式, 当然也可以进入到 **Maskrom** 模式。首先开发板需要先连接电源适配器以及 **TYPE-C0** 口, 如下图所示:

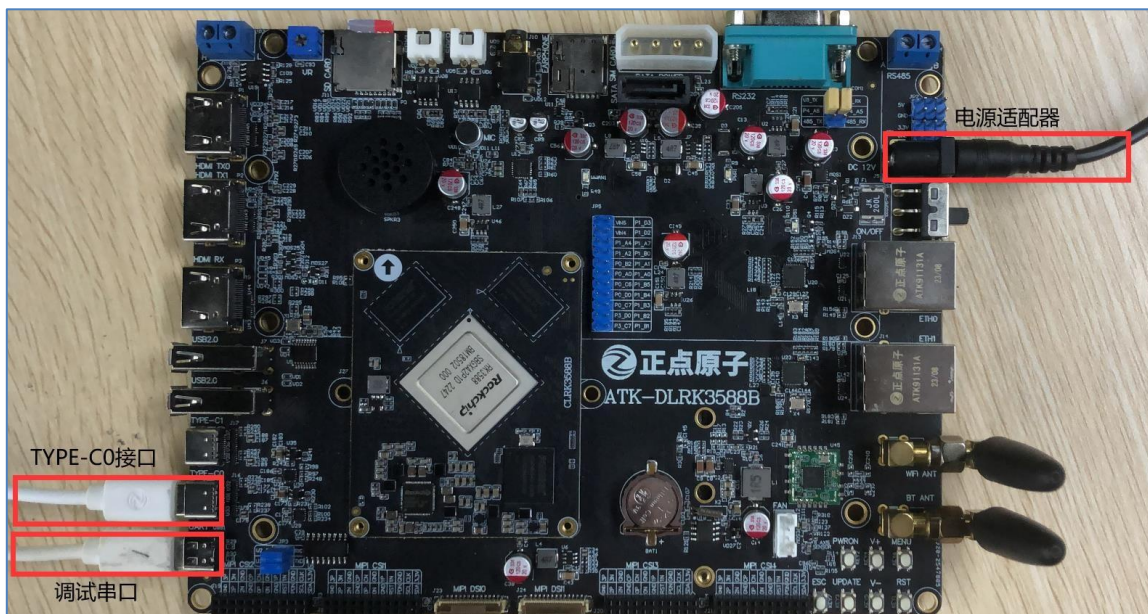


图 2.9.2.10 开发板硬件连接示意图 电源适配器使用 ATK-DLRK3588 开发板配套的 12V-2.5A 电源适配器；通过 USB 线将开发板的调试串口 UART 连接到电脑（先安装好 CH343 驱动），同样使用 USB 线将开发板的 TYPE-C0 接口连接到电脑。

硬件连接好之后，按住开发板上的 **V+**（音量+）按键，然后给开发板上电或复位，系统将会进入到 Loader 模式；此时瑞芯微开发工具便会显示“**发现一个 LOADER 设备**”：

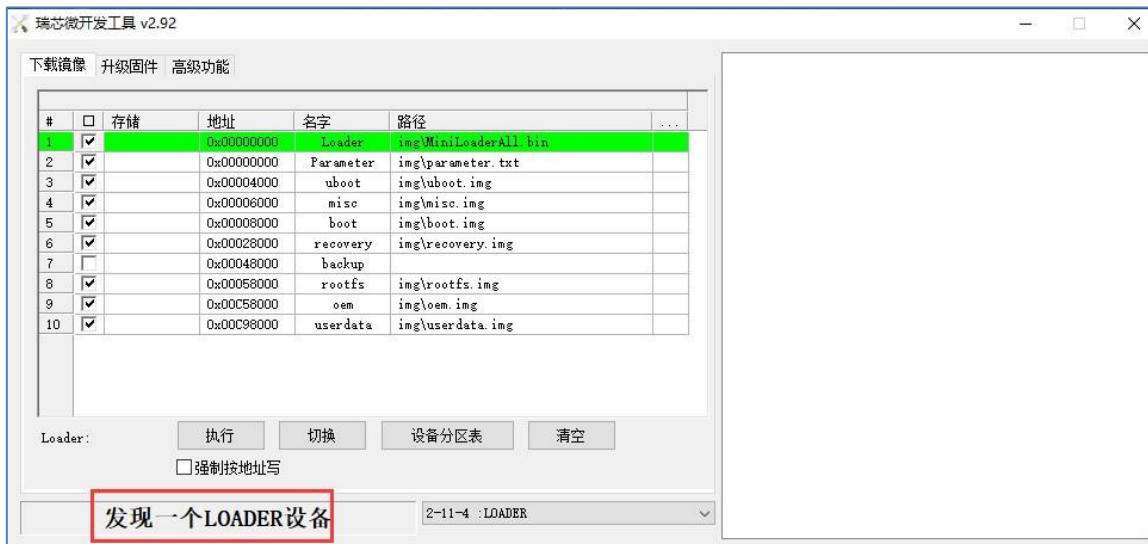


图 2.9.2.11 瑞芯微开发工具检测到 loader 设备

2.9.3 update.img 镜像的烧录方法

update.img 是多个镜像的集合体（由多个镜像打包合并而成），使用 RK 提供的工具可以将各个分立镜像（uboot.img、boot.img、MiniLoaderAll.bin、parameter.txt、misc.img、rootfs.img、oem.img、userdata.img、recovery.img 等）打包成一个 update.img 固件，方便用户烧录、升级。本小节介绍如何使用瑞芯微开发工具烧写 update.img，首先打开瑞芯微开发工具，选择“**升级固件**”：



图 2.9.3.1 升级固件选项卡

然后点击“**固件**”按钮可以选择我们需要进行升级、烧录的 update.img 固件（开发板资料盘中并未给用户提供 update.img 固件，这里只是演示烧写 update.img 固件的方法），加载镜像之后会显示出该固件的一些信息：

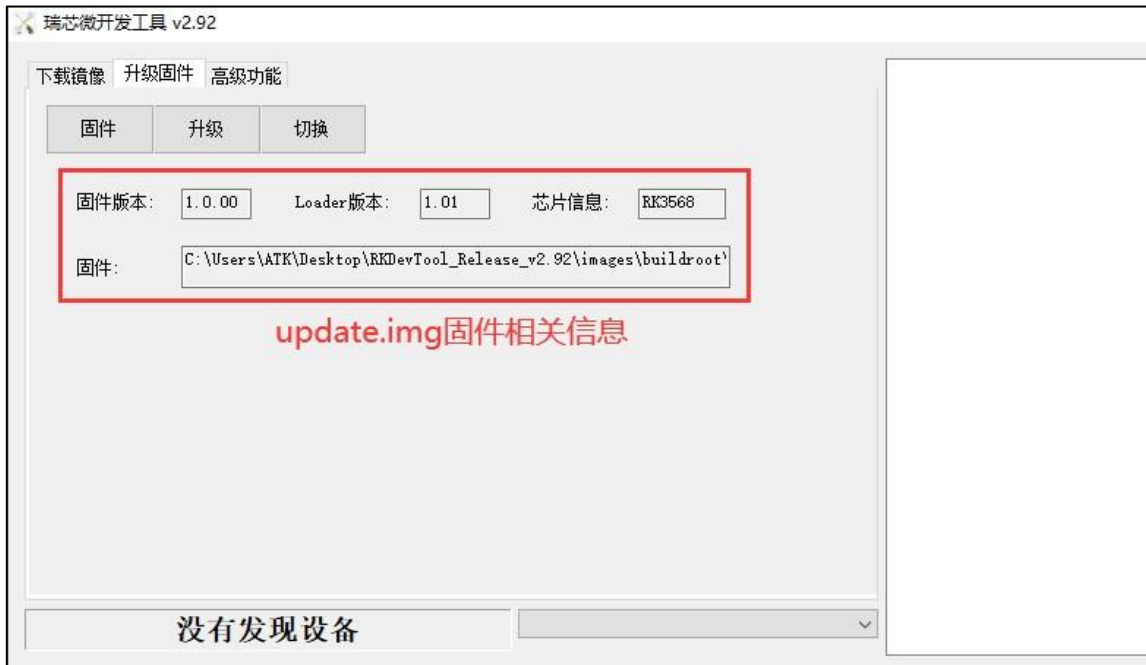


图 2.9.3.2 导入 update.img 镜像

首先让设备进入 Maskrom 或 Loader 模式，然后点击“**升级**”按钮进行固件升级、更新：

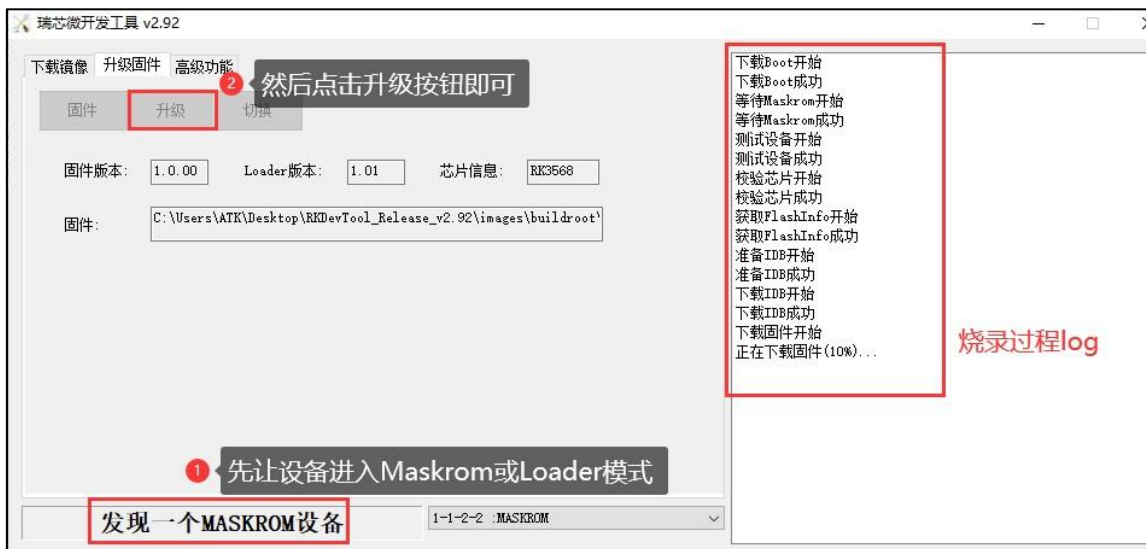


图 2.9.3.3 update.img 升级
固件烧录完成后，会自动重启开发板。

如果不幸，烧写过程中出现了错误、导致烧写失败，那么大家可以参考 RK 官方提供的文档《[RKDevTool_manual_v1.2_cn.pdf](#)》，该文档中有针对几种常见的错误进行说明。

需要说明的是，adb 虽然是 Android 设备的调试工具，但随着 adb 的普及，不仅仅是 Android 设备，在嵌入式开发中，很多 Linux 设备也同样支持 adb 调试，例如 Rockchip 平台。

向用户介绍如何在 Windows 系统和 Ubuntu 系统下安装 adb 工具。

本章将分为如下几个小节：

1.2 Windows 下安装 adb 工具

1.3 Ubuntu 下安装 adb 工具

1.2 Windows 下安装 adb 工具

开发板资料盘中已经给用户提供了 adb 工具的安装包，路径为：[开发板光盘 A 盘-基础资料 005、开发工具 05adb 调试工具 05platform-tools_r33.0.3-windows.zip](#)，将 zip 压缩文件解压到 Windows 系统某个目录下，譬如 **D:**，解压得到一个名为“platform-tools”的文件夹，如图

1.2.1 所示:

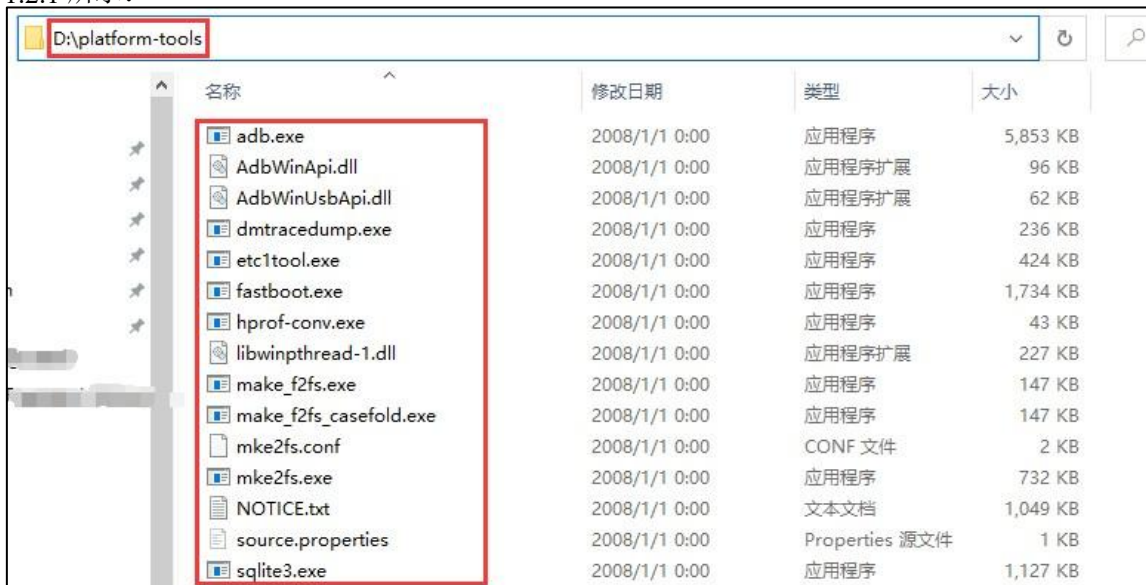


图 1.2.1 解压 platform-tools_r33.0.3-windows.zip

下来需要将 platform-tools 目录（也就是 adb 工具的安装目录）添加到 Windows 系统 Path 环境变量中。首先，在 Windows 桌面右键单击“此电脑”图标，然后选择“属性”，并按照图 1.2.3~1.2.9 所示步骤设置 Path 环境变量：



图 1.2.3 设置 Path 环境变量(1)

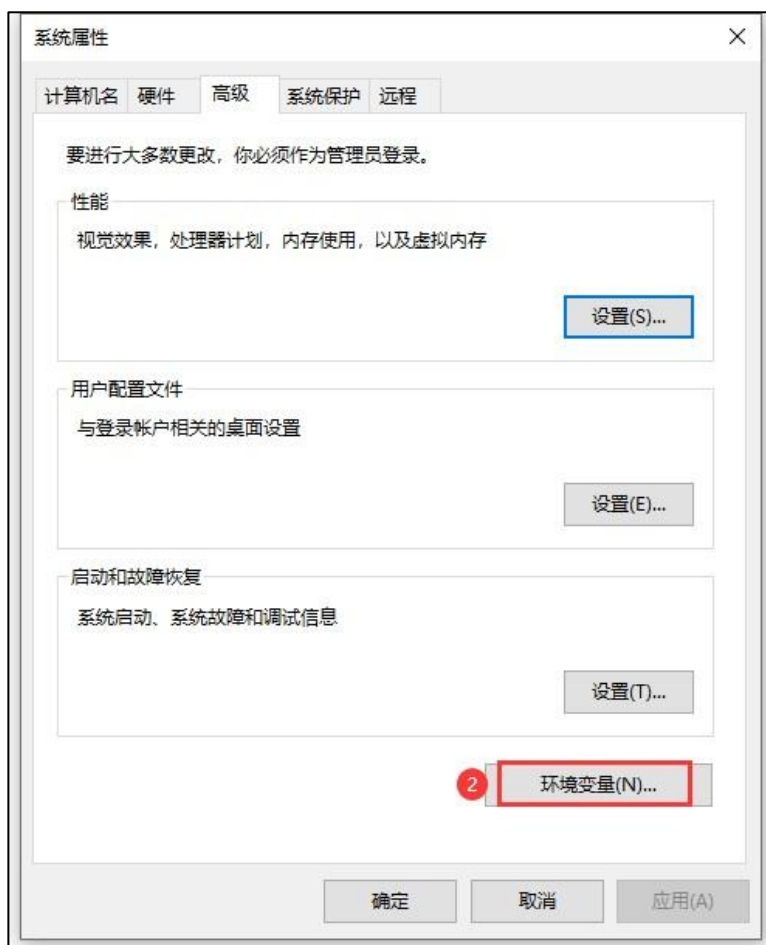


图 1.2.4 设置 Path 环境变量(2)

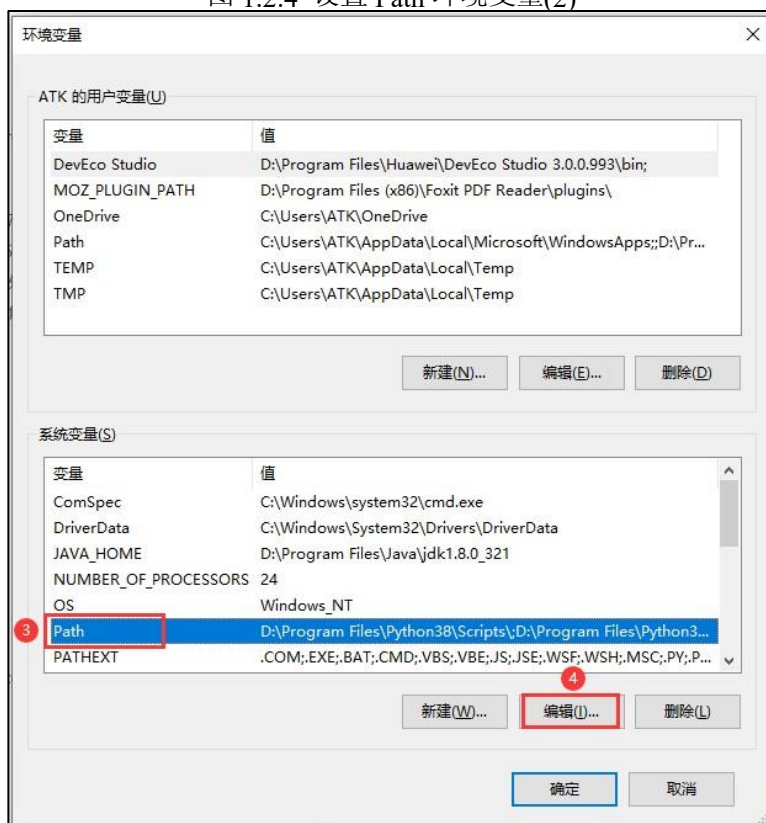


图 1.2.5 设置 Path 环境变量(3)

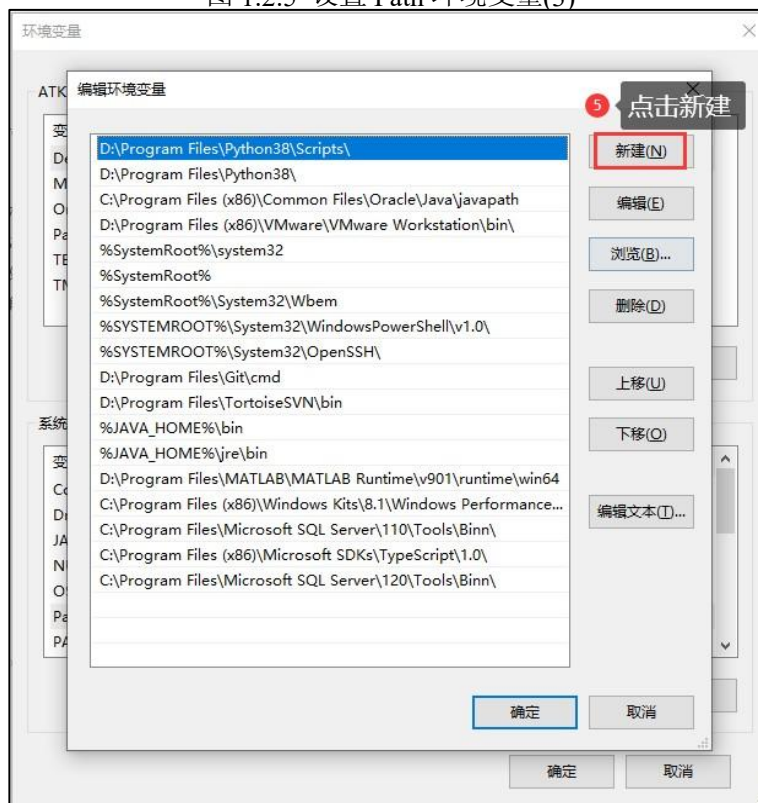


图 1.2.6 设置 Path 环境变量(4)

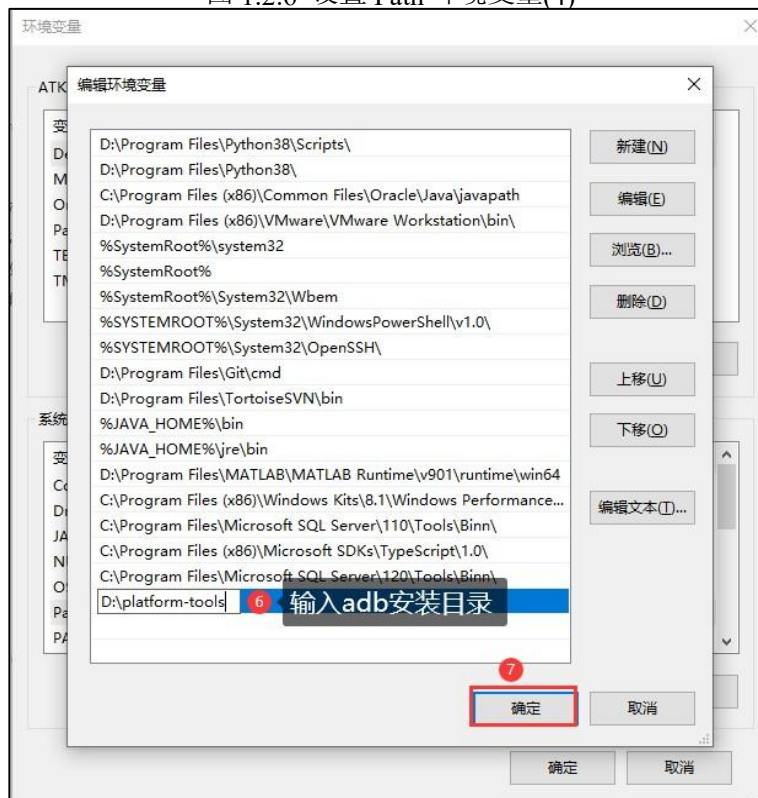


图 1.2.7 设置 Path 环境变量(5)

接下来再点击两次“确定”按钮退出窗口:

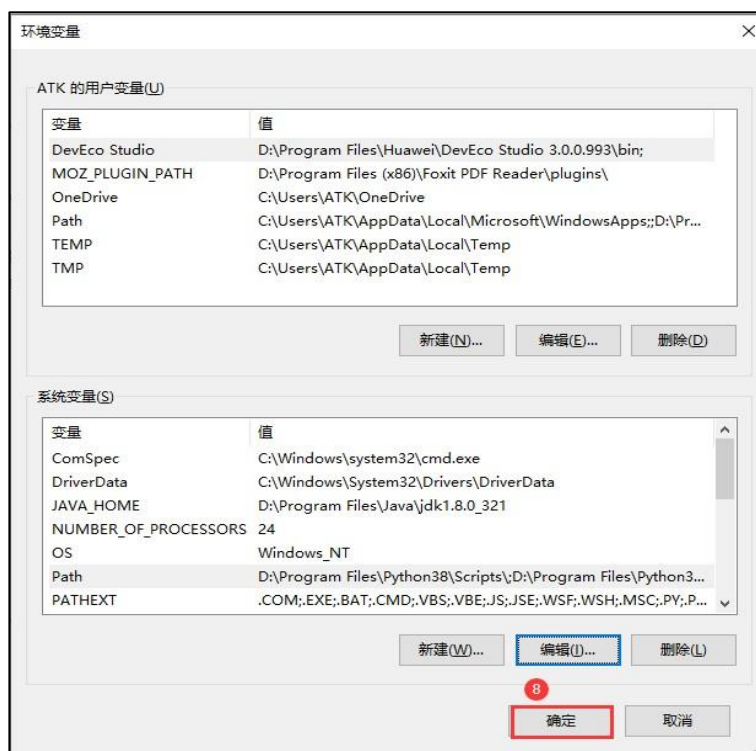


图 1.2.8 设置 Path 环境变量(6)



图 1.2.9 设置 Path 环境变量(7)

至此, adb 安装完成。接下来进行测试, 按住“Win + R”组合键打开 Windows 运行窗口, 输入“cmd”按回车键打开 Windows 命令行窗口, 如图 1.2.10 所示:



图 1.2.10 Windows 运行窗口



图 1.2.11 Windows cmd 命令行窗口

执行如下命令, 确认 adb 安装是否成功:

adb version

命令执行结果如图 1.2.12 所示:



图 1.2.12 查询 adb 版本

如果能够成功获取到 adb 版本信息, 则表示 adb 安装成功!

1.3 Ubuntu 下安装 adb 工具

接下来向用户介绍如何在 Ubuntu 系统下安装 adb 工具。开发板资料盘中已经给用户提供了 Linux 系统环境下的 adb 工具安装包，路径为：开发板光盘 A 盘-基础资料⑦05、开发工具⑦adb 调试工具⑦platform-tools_r33.0.3-linux.zip，首先打开 Ubuntu 系统的命令行终端，执行如下命令在用户家目录下创建一个名为“tools”的目录：

```
mkdir ~/tools
```

将 **platform-tools_r33.0.3-linux.zip** 压缩文件拷贝到 tools 目录下，并执行如下命令将其解压到 tools 目录：

```
cd ~/tools/
```

```
unzip platform-tools_r33.0.3-linux.zip
```

```
tgg@tgg-virtual-machine:~$ cd ~/tools/
tgg@tgg-virtual-machine:~/tools$
tgg@tgg-virtual-machine:~/tools$ ls
platform-tools_r33.0.3-linux.zip
tgg@tgg-virtual-machine:~/tools$
tgg@tgg-virtual-machine:~/tools$ unzip platform-tools_r33.0.3-linux.zip
Archive: platform-tools_r33.0.3-linux.zip
  inflating: platform-tools/NOTICE.txt
  inflating: platform-tools/adb
  inflating: platform-tools/dmtracedump
  inflating: platform-tools/e2fsdroid
  inflating: platform-tools/etc1tool
  inflating: platform-tools/fastboot
  inflating: platform-tools/hprof-conv
  inflating: platform-tools/make_f2fs
  inflating: platform-tools/make_f2fs_casefold
  inflating: platform-tools/mke2fs
  inflating: platform-tools/mke2fs.conf
  inflating: platform-tools/sload_f2fs
  extracting: platform-tools/source.properties
  inflating: platform-tools/sqlite3
  inflating: platform-tools/lib64/libc++.so
tgg@tgg-virtual-machine:~/tools$
tgg@tgg-virtual-machine:~/tools$ ls
platform-tools  platform-tools_r33.0.3-linux.zip
tgg@tgg-virtual-machine:~/tools$
```

图 1.3.1 解压 platform-tools_r33.0.3-linux.zip

解压完成得到 platform-tools 文件夹，其内容如图 1.3.2 所示：

```
tgg@tgg-virtual-machine:~/tools$ cd platform-tools/
tgg@tgg-virtual-machine:~/tools/platform-tools$
tgg@tgg-virtual-machine:~/tools/platform-tools$ ls -l
总用量 17652
-rwxr-xr-x 1 tgg tgg 8104656 1月 1 2008 adb
-rwxr-xr-x 1 tgg tgg 56824 1月 1 2008 dmtracedump
-rwxr-xr-x 1 tgg tgg 1645440 1月 1 2008 e2fsdroid
-rwxr-xr-x 1 tgg tgg 305400 1月 1 2008 etc1tool
-rwxr-xr-x 1 tgg tgg 2417136 1月 1 2008 fastboot
-rwxr-xr-x 1 tgg tgg 12848 1月 1 2008 hprof-conv
drwxrwxr-x 2 tgg tgg 4096 12月 5 15:35 lib64
-rwxr-xr-x 1 tgg tgg 262728 1月 1 2008 make_f2fs
-rwxr-xr-x 1 tgg tgg 262712 1月 1 2008 make_f2fs_casefold
-rwxr-xr-x 1 tgg tgg 879656 1月 1 2008 mke2fs
-rw-r--r-- 1 tgg tgg 1157 1月 1 2008 mke2fs.conf
-rw-r--r-- 1 tgg tgg 1075211 1月 1 2008 NOTICE.txt
-rwxr-xr-x 1 tgg tgg 1701544 1月 1 2008 sload_f2fs
-rw-r--r-- 1 tgg tgg 38 1月 1 2008 source.properties
-rwxr-xr-x 1 tgg tgg 1312888 1月 1 2008 sqlite3
tgg@tgg-virtual-machine:~/tools/platform-tools$
```

图 1.3.2 platform-tools 目录下的文件

执行如下命令将 platform-tools 目录所在路径添加到系统 PATH 环境变量中:

```
export PATH=$PATH:~/tools/platform-tools
```

也可以将该条命令添加至 ~/.bashrc 文件末尾, 这样每次打开新的终端时都会自动执行该条命令:

```
vi ~/.bashrc
```

```
104 if [ -f ~/.bash_aliases ]; then
105     . ~/.bash_aliases
106 fi
107
108 # enable programmable completion features (you don't need to enable
109 # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
110 # sources /etc/bash.bashrc).
111 if ! shopt -oq posix; then
112     if [ -f /usr/share/bash-completion/bash_completion ]; then
113         . /usr/share/bash-completion/bash_completion
114     elif [ -f /etc/bash_completion ]; then
115         . /etc/bash_completion
116     fi
117 fi
118
119 export PATH=$PATH:~/tools/platform-tools
```

在文件末尾添加这条命令

图 1.3.3 .bashrc 文件

添加完成后保存退出! 然后执行如下命令使其在当前终端生效:

```
source ~/.bashrc
```

接下来进行测试, 执行如下命令, 确认 adb 安装是否成功:

```
adb version
```

打印信息如图 1.3.4 所示:

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb version
Android Debug Bridge version 1.0.41
Version 33.0.3-8952118
Installed as /home/tgg/tools/platform-tools/adb
tgg@tgg-virtual-machine:~$
```

图 1.3.4 查询 adb 版本

执行命令能成功获取到 adb 版本信息, 则表示 adb 安装成功!

第二章 使用 adb 工具

本章向用户介绍 adb 工具的使用

本文档仅介绍 adb 工具的常见用法，详细使用方法请参考 Google 官方用户指南：

<https://developer.android.google.cn/studio/command-line/adb?hl=zh-cn>

2.3 查看当前连接的设备

执行如下命令可以查看当前连接的设备：

```
adb devices
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb devices
* daemon not running; starting now at tcp:5037
* daemon started successfully
List of devices attached
tgg@tgg-virtual-machine:~$
```

启动adb server打印的信息

列举当前adb server所连接的设备

图 2.3.1 查询当前连接的设备

当我们执行 adb 命令时，此时就启动了一个 adb client，adb client 会先检查是否有 adb server 进程正在运行，如果没有，它会先启动 adb server 进程，并输出如下信息：

```
* daemon not running; starting now at tcp:5037 * daemon started successfully
```

当输出“**daemon started successfully**”信息时，表示 adb server 已经启动成功了。adb server 启动成功后，它会与本地 TCP 端口 5037 进行绑定，并监听 adb client 发出的命令。“adb devices”命令用于查询当前连接的设备，并将其显示在“**List of devices attached**”下方；当前我这里并未连接任何设备，所以显示为空。

2.4 连接设备

本小节介绍如何连接设备，windows 操作与 ubuntu 虚拟机操作一致，故不赘述

2.4.1 连接 RK3588 Linux buildroot 系统

1. 通过 USB 连接

首先 ATK-DLRK3588 开发板烧录 Linux buildroot 系统镜像，然后使用 USB 线一头连接开发板的 TYPE-C0 口、另一头连接到电脑的 USB 口，如图 2.4.1.1 所示：

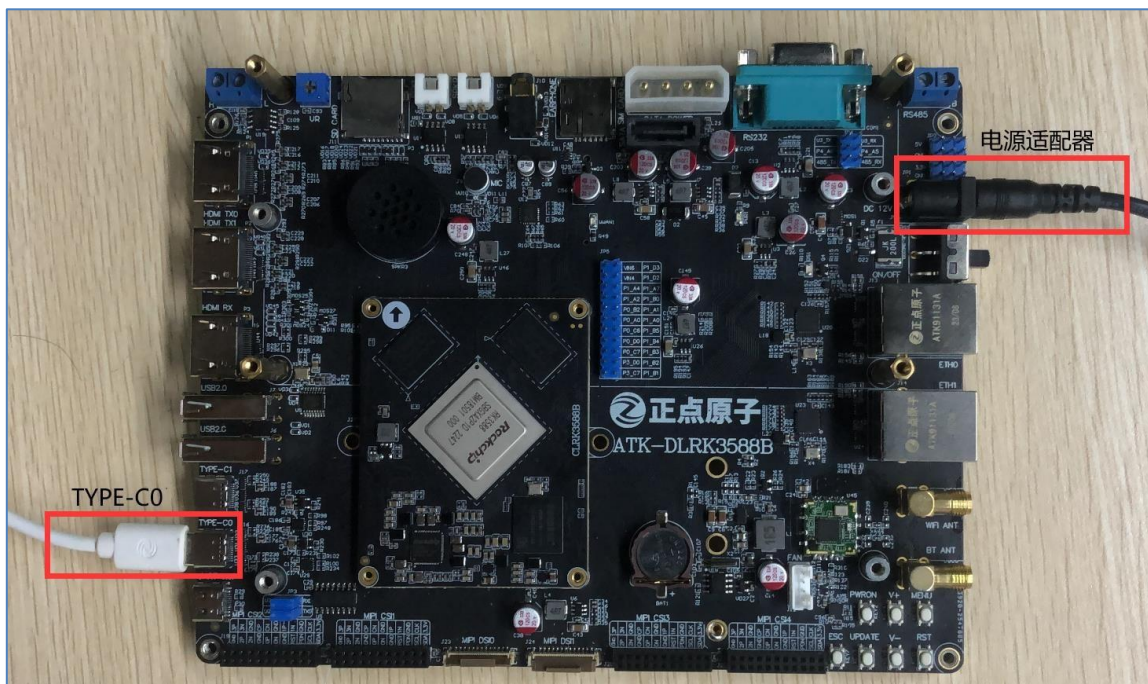


图 2.4.1.1 开发板硬件连接

连接好硬件上电启动开发板，进入 Linux buildroot 系统后，打开 Ubuntu 系统，在“虚拟机可移动设备”下查找是否存在一个名为“Fuzhou Rockchip Android ADB Interface”的设备，如下图所示：

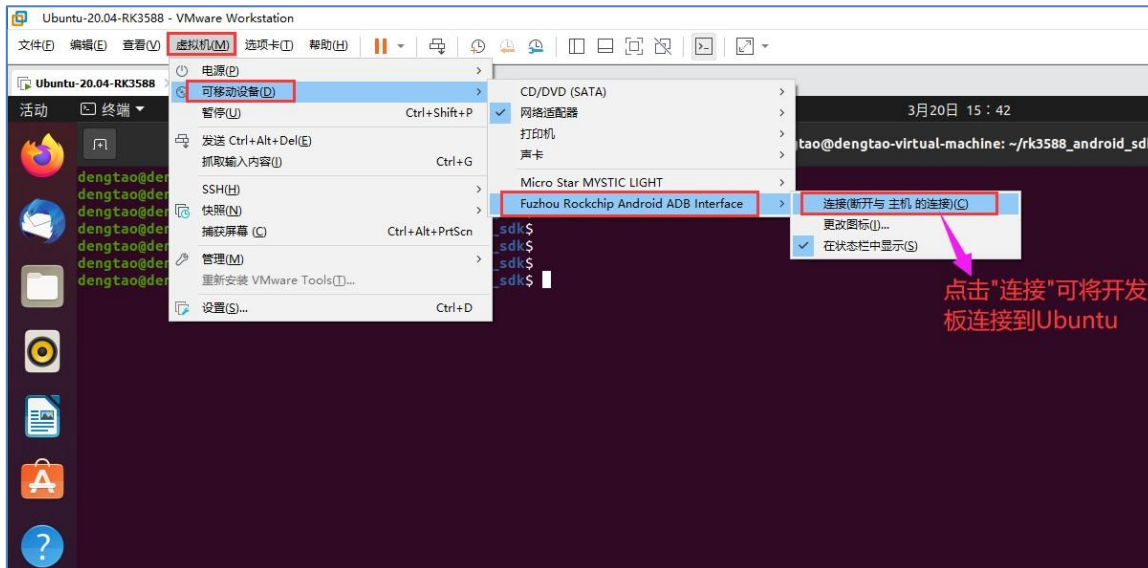


图 2.4.1.2 开发板连接到 Ubuntu

如果存在该设备则表示开发板已经连接到 PC 机了；如果不存在该设备，请检查硬件连接是否有误，并尝试重新拔插 USB 线或更换电脑的 USB 口。鼠标点击“连接(断开与主机的连接)(C)”将开发板连接到 Ubuntu。

连接成功后，Ubuntu 左侧导航栏会出现一个类似手机的图标（稍微等待一会），如图 2.4.1.3 所示：



图 2.4.1.3 连接成功

此时在终端执行“adb devices”命令:

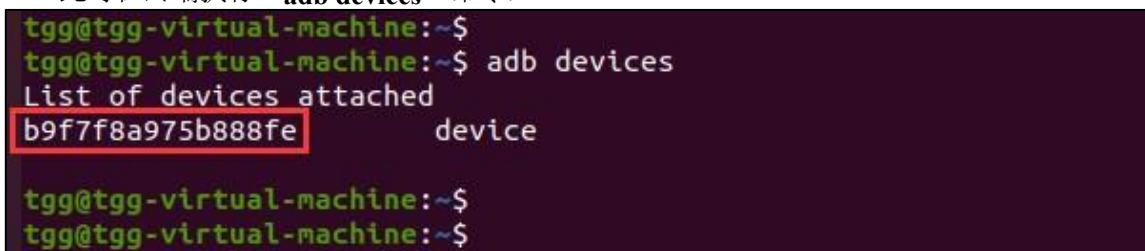


图 2.4.1.4 查询当前连接的设备 从上图可知, 当前已经有一个设备和 Ubuntu 建立了 adb 连接, “b9f7f8a975b888fe”是设备的序列号。

2.5 运行 shell

执行“adb shell”命令可以运行设备端的 shell (命令行终端), 如下所示:

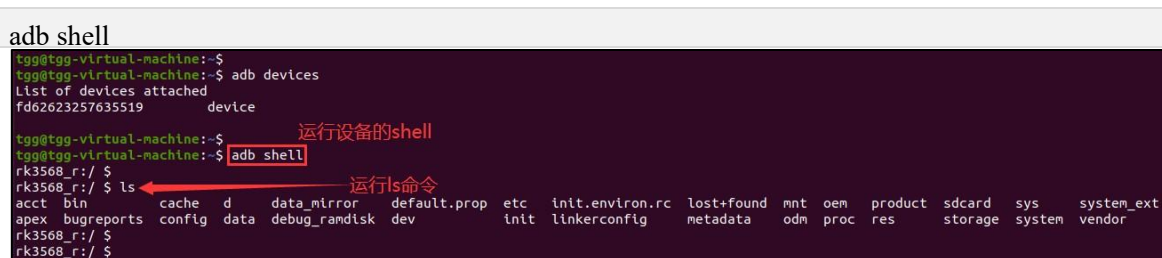


图 2.5.1 运行设备的 shell

可执行 exit 退出 shell。

也以执行如下命令、在设备端运行一条 shell 命令:

adb shell command

command 表示需要在设备端运行的 Linux 命令, 譬如:

```
adb shell ls adb
shell ifconfig adb
shell pwd
```

```
adb shell echo "Hello world"
```

2.6 文件传输

通过“adb push”命令可以将本地文件拷贝至设备：

```
adb push local remote
```

local 表示本地文件，remote 表示远端设备的路径，譬如将本地文件 helloWorld 拷贝至设备的/sdcard/目录：

```
adb push helloWorld /sdcard
```

```
tgg@tgg-virtual-machine:~$ touch helloWorld
tgg@tgg-virtual-machine:~$ echo "Hello World" > helloWorld
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb push helloWorld /sdcard/
helloWorld: 1 file pushed, 0 skipped. 0.0 MB/s (12 bytes in 0.000s)
tgg@tgg-virtual-machine:~$
```

图 2.6.1 文件拷贝

此时设备的/sdcard/目录下便会存在 helloWorld 文件：

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb shell
rk3568_r:/ $
rk3568_r:/ $ cat /sdcard/helloWorld
Hello World
rk3568_r:/ $
rk3568_r:/ $
```

图 2.6.2 查看文件内容

通过“adb pull”命令可以将远端设备的文件拷贝至本地：

```
adb pull remote local
```

remote 表示远端设备的文件，local 表示本地路径，譬如将设备的/sdcard/helloWorld 文件拷贝至本地：

```
tgg@tgg-virtual-machine:~$ rm -rf helloWorld
tgg@tgg-virtual-machine:~$ adb pull /sdcard/helloWorld ./
/sdcard/helloWorld: 1 file pulled, 0 skipped. 0.0 MB/s (12 bytes in 0.047s)
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ cat helloWorld
Hello World
tgg@tgg-virtual-machine:~$
```

图 2.6.3 文件拷贝

2.8 启动/停止 adb server

有时我们需要重启 adb server，可以先执行“adb kill-server”命令将 adb server 终止，然后再执行“adb start-server”命令启动 adb server，实现重启操作：

```
adb kill-server
adb start-server
```

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb kill-server
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ adb start-server
* daemon not running; starting now at tcp:5037
* daemon started successfully
tgg@tgg-virtual-machine:~$
```

图 2.8.1 重启 adb server

2.9 获取 root 权限

通过“adb root”命令获取设备的 root 权限，使得 adb 操作具有更高的权限。

```
adb root
```

2.10 重新挂载 remount

我们在使用“adb push”命令拷贝文件时，可能会失败，并且会有如下提示信息：“**Read-only file system**”：

```
remote secure_mkdirs failed: Read-only file system
```

这个信息表示设备端的文件系统是只读模式，无法写入。这个时候需要进行重新挂载，将文件系统重新挂载成可读可写模式，执行如下命令：

```
adb root
adb remount
```

```
tgg@tgg-virtual-machine:~$ adb remount
Using overlayfs for /system
Using overlayfs for /vendor
Using overlayfs for /odm
Using overlayfs for /product
Using overlayfs for /system_ext
Now reboot your device for settings to take effect
remount succeeded
tgg@tgg-virtual-machine:~$
```

执行“adb remount”命令之前，需要先执行“adb root”命令获取到设备的 root 权限。重新挂载之后，再次 push 时就不会出现问题了。

第四章 RK3588 Linux SDK 软件包

本章向用户介绍正点原子 ATK-DLRK3588 硬件平台 Linux SDK 软件包的安装以及使用方法, 该 SDK 适用于正点原子 ATK-DLRK3588 开发板及基于此开发板进行二次开发的所有 Linux 产品; 基于本 SDK, 可以有效实现系统定制和应用移植开发, 帮助用户快速开发、提高开发效率!

RK3588 Linux SDK 集成了 Linux Kernel 源码、U-Boot 源码以及 RK 提供的各种开发工具、文档等, 该 SDK 支持 buildroot、Yocto 以及 Debian 三种根文件系统; Linux 内核版本为 5.10、U-Boot 版本为 2017.09。

RK 官方参考文档:

[<SDK>/docs/cn/Rockchip_Developer_Guide_Linux_Software_CN.pdf](#)

4.1 安装 RK3588 Linux SDK

本小节向用户介绍如何在 Ubuntu 系统下安装正点原子 ATK-DLRK3588 开发板 Linux SDK 软件包。

4.1.1 安装依赖软件包

首先需要在 Ubuntu 系统下安装 SDK 编译环境所依赖的软件包, 执行如下命令安装软件包:

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \ libmpc-dev bc
python-is-python3 python2
```

安装过程中确保 Ubuntu 系统网络连接正常, 安装过程需要一定的时间, 请用户耐心等待!

执行如下命令, 设置 DNS 支持 kgithub.com:

```
sudo sed -i '$a 43.154.68.204\tkgithub.com' /etc/hosts
sudo sed -i '$a 43.155.83.75\traw.kgithub.com objects.githubusercontent.kgithub.com' /etc/hosts
```

4.1.3 Git 配置

使用 repo 之前需要用户配置自己的 git 信息, 否则后面的操作可能会遇到 hook 检查的障碍:

```
git config --global user.name "your name"
git config --global user.email "your email"
请用户根据实际情况配置。可通过如下命令查看 git 配置信息:
```



```
git config --list
```

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ git config --list
user.name=
user.email=71336161@qq.com
tgg@tgg-virtual-machine:~$
```

图 4.1.3.1 查看 git 配置信息

4.1.4 安装 SDK

接下来开始安装 RK3588 Linux SDK，开发板资料盘中已经给用户提供了 SDK 的压缩包文件，路径为：**开发板光盘 B 盘-开发环境及 SDK002、ATK-DLRK3588 开发板 SDK002atk3588_linux_release_v1.0_20240601.tgz**，随着版本的更新，SDK 压缩包文件的名称也将会发生改变，但均以 **atk-rk3588_linux_release_版本_发布日期.tgz** 方式进行命名。每次发布新版本 SDK 时，将会替换网盘中的旧版本 SDK，如果用户需要获取旧版本 SDK，请联系正点原子 Linux 技术支持获取！

将 tgz 压缩文件拷贝到 Ubuntu 系统的用户家目录下，并执行如下命令将其解压到家目录下的 rk3588 linux sdk 目录：

```
mkdir ~/rk3588_linux_sdk
```

```
tar xvf atk-rk3588_linux_release_v1.0_20240601.tgz -C ~/rk3588_linux_sdk
```

```
atk@atk-virtual-machine:~$
atk@atk-virtual-machine:~$ ls
公共的 模板 视频 图片 文档 下载 音乐 桌面 atk-rk3588_linux_release_v1.0_20240601.tgz bin
atk@atk-virtual-machine:~$
atk@atk-virtual-machine:~$ mkdir ~/rk3588_linux_sdk
atk@atk-virtual-machine:~$ tar xvf atk-rk3588_linux_release_v1.0_20240601.tgz -C ~/rk3588_linux_sdk
```

图 4.1.4.1 SDK 包解压

解压完成后，~/rk3588_linux_sdk/目录下会存在一个.repo 文件夹，如图 4.1.4.2 所示：

```
atk@atk-virtual-machine:~$
atk@atk-virtual-machine:~$ ls -lha ~/rk3588_linux_sdk/
总用量 12K
drwxrwxr-x  3 atk atk 4.0K 4月  3 15:19 .
drwxr-xr-x 18 atk atk 4.0K 4月  3 15:18 ..
drwxrwxr-x  7 atk atk 4.0K 3月 25 21:56 .repo
atk@atk-virtual-machine:~$
```

图 4.1.4.2 .repo 文件夹

执行如下命令可检出源码：

```
cd ~/rk3588_linux_sdk/
```

```
.repo/repo sync -l -j10
```

```
atk@atk-virtual-machine:~$ cd ~/rk3588_linux_sdk/
atk@atk-virtual-machine:~/rk3588_linux_sdk$ .repo/repo/repo sync -l -j10
正在更新文件: 100% (636/636), 完成.
正在更新文件: 100% (6233/6233), 完成.
正在更新文件: 100% (13693/13693), 完成.
正在更新文件: 100% (2114/2114), 完成.
正在更新文件: 100% (2010/2010), 完成.
正在更新文件: 100% (861/861), 完成.
正在更新文件: 100% (754/754), 完成.
正在更新文件: 100% (219/219), 完成.
正在更新文件: 100% (81635/81635), 完成.
正在更新文件: 100% (2181/2181), 完成.
正在更新文件: 100% (5939/5939), 完成.
正在更新文件: 100% (190/190), 完成.
正在更新文件: 100% (121/121), 完成.
正在更新文件: 100% (5722/5722), 完成.
正在更新文件: 100% (469/469), 完成.
正在更新文件: 100% (248/248), 完成.
正在更新文件: 100% (13701/13701), 完成.
正在更新文件: 100% (36111/36111), 完成.
正在更新文件: 100% (44/44), 完成.
正在更新文件: 100% (5515/5515), 完成.
正在更新文件: 100% (166/166), 完成.
正在更新文件: 100% (163/163), 完成.
正在更新文件: 100% (1197/1197), 完成.
Checking out: 100% (59/59), done in 18m28.859s
repo sync has finished successfully.
atk@atk-virtual-machine:~/rk3588_linux_sdk$
```

图 4.1.4.3 检出源码

整个过程将会持续比较长的时间，大家需要耐心等待，直到同步完成。

同步完成后，~/rk3588_linux_sdk/目录下的内容如图 4.1.4.4 所示：

```
atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$ ls
app      debian  envsetup.sh  Makefile  rkflash.sh  uefi
buildroot device  external     prebuilts tools        yocto
build.sh docs    kernel       rkbin     u-boot
```

图 4.1.4.4 Linux SDK 工程目录

至此，RK3588 Linux SDK 就安装完成了。

4.2.3 SDK 版本查询

正点原子 RK3588 Linux SDK 的版本可分为 RK 版本和 ATK 版本；所谓 RK 版本，则表示 Rockchip（瑞芯微）发布 SDK 时所定义版本号；而 ATK 版本则表示正点原子 Linux 技术团队发布 SDK 时所定义版本号，每次发布新版本 SDK 时都会有一个版本号与之对应。

从我们提供的 SDK 压缩包文件名便可知道其对应的 ATK 版本以及发布日期；也可在 Linux SDK 源码根目录下，执行如下命令查询 SDK 的 ATK 版本：

```
atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$ realpath .repo/manifests/atk-rk3588_linux5.10_release.xml
/home/atk/rk3588_linux_sdk/.repo/manifests/rk3588_linux/atk-rk3588_linux5.10_release_v1.0_20240601.xml
atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$
```

realpath .repo/manifests/atk-rk3588_linux5.10_release.xml

图 4.2.3.1 查询 SDK 的 ATK 版本

从图中可知, 当前 SDK 的 ATK 版本为 V1.0, 发布日期为 2024-06-01。

在 SDK 源码根目录下, 执行如下命令可查询当前 SDK 的 RK 版本:

```
ls .repo/manifests/rk3588_linux/rk3588_linux_release*
atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$ ls .repo/manifests/rk3588_linux/rk3588_linux_release*
.repo/manifests/rk3588_linux/rk3588_linux_release_v1.3.0_20230920.xml
atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$
```

图 4.2.3.2 查询 SDK 的 RK 版本

从图中可知, SDK 的 RK 版本为 V1.3.0, 发布日期为 2023 年 09 月 20 日。

4.3 SDK 全自动编译

本小节向用户介绍如何一键全自动编译整个 RK3588 Linux SDK, 首先进入到 SDK 根目录下, 编译之前先执行如下命令指定板级配置文件:

```
./build.sh lunch
atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$ ./build.sh lunch

##### Rockchip Linux SDK #####

Manifest: atk-rk3588_linux5.10_release_v1.0_20240601.xml

Log saved at /home/atk/rk3588_linux_sdk/output/sessions/2024-04-03_20-17-21

Pick a defconfig:

1. rockchip_defconfig
2. alientek_rk3588_defconfig
3. rockchip_rk3588_evb1_lp4_v10_defconfig
4. rockchip_rk3588_evb7_v11_defconfig
5. rockchip_rk3588s_evb1_lp4x_v10_defconfig
Which would you like? [1]: 2
```

图 4.3.1 选择板级配置文件 输入 “alientek_rk3588_defconfig”

对应的序号、然后按回车键确认:


```
Which would you like? [1]: 2
Switching to defconfig: /home/atk/rk3588_linux_sdk/device/rockchip/.chip/alientek_rk3588_defconfig
make: Entering directory '/home/atk/rk3588_linux_sdk/device/rockchip/common'
mkdir -p /home/atk/rk3588_linux_sdk/output/kconf/lxdialog
make CC="gcc" HOSTCC="gcc" \
  obj=/home/atk/rk3588_linux_sdk/output/kconf -C /home/atk/rk3588_linux_sdk/device/rockchip/common
make[1]: Entering directory '/home/atk/rk3588_linux_sdk/device/rockchip/common/kconfig'
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE
conf/.depend 2>/dev/null || :
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE
ut/kconf/conf.o
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE
f/zconf.tab.c -o /home/atk/rk3588_linux_sdk/output/kconf/zconf.tab.o
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE
.o /home/atk/rk3588_linux_sdk/output/kconf/zconf.tab.o -o /home/atk/rk3588_linux_sdk/output/kconf/
rm /home/atk/rk3588_linux_sdk/output/kconf/zconf.tab.c
make[1]: Leaving directory '/home/atk/rk3588_linux_sdk/device/rockchip/common/kconfig'
#
# configuration written to /home/atk/rk3588_linux_sdk/output/.config
#
make: Leaving directory '/home/atk/rk3588_linux_sdk/device/rockchip/common'
atk@atk-virtual-machine:~/rk3588_linux_sdk$
```

图 4.3.2 确认选择对应的板级配置文件

Tips: 上述操作过程可简化为如下命令:

```
./build.sh alientek_rk3588_defconfig
```

build.sh 是 RK 封装好的一个编译脚本, 使用该脚本可以方便用户快速构建出系统镜像文件以及对镜像进行打包操作, 既可以一键全自动编译整个 Linux SDK, 也可以单独编译 U-Boot、Linux Kernel、buildroot 等, 非常方便!

整个 SDK 编译过程中最耗时的部分便是根文件系统的编译了, 在编译根文件系统的过程中会通过网络下载很多的第三方开源库; 首先, 下载过程会占用很多时间导致编译时间拉长; 其次, 如果用户的网络环境不稳定或者第三方开源库的下载地址发生变更, 很容易导致下载失败, 进而导致根文件系统编译出错; 所以, 为了加快根文件系统的编译过程、也为了降低编译根文件系统出现问题的概率, 我们可以提前把所需的第三方开源库拷贝到 SDK 中。

正点原子 ATK-DLRK3588 开发板资料盘中已经给用户提供了这些所需的第三方开源库, 具体路径为: 开发板光盘 **B 盘-开发环境及 SDK002、ATK-DLRK3588 开发板 SDK001、linux_sdk01.tgz**, 首先将该压缩文件拷贝到 Ubuntu 系统家目录下, 如下所示:

```
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$ ls -l dl.tgz
-rw-rw-r-- 1 tgg tgg 1205119367 4月 6 17:34 dl.tgz
tgg@tgg-virtual-machine:~$
tgg@tgg-virtual-machine:~$
```

图 4.3.5 dl.tgz 压缩文件

执行如下命令将其解压到 SDK 根目录下的 buildroot 目录:

```
tar -xzf dl.tgz -C ~/rk3588_linux_sdk/buildroot/
```

```
atk@atk-virtual-machine:~$
atk@atk-virtual-machine:~$
atk@atk-virtual-machine:~$ tar -xzf dl.tgz -C ~/rk3588_linux_sdk/buildroot/
atk@atk-virtual-machine:~$
```

图 4.3.6 dl.tgz 解压

解压完成后, 可以进入到~/rk3588_linux_sdk/buildroot/目录, 该目录下会存在一个 dl 目录,

dl 目录下存放的便是第三方开源库，如下所示：

```
atk@atk-virtual-machine:~$
atk@atk-virtual-machine:~$ cd ~/rk3588_linux_sdk/buildroot/
atk@atk-virtual-machine:~/rk3588_linux_sdk/buildroot$
atk@atk-virtual-machine:~/rk3588_linux_sdk/buildroot$ cd dl/
atk@atk-virtual-machine:~/rk3588_linux_sdk/buildroot/dl$
atk@atk-virtual-machine:~/rk3588_linux_sdk/buildroot/dl$ ls
acl                gawk                libogg              pcre                qt5websockets
alsa-lib           gcc                 libopenSSL          pcre2               qt5xmlpatterns
alsa-plugins       gdb                 libpng              perl                 quazip
alsa-ucm-conf      gdk-pixbuf          libpthread-stubs    pixman              readline
alsa-utils         gesftpserver        libsndfile          pkgconf             rpcbind
android-tools      gettext-gnu         libsocketcan        pm-utils            rsync
at-spi2-core       gettext-tiny        libtheora            popt                rtmpdump
attr               ghostscript-fonts   libtirpc             procs-ng            rt-tests
autoconf           glibc              libtool             procrank_linux      sbc
autoconf-archive   glmark2             libv4l               protobuf            sdl2
autonake           gmp                 libv4l-rkmp         pulseaudio           source-han-sans-cn
bash              gperf              libvorbis            python3              sox
binutils           gst1-plugins-bad     libvp8               python-cython         speex
bison              gst1-plugins-base   libxcb               python-numpy          sqlite
bitstream-vera     gst1-plugins-good    libxkbcommon         python-pip            squashfs
bluez5_utils       gst1-plugins-ugly    libxml2              python-protobuf       strace
bluez-alsa         gstreamer1           libxslt              python-protobuf       stress
busybox            harfbuzz             libzlib              python-psutil         stressapptest
bzip2              hostapd              lmbench              python-ruamel-yaml    stress-ng
cairo              i2c-tools           lockfile-progs       python-ruamel-yaml-clib
cantarell          iniconfig            lrzsz                 python-setuptools     tiff
can-utils          input-event-daemon   lz4                   python-six            unixbench
ccache             iperf               lzo                   qemu                  usbmount
chromium-wayland   iperf3              m4                    qextserialport       util-linux
valgrind
```

图 4.3.7 dl 目录下的第三方开源库

接下来进入到 SDK 根目录下，执行如下命令编译整个 Linux SDK：

./build.sh all

```
atk@atk-virtual-machine:~/rk3588_linux_sdk$ ./build.sh all

##### Rockchip Linux SDK #####

Manifest: atk-rk3588_linux5.10_release_v1.0_20240601.xml

Log saved at /home/atk/rk3588_linux_sdk/output/sessions/2024-04-04_11-50-57

=====
Final configs
=====
RK_BOOT_FIT_ITS=boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT_CFG=rockchip_atk_dlrk3588
RK_CHIP=rk3588
RK_CHIP_FAMILY=rk3588
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_ARM64=y
RK_DEBIAN_VERSION=bullseye
RK_DEFCONFIG=/home/atk/rk3588_linux_sdk/device/rockchip/.chips/rk3588/alientek_rk3588_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=oem:oem:/oem:ext4:defaults:normal:auto:@userdata:userdata:/userdata:ext4:defaults:
RK_KERNEL_ARCH=arm64
RK_KERNEL_ARM64=y
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_CFG_FRAGMENT=rockchip_linux.config
RK_KERNEL_DTB=kernel/arch/arm64/boot/dts/rockchip/rk3588-atk-devkit.dtb
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/rk3588-atk-devkit.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
RK_KERNEL_KBUILD_ARCH=host
RK_KERNEL_KBUILD_HOST=y
RK_KERNEL_VERSION=5.10
RK_KERNEL_VERSION_REAL=5.10
RK_MISC=y
RK_MISC_BLANK=y
RK_PARAMETER=parameter.txt
RK_PCBA_CFG=rockchip_rk3588_pcba
RK_RECOVERY_CFG=rockchip_rk3588_recovery
RK_RECOVERY_FIT_ITS=boot4recovery.its
```

图 4.3.8 全自动编译整个 SDK

整个编译过程将会持续很长一段时间，编译所花费的时间长短与个人电脑配置有关，快则一个半小时左右、慢则 3、4 个小时，甚至更长的时间。

编译过程中，可能会遇到一些问题，导致编译失败！毕竟每个人的开发环境或多或少的存在一些差异，所以大家应尽量按照本文档说明进行操作，包括 Ubuntu 版本的选择、以及 4.1.1 小节中所要求安装的依赖软件包。除此之外，保证 Ubuntu 内存至少大于或等于 16GB，如果由于内存不足导致编译失败，可以将内存扩大再次尝试编译；如果电脑硬件条件不允许，已无法再继续扩大虚拟机 Ubuntu 的内存，这个时候我们可以尝试扩充 Ubuntu 的 swap 交换空间，扩充方法请参考：[开发板光盘 A 盘-基础资料❶10、用户手册❶03、辅助文档❶【正点原子】Ubuntu](#)

[扩充 swap 交换分区.pdf](#)。除此之外，Ubuntu 的磁盘空间尽量保证至少有 150~200G。

如果没有出现意外，编译将会成功，如图 4.3.10 所示：

```

userdata          userdata.img
Packing /home/atk/rk3588_linux_sdk/rockdev/update.img for update...
Android Firmware Package Tool v2.2
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xe1,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x227,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x731c0,userspace=0xe7
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x75000,size=0x400000,userspace=0x800,flash_address=0x00004000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x475000,size=0xc000,userspace=0x18,flash_address=0x00006000
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x481000,size=0x2328a00,userspace=0x4652,flash_address=0x00008000
Add file: ./recovery.img
recovery,Add file: ./recovery.img done,offset=0x27aa000,size=0x296f000,userspace=0x52de,flash_address=0x
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x5119000,size=0x45c00000,userspace=0x8b800,flash_address=0x00
Add file: ./oem.img
oem,Add file: ./oem.img done,offset=0x4ad19000,size=0x10a6000,userspace=0x214c,flash_address=0x01c78000
Add file: ./userdata.img
userdata,Add file: ./userdata.img done,offset=0x4bdf000,size=0x444000,userspace=0x888,flash_address=0x0
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.23*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Running mk-updateimg.sh - build_updateimg succeeded.
Running 99-all.sh - build_all succeeded.
atk@atk-virtual-machine:~/rk3588_linux_sdk$

```

图 4.3.10 SDK 编译成功

出现了“**Running 99-all.sh - build_all succeeded.**”打印信息则表示编译成功了。

编译完成后，生成的镜像存放在 output/firmware/目录，如下图所示：

```

atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$ cd output/firmware/
atk@atk-virtual-machine:~/rk3588_linux_sdk/output/firmware$
atk@atk-virtual-machine:~/rk3588_linux_sdk/output/firmware$ ls -l
总用量 14628
lrwxrwxrwx 1 atk atk      21 4月  5 11:06 boot.img -> ../../kernel/boot.img
lrwxrwxrwx 1 atk atk      44 4月  5 12:45 MiniLoaderAll.bin -> ../../u-boot/rk3588_spl_loader_v1.13.112.bin
-rw-rw-r-- 1 atk atk    49152 4月  5 12:45 misc.img
-rw-rw-r-- 1 atk atk   17457152 4月  5 12:45 oem.img
lrwxrwxrwx 1 atk atk      49 4月  5 12:45 parameter.txt -> ../../device/rockchip/.chips/rk3588/parameter.txt
lrwxrwxrwx 1 atk atk      67 4月  5 12:45 recovery.img -> ../../buildroot/output/rockchip_rk3588_recovery/images/recovery.img
lrwxrwxrwx 1 atk atk      63 4月  5 12:29 rootfs.img -> ../../buildroot/output/rockchip_atk_dlrk3588/images/rootfs.ext2
lrwxrwxrwx 1 atk atk      22 4月  5 12:45 uboot.img -> ../../u-boot/uboot.img
lrwxrwxrwx 1 atk atk      26 4月  5 12:46 update.img -> ../update/Image/update.img
-rw-rw-r-- 1 atk atk   4472832 4月  5 12:45 userdata.img
atk@atk-virtual-machine:~/rk3588_linux_sdk/output/firmware$

```

4.4.5 制作 update.img 固件

update.img 固件是多个镜像的集合体，它由多个镜像打包而成。使用 build.sh 脚本可以将 output/firmware/目录下的各分立镜像打包成一个 update.img 固件，方便用户烧写、更新升级。在 SDK 根目录下执行如下命令制作 update.img：

```
./build.sh updateimg
```

```
atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$ ./build.sh updateimg

##### Rockchip Linux SDK #####

Manifest: atk-rk3588_linux5.10_release_v1.0_20240601.xml

Log saved at /home/atk/rk3588_linux_sdk/output/sessions/2024-04-06_13-27-02

=====
                Final configs
=====
RK_BOOT_FIT_ITS=boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT_CFG=rockchip_atk_dlrk3588
RK_CHIP=rk3588
RK_CHIP_FAMILY=rk3588
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_ARM64=y
RK_DEBIAN_VERSION=bullseye
RK_DEFCONFIG=/home/atk/rk3588_linux_sdk/device/rockchip/.chips/rk3588/alientek_rk3588_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=oem:oem:/oem:ext4:defaults:normal:auto:@userdata:userdata:/userdata:ext4:defaults:
RK_KERNEL_ARCH=arm64
RK_KERNEL_ARM64=y
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_CFG_FRAGMENTS=rk3588_linux.config
RK_KERNEL_DTB=kernel/arch/arm64/boot/dts/rockchip/rk3588-atk-devkit.dtb
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/rk3588-atk-devkit.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
RK_KERNEL_KBUILD_ARCH=host
RK_KERNEL_KBUILD_HOST=y
RK_KERNEL_VERSION=5.10
RK_KERNEL_VERSION_REAL=5.10
RK_MISC=y
RK_MISC_BLANK=y
RK_PARAMETER=parameter.txt
RK_PCBA_CFG=rockchip_rk3588_pcba
RK_RECOVERY_CFG=rockchip_rk3588_recovery
RK_RECOVERY_FIT_ITS=boot4recovery.its
```



```

parameter      parameter.txt
bootloader      MiniLoaderAll.bin
uboot    uboot.img
misc      misc.img
boot      boot.img
recovery  recovery.img
backup    RESERVED
rootfs    rootfs.img
oem        oem.img
userdata  userdata.img
Packing /home/atk/rk3588_linux_sdk/rockdev/update.img for update...
Android Firmware Package Tool v2.2
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xe1,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x227,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x731c0,userspace=0xe7
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x75000,size=0x400000,userspace=0x800,flash_address=0x00004000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x475000,size=0xc000,userspace=0x18,flash_address=0x00006000
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x481000,size=0x2328a00,userspace=0x4652,flash_address=0x00008000
Add file: ./recovery.img
recovery,Add file: ./recovery.img done,offset=0x27aa000,size=0x296f000,userspace=0x52de,flash_address=0x0000a000
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x5119000,size=0x45e00000,userspace=0x8bc00,flash_address=0x0000c000
Add file: ./oem.img
oem,Add file: ./oem.img done,offset=0x4af19000,size=0x10a6000,userspace=0x214c,flash_address=0x01c78000
Add file: ./userdata.img
userdata,Add file: ./userdata.img done,offset=0x4bfbf000,size=0x444000,userspace=0x888,flash_address=0x0000e000
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.23*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Running 90-updateimg.sh - build_updateimg succeeded.

```

图 4.4.5.1 制作 update.img 固件

打包成功后,会在 output/firmware/目录下生成 update.img 固件,如下所示:

```

atk@atk-virtual-machine:~/rk3588_linux_sdk$
atk@atk-virtual-machine:~/rk3588_linux_sdk$ cd output/firmware/
atk@atk-virtual-machine:~/rk3588_linux_sdk/output/firmware$
atk@atk-virtual-machine:~/rk3588_linux_sdk/output/firmware$ ls -l update.img
lrwxrwxrwx 1 atk atk 26 4月  6 13:27 update.img -> ../update/Image/update.img
atk@atk-virtual-machine:~/rk3588_linux_sdk/output/firmware$

```

图 4.4.5.2 update.img 固件

执行“./build.sh all”编译完成后,会自动将所有分立镜像打包成 update.img。

RKNN AI 例程测试开发环境搭建

本章，我们将向大家介绍如何搭建 AI 例程测试开发环境，方便大家拿到开发板以后，可以直接编译 AI 例程，然后进行测试。

本章将分为如下几个小节：

- 1、 常见 IDE 工具简介
- 2、 安装交叉编译工具链
- 3、 anaconda 的安装与环境配置

1.2.1 拷贝交叉编译工具链

在进行编译程序之前，我们需要先将交叉编译工具链装到 ubuntu 下面。这个工具链是通过 RK3588 buildroot SDK 进行编译出来的，包含了我们 AI 例程所需要用到的库，可以直接在板上使用。将资料盘“开发板光盘 A 盘-基础资料→05、开发工具→03、交叉编译工具→atk-dlrk3588-toolchain-arm-buildroot-linux-gnueabi-hf-x86_64-xxxxxx-x.x.x.run”拷贝到 ubuntu 里面任意目录下（交叉编译工具链会持续更新，以实际目录工具链名字为准），如图 1.2.1.1 所示。



图 1.2.1.1 交叉编译工具链

将交叉编译工具链拷贝到家目录新建的 toolchain 目录下，可以执行 ls -l 进行查看可执行权限等等。

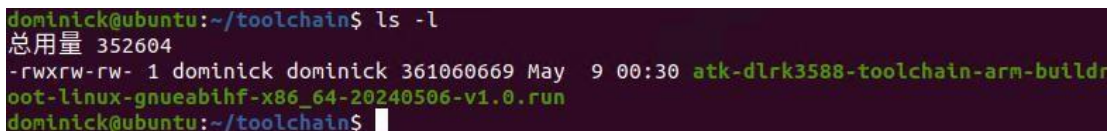


图 1.2.1.3 查看交叉编译工具链安装文件

如果没有可执行权限，使用的是 RK3588 开发板，可以执行以下命令给可执行权限。

!!! 一定要注意可执行权限 !!!

```
chmod a+x atk-dlrk3588-toolchain-arm-buildroot-linux-gnueabi-hf-x86_64-20240506-v1.0.run
```

1.2.2 安装交叉编译工具链

如果使用的开发板是 rk3588 开发板，打开终端执行以下命令进行安装编译工具链，安装过程如下图 2.2.2.1 所示。

```
./atk-dlrk3588-toolchain-arm-buildroot-linux-gnueabi-hf-x86_64-20240506-v1.0.run
```

当提示“Enter target directory for toolchain (default: /opt/atk-dlrk3588-toolchain):”时，表示是否选择默认安装在/opt/atk-dlrk3588-toolchain 目录下，建议直接选择默认安装路径，直接按下回车键即可。当提示“You are about to install the toolchain to "/opt/atk-dlrk358 8-toolchain".

Proceed[Y/n]?”时，直接按下“Y”回车即可。输入 ubuntu 密码回车后，当弹出提示“\$. source /opt/atk-dlrk3588-toolchain/environment-setup”时，表示已经安装完成。

```
dominick@ubuntu:~/toolchain$ ./atk-dlrk3588-toolchain-arm-buildroot-linux-gnueabi-hf-x86_64-20240506-v1.0.run
ATK-DLRK3588 toolchain installer version 1.0 Generated by Buildroot!
=====
Enter target directory for toolchain (default: /opt/atk-dlrk3588-toolchain):
You are about to install the toolchain to "/opt/atk-dlrk3588-toolchain". Proceed[Y/n]? Y
[sudo] password for dominick:
Extracting toolchain.....done
Relocating the toolchain to /opt/atk-dlrk3588-toolchain...
Toolchain has been successfully set up and is ready to be used.
Each time you wish to use the Toolchain in a new shell session, you need to source the environment setup script e.g.
$. source /opt/atk-dlrk3588-toolchain/environment-setup
dominick@ubuntu:~/toolchain$
```

.2.2.1 安装交叉编译工具链安装至此，交叉编译工具链安装完毕。

运行以下代码使能交叉编译器

```
export GCC_COMPILER=/opt/atk-dlrk3588-toolchain/bin/aarch64-buildroot-linux-gnu
```

使能后可执行

```
${GCC_COMPILER}-g++ name.cpp -o name -std=c++11 -march=armv8.2-a
```

将对应的.cpp 文件编译成 arm64 架构的可执行文件 elf

注意在 ubuntu 系统 shell 下执行 file name 检查其是否为 arm64 架构的 elf

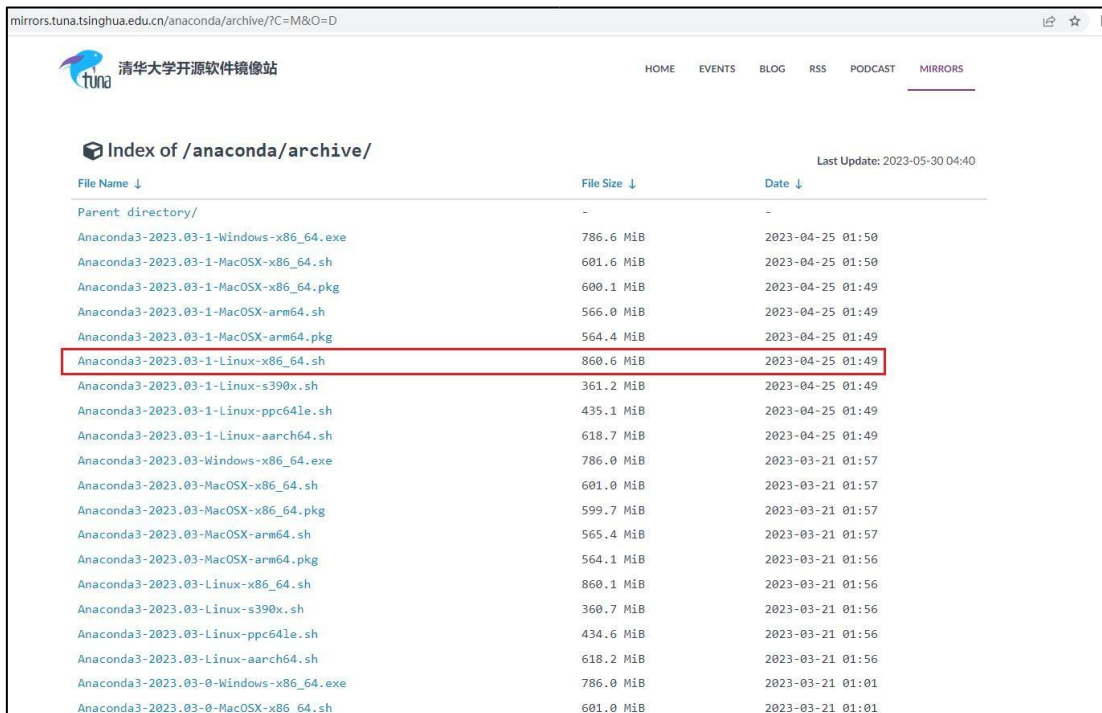
用 adb 工具将对应的 elf 推送至开发板后，在对应目录下运行./name 即可运行

1.3 anaconda 的安装与环境配置

1.3.1 anaconda 的下载安装

打开清华镜像库下载 <https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/?C=M&O=D>，因为我们选择在 ubuntu 环境下进行模型转换及性能评估等操作，这里选择 Anaconda3-2023.03-1-Linux-x86_64.sh 进行安装使用，我们也提供在开发板光盘 A 盘-基础资料→04、软件→A

naconda3-2023.03-1-Linux-x86_64.sh。



File Name ↓	File Size ↓	Date ↓
Parent directory/	-	-
Anaconda3-2023.03-1-Windows-x86_64.exe	786.6 MiB	2023-04-25 01:50
Anaconda3-2023.03-1-MacOSX-x86_64.sh	601.6 MiB	2023-04-25 01:50
Anaconda3-2023.03-1-MacOSX-x86_64.pkg	600.1 MiB	2023-04-25 01:49
Anaconda3-2023.03-1-MacOSX-arm64.sh	566.0 MiB	2023-04-25 01:49
Anaconda3-2023.03-1-MacOSX-arm64.pkg	564.4 MiB	2023-04-25 01:49
Anaconda3-2023.03-1-Linux-x86_64.sh	860.6 MiB	2023-04-25 01:49
Anaconda3-2023.03-1-Linux-s390x.sh	361.2 MiB	2023-04-25 01:49
Anaconda3-2023.03-1-Linux-ppc64le.sh	435.1 MiB	2023-04-25 01:49
Anaconda3-2023.03-1-Linux-aarch64.sh	618.7 MiB	2023-04-25 01:49
Anaconda3-2023.03-Windows-x86_64.exe	786.0 MiB	2023-03-21 01:57
Anaconda3-2023.03-MacOSX-x86_64.sh	601.0 MiB	2023-03-21 01:57
Anaconda3-2023.03-MacOSX-x86_64.pkg	599.7 MiB	2023-03-21 01:57
Anaconda3-2023.03-MacOSX-arm64.sh	565.4 MiB	2023-03-21 01:57
Anaconda3-2023.03-MacOSX-arm64.pkg	564.1 MiB	2023-03-21 01:56
Anaconda3-2023.03-Linux-x86_64.sh	860.1 MiB	2023-03-21 01:56
Anaconda3-2023.03-Linux-s390x.sh	360.7 MiB	2023-03-21 01:56
Anaconda3-2023.03-Linux-ppc64le.sh	434.6 MiB	2023-03-21 01:56
Anaconda3-2023.03-Linux-aarch64.sh	618.2 MiB	2023-03-21 01:56
Anaconda3-2023.03-0-Windows-x86_64.exe	786.0 MiB	2023-03-21 01:01
Anaconda3-2023.03-0-MacOSX-x86_64.sh	601.0 MiB	2023-03-21 01:01

图 1.3.1.1 交叉编译工具链安装界面

这里可以打开终端输入以下命令，先新建一个 **software** 文件夹方便后续使用，进入到 **software** 文件夹，在通过 **wget** 下载 **anaconda**（这里加入 **-c** 代表可以断点续传）。

```
mkdir ~/software
```

```
cd ~/software
```

```
wget -c https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-2023.03-1-Linux-x86_64.sh
```

```
dominick@ubuntu:~/software$ wget -c https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-2023.03-1-Linux-x86_64.sh
--2023-05-29 18:43:04-- https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-2023.03-1-Linux-x86_64.sh
正在解析主机 mirrors.tuna.tsinghua.edu.cn (mirrors.tuna.tsinghua.edu.cn)... 101.6.15.130, 2402:f000:1001:fde4::6475:c52d
正在连接 mirrors.tuna.tsinghua.edu.cn (mirrors.tuna.tsinghua.edu.cn)|101.6.15.130|:443... 已连接。
已发出 HTTP 请求，正在等待回应... 200 OK
长度: 902411137 (861M) [application/octet-stream]
正在保存至: "Anaconda3-2023.03-1-Linux-x86_64.sh"

Anaconda3 12%[=>] 108.57M 179KB/s 剩余 41m 55s
```

图 1.3.1.2 下载 Anaconda3 软件

下载完成后使用 **bash** 命令安装，然后按一次回车继续安装。

```
bash Anaconda3-2023.03-1-Linux-x86_64.sh
```

```
dominick@ubuntu:~/software$ bash Anaconda3-2023.03-1-Linux-x86_64.sh
Welcome to Anaconda3 2023.03-1

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>>
=====
End User License Agreement - Anaconda Distribution
=====

Copyright 2015-2023, Anaconda, Inc.

All rights reserved under the 3-clause BSD License:

This End User License Agreement (the "Agreement") is a legal agreement between you and Anaconda, Inc. ("Anaconda") and g
overns your use of Anaconda Distribution (which was formerly known as Anaconda Individual Edition).

Subject to the terms of this Agreement, Anaconda hereby grants you a non-exclusive, non-transferable license to:

* Install and use the Anaconda Distribution (which was formerly known as Anaconda Individual Edition),
* Modify and create derivative works of sample source code delivered in Anaconda Distribution from Anaconda's reposi
```

图 1.3.1.3 安装 Anaconda3 软件

再长按回车阅读 license，直到出现[no] >>>,输入 yes 并回车同意，出现路径的时候默认安装到用户目录下的 anaconda3 文件夹下，也可以自己指定路径安装，我们这里直接安装到默认位置，按下回车即可。

```
Last updated February 25, 2022

Do you accept the license terms? [yes|no]
[no] >>> yes 输入yes

Anaconda3 will now be installed into this location:
/home/dominick/anaconda3

- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

[/home/dominick/anaconda3] >>> 按下回车
PREFIX=/home/dominick/anaconda3
Unpacking payload ...
Extracting : anyio-3.5.0-py310h06a4308_0.conda: 1%| 4/439 [00:00<00:04, 103.19it/s]
```

图 1.3.1.4 同意 license 协议并指定安装路径

再输入 yes 并回车，出现 Thank you for installing Anaconda3!即完成安装。

```
by running conda init? [yes|no]
[no] >>> yes 输入yes完成安装
no change /home/dominick/anaconda3/condabin/conda
no change /home/dominick/anaconda3/bin/conda
no change /home/dominick/anaconda3/bin/conda-env
no change /home/dominick/anaconda3/bin/activate
no change /home/dominick/anaconda3/bin/deactivate
no change /home/dominick/anaconda3/etc/profile.d/conda.sh
no change /home/dominick/anaconda3/etc/fish/conf.d/conda.fish
no change /home/dominick/anaconda3/shell/condabin/Conda.psm1
no change /home/dominick/anaconda3/shell/condabin/conda-hook.ps1
no change /home/dominick/anaconda3/lib/python3.10/site-packages/xontrib/conda.xsh
no change /home/dominick/anaconda3/etc/profile.d/conda.csh
modified /home/dominick/.bashrc

==> For changes to take effect, close and re-open your current shell. <==

If you'd prefer that conda's base environment not be activated on startup,
set the auto_activate_base parameter to false:

conda config --set auto_activate_base false

Thank you for installing Anaconda3!
```

图 1.3.1.5 Anaconda 完成安装

重启 ubuntu 后打开终端会发现前面多了个 (base)，这意味着进入已经进入到了 conda 环境里，输入 conda deactivate 即可退出 conda 环境回到 ubuntu 环境。

```
(base) dominick@ubuntu:~/Desktop$ conda deactivate
```

图 1.3.1.6 退出 conda 环境

注意!!! 这样子每次开机都是默认进入到 conda 环境会非常不方便，为了防止和我们编译 RK3568/RK3588 sdk 等其它环境混乱，我们通过修改 conda 环境变量去设置不自动进入 conda 的虚拟环境，在终端执行以下命令。

```
conda config --set auto_activate_base false
```

```
dominick@ubuntu:~/Desktop$ conda config --set auto_activate_base false
dominick@ubuntu:~/Desktop$
```

图 1.3.1.7 配置不自动进入 conda 环境重启后打开终端发现默认不会进入 conda 环境。

1.3.2 anaconda 的环境配置

配置 anaconda 环境的下载源，因为官方的下载源是在国外会比较慢，所以建议将下载源改为国内的 anaconda 镜像源。a、首先执行以下命令查看下当前镜像源，显示只有个 defaults 默认源。

```
conda config --show channels
```

```
dominick@ubuntu:~/Desktop$ conda config --show channels
channels:
- defaults
```

图 1.3.2.1 查看 conda 环境配置

b、接下来添加清华镜像源

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/ conda
config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free/ conda config --
add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/r/ conda config --add
channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/msys2/ conda config --add
channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge/
```

```
dominick@ubuntu:~/Desktop$ conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/
dominick@ubuntu:~/Desktop$ conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free/
dominick@ubuntu:~/Desktop$ conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/r/
dominick@ubuntu:~/Desktop$ conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/msys2/
dominick@ubuntu:~/Desktop$ conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge/
```

图 1.3.2.2 配置 conda 环境镜像源

c、查看配置

```
conda config --show
```



```
dominick@ubuntu:~/Desktop$ conda config --show
add_anaconda_token: True
add_pip_as_python_dependency: True
aggressive_update_packages:
  - ca-certificates
  - certifi
  - openssl
allow_conda_downgrades: False
allow_cycles: True
allow_non_channel_urls: False
allow_softlinks: False
allowlist_channels: []
always_copy: False
always_softlink: False
always_yes: None
anaconda_upload: None
auto_activate_base: False
auto_stack: 0
auto_update_conda: True
bld_path:
changeups1: True
channel_alias: https://conda.anaconda.org
channel_priority: flexible
channel_settings: []
channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge/
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2/
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r/
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
  - defaults
```

图 1.3.2.3 查看是否配置好 conda 环境镜像源

第二章更新 NPU 驱动

RKNN-Toolkit2 是基于 Python 语言实现的模型转换工具，可将其他训练框架导出的模型转为 RKNN 模型，并提供较为有限的推理接口，协助用户测试模型转换效果。网址链接为：<https://github.com/rockchip-linux/rknn-toolkit2>。RKNP2 是板端的组件，提供 NPU 驱动，并基于 C 语言提供模型加载、模型推理等功能。RKNP2 具有一致的版本号。不同版本之间可能存在不兼容问题，为了避免造成不必要的麻烦，推荐用户使用同一版本的 RKNN-Toolkit2、RKNP2。

本章分为以下小节：

- 1、 下载 rknp2 驱动压缩包
- 2、 更新 npu 连板推理运行库

2.1 下载 rknpu2 驱动压缩包

为了后续方便使用 otg 和开发板连接来做 ai 相关的开发性能测试和连板推理等功能, 需要更新 rk3588/rk3568 的 npu 驱动 (准确来说不是驱动, 而是一个板端的连板服务, 关于 rk npu 驱动我们已经更新至目前最新的 0.9.2 版本了), 可以通过以下命令查看 npu 驱动。

```
dmesg | grep -i rknpu
```

如下图所示, 当前 RKNPU2 驱动版本为 0.9.2 版本。

```
root@ATK-DLRK3588:~# dmesg | grep -i rknpu
[ 4.775736] RKNPU fdab0000.npu: Adding to iommu group 0
[ 4.775887] RKNPU fdab0000.npu: RKNPU: rknpu iommu is enabled, using iommu mode
[ 4.777183] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdab0000-0xfdabffff]
[ 4.777207] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdac0000-0xfdacffff]
[ 4.777220] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdad0000-0xfdadffff]
[ 4.777493] [drm] Initialized rknpu 0.9.2 20230825 for fdab0000.npu on minor 1
[ 4.782913] RKNPU fdab0000.npu: RKNPU: bin=0
[ 4.783119] RKNPU fdab0000.npu: leakage=9
[ 4.783214] debugfs: Directory 'fdab0000.npu-rknpu' with parent 'vdd_npu_s0' already present!
[ 4.797418] RKNPU fdab0000.npu: pvtm=874
[ 4.804047] RKNPU fdab0000.npu: pvtm-volt-sel=3
[ 4.805004] RKNPU fdab0000.npu: avs=0
[ 4.805169] RKNPU fdab0000.npu: l=10000 h=85000 hyst=5000 l_limit=0 h_limit=800000000 h_table=0
[ 4.829745] RKNPU fdab0000.npu: failed to find power_model node
[ 4.829828] RKNPU fdab0000.npu: RKNPU: failed to initialize power model
[ 4.829834] RKNPU fdab0000.npu: RKNPU: failed to get dynamic-coefficient
```

图 2.1.1.1 查看 NPU 驱动版本

下面实操如何更新 rk3588 的 npu 驱动和连板服务, 在开发板资料光盘已经有下载好了的压缩包, 使用 RK3588/RK3568 开发板可以打开开发板光盘 A 盘-基础资料→01、程序源码→01、AI 例程→03、软件和驱动里的 rknn-toolkit2-2.0.0-beta0.zip 直接使用。

将压缩包拷贝解压到 ubuntu 下的 software 目录下

2.2 更新 npu 连板推理运行库

要更新 npu 连板推理运行库, 我们需要将下载下来的 rknpu2 文件夹里面的 runtime 目录下的 librknrt.so 和 rknn_server 文件及脚本传到开发板上, 主要是以下两个目录的文件。一个是 rknpu2/runtime/Linux/librknn_api/aarch64/目录。

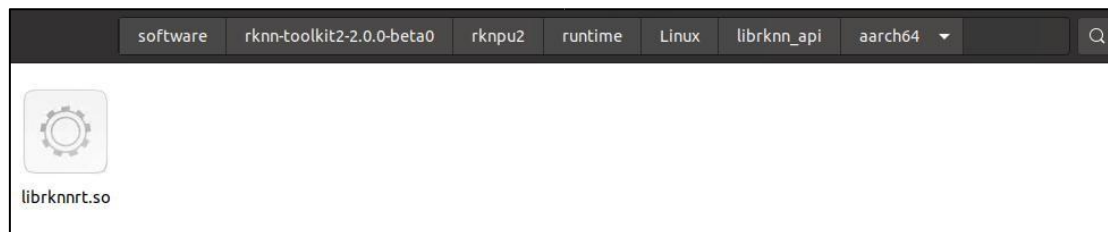


图 2.2.1.1 librknrt.so 文件

另一个是 rknpu2/runtime/Linux/rknn_server/aarch64/usr/bin/目录。

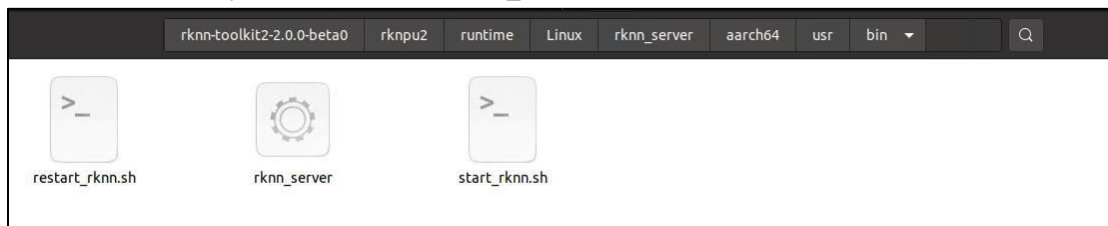


图 2.2.1.2 rknn_server 文件及相关脚本

在更新 npu 驱动服务前需要先将开发板电源接上以及开发板 otg 通过 Type-C 线接到电

脑，进到 rknpu2 驱动目录，在 npu 驱动目录打开终端，使用 adb 进行传输。

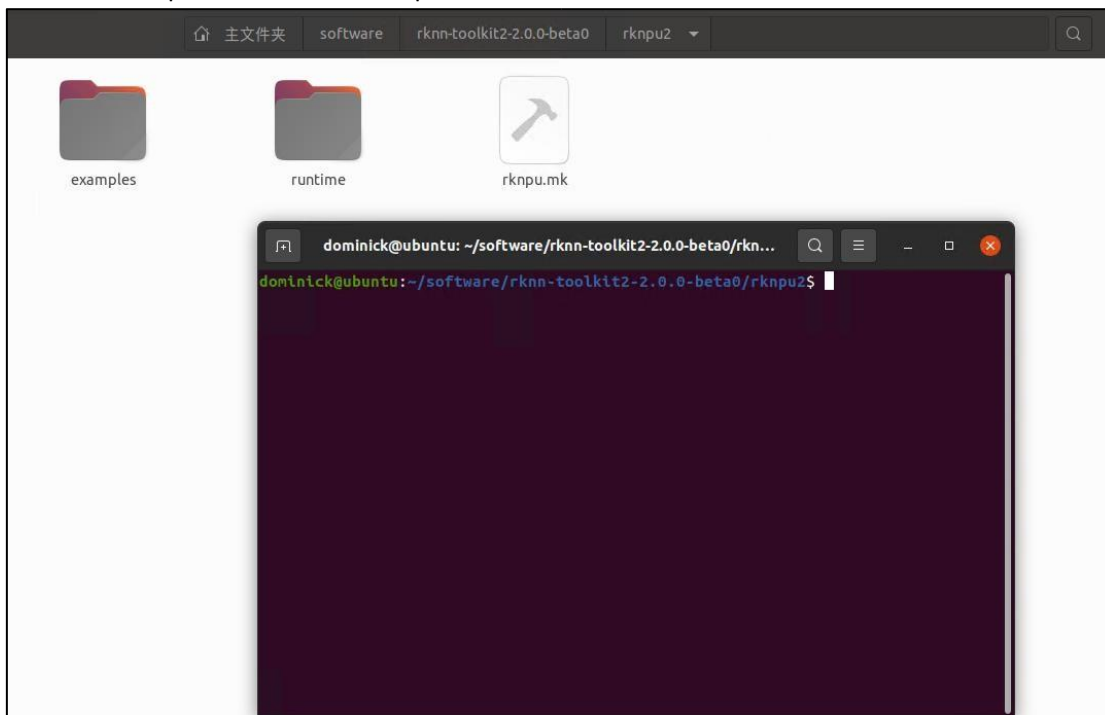


图 2.2.1.3 在 rknpu2 目录下打开终端

首先检测 adb 是否安装，如果没有安装 adb 的话，可以执行以下命令进行安装。

```
sudo apt install adb
```

在 ubuntu 终端里依次执行以下命令将文件传到开发板里。

```
adb push runtime/Linux/librknn_api/aarch64/librknnrt.so /usr/lib
```

```
dominick@ubuntu:~/software/rknn-toolkit2-2.0.0-beta0/rknpu2$ adb push runtime/Linux/librknn_api/aarch64/librknnrt.so /usr/lib
* daemon not running; starting now at tcp:5037
* daemon started successfully
runtime/Linux/librknn_api/aarch64/librknnrt.so: ... file pushed. 1.0 MB/s (6863912 bytes in 6.664s)
```

图 2.2.1.4 adb 传送 rknn 推理库到开发板

```
adb push runtime/Linux/rknn_server/aarch64/usr/bin/* /usr/bin
```

```
dominick@ubuntu:~/software/rknn-toolkit2-2.0.0-beta0/rknpu2$ adb push runtime/Linux/rknn_server/aarch64/usr/bin/* /usr/bin/
runtime/Linux/rknn_server/aarch64/usr/bin/restart_rknn.sh: 1 file pushed. 0.0 MB/s (252 bytes in 0.010s)
runtime/Linux/rknn_server/aarch64/usr/bin/rknn_server: 1 file pushed. 1.0 MB/s (442944 bytes in 0.425s)
runtime/Linux/rknn_server/aarch64/usr/bin/start_rknn.sh: 1 file pushed. 0.0 MB/s (71 bytes in 0.009s)
3 files pushed. 0.9 MB/s (443267 bytes in 0.454s)
```

图 2.2.1.5 adb 传送 rknn_server 及脚本到开发板在 ubuntu 终端

里面进入到开发板 shell 环境，给予可执行权限，并重启服务。

```
adb shell
```

```
chmod +x /usr/bin/rknn_server
chmod +x /usr/bin/start_rknn.sh
chmod +x /usr/bin/restart_rknn.sh
restart_rknn.sh
```

```
dominick@ubuntu:~/software/rknn-toolkit2-2.0.0-beta0/rknpu2$ adb shell
root@ATK-DLRK3588:/# chmod +x /usr/bin/rknn_server
root@ATK-DLRK3588:/# chmod +x /usr/bin/start_rknn.sh
root@ATK-DLRK3588:/# chmod +x /usr/bin/restart_rknn.sh
root@ATK-DLRK3588:/# restart_rknn.sh
```

图 2.2.1.6 进入开发板 shell 命令行

如果在开发板上执行 `restart_rknn.sh` 脚本后, 重启服务后会打印版本号。

```
root@ATK-DLRK3588:~# restart_rknn.sh
root@ATK-DLRK3588:~# start rknn server, version:2.0.0b0 (18eacd0 build@2024-03-22T14:07:19)
I NPUTransfer: Starting NPU Transfer Server, Transfer version 2.1.0 (b5861e7@2020-11-23T11:50:51)
```

图 2.2.1.7 查看 rknn 版本号

执行以下命令查询 `rknn_server` 版本, 会打印 `rknn_server` 的版本。

```
strings /usr/bin/rknn_server | grep build
root@ATK-DLRK3588:~# strings /usr/bin/rknn_server | grep build
2.0.0b0 (18eacd0 build@2024-03-22T14:07:19)
rknn_server version: 2.0.0b0 (18eacd0 build@2024-03-22T14:07:19)
.note.gnu.build-id
```

图 2.2.1.8 查看 rknn_server 服务版本

执行以下命令查询 `librknnrt.so` 版本, 会打印 `librknnrt` 的版本。

```
strings /usr/lib/librknnrt.so | grep version
root@ATK-DLRK3588:~# strings /usr/lib/librknnrt.so | grep version
librknnrt version: 2.0.0b0 (35a6907d79@2024-03-24T10:31:14)
rknn_set_input_shape is deprecated next version! please call rknn_set_input_shapes()
rknn_query, info_len(%d) < sizeof(rknn_sdk_version)(%d)!
RKNN model version is
, but current librknnrt.so is support model version <=
please try updating to the latest version of the toolkit2 and runtime from: https://console.zbox.filez.com/l/I00fc3 (PWD: rknn)
RKNN Model Information, version: %d, toolkit version: %s, target: %s, target platform: %s, framework name: %s, framework layout
: %s, model inference type: %s
RKNN Model version: %d.%d.%d not match with rknn runtime version: %d.%d.%d
version
(compiler version:
Current driver version: %d.%d.%d, recommend to upgrade the driver to the new version: >= %d.%d.%d
This shared library is for RK356X/RK3588/RV1103/RV1106/RK3576/RK2118 platforms only, hw_version = %d
RKNN Driver Information, version: %d.%d.%d
Mismatch driver version, %s requires driver version >= %d.%d.%d, but you have driver version: %d.%d.%d which is incompatible!
Illegal core num, rknn model version illegal
wrong version
failed to check rknpu hardware version: %x
Unsupport LayerNormalization! Please lower the OPSET version of the onnx model to below 16.
Do not support OpenCL version:
Parse device version[
.gnu.version
.gnu.version r
```

图 2.2.1.9 查看 librknnrt 运行时版本

输入 `exit` 退出开发板系统, 回到 `ubuntu` 终端, RK3568 更新库的方法也一样。

第三章安装 rknn-toolkit2 转换环境

随着深度学习在各个领域的应用日益广泛, 对于嵌入式和边缘计算设备来说, 能够实时、高效地处理深度学习任务变得越来越重要。然而, 由于这些设备的硬件限制 (如计算能力、内存大小等), 直接在它们上运行传统的深度学习模型往往是不切实际的。

`rknn-toolkit2` 的出现解决了这一难题。它提供了一个完整的解决方案, 允许开发者将已经训练好的深度学习模型 (如传统的 `TensorFlow`、`Pytorch`、`onnx` 等格式的模型) 转换为优化的 `rknn` 格式, 这些格式的模型能够充分利用 RK3568、RK3588 等设备的硬件特性, 从而在保证模型性能的同时, 降低了对计算资源的需求。

本章将分为以下章节:

- 1、新建 `conda` 环境
- 2、安装 `rknn-toolkit2` 工具

3.1 新建 conda 环境

前面我们下载了 anaconda, 为了防止后面编译环境混乱, 在这里就派上用场了, 我们首先新建一个 conda 虚拟环境再进行安装 rknn-toolkit2, 打开终端执行以下命令, 新建一个 conda 环境名 python3.8-tk2-2.0 的且 Python 版本为 Python3.8 版本的 conda 环境。

```
conda create --name python3.8-tk2-2.0 python=3.8
```

```
dominick@ubuntu:~/Desktop$ conda create --name python3.8-tk2-2.0 python=3.8
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 23.3.1
  latest version: 24.4.0

Please update conda by running

    $ conda update -n base -c defaults conda

Or to minimize the number of packages updated during conda update use

    conda install conda=24.4.0

## Package Plan ##

environment location: /home/dominick/anaconda3/envs/python3.8-tk2-2.0

added / updated specs:
- python=3.8
```

图 3.1.1.1 创建 conda 环境

程序停在 Proceed, 按下 y 回车。

```
_libgcc_mutex      anaconda/cloud/conda-forge/linux-64::_libgcc_mutex-0.1-conda_forge
_openmp_mutex      anaconda/cloud/conda-forge/linux-64::_openmp_mutex-4.5-2_gnu
bzip2              anaconda/cloud/conda-forge/linux-64::bzip2-1.0.8-h7f98852_4
ca-certificates    anaconda/cloud/conda-forge/linux-64::ca-certificates-2023.5.7-hbcca054_0
ld_impl_linux-64   anaconda/cloud/conda-forge/linux-64::ld_impl_linux-64-2.40-h41732ed_0
libffi             anaconda/cloud/conda-forge/linux-64::libffi-3.4.2-h7f98852_5
libgcc-ng          anaconda/cloud/conda-forge/linux-64::libgcc-ng-12.2.0-h65d4601_19
libgomp            anaconda/cloud/conda-forge/linux-64::libgomp-12.2.0-h65d4601_19
libns1             anaconda/cloud/conda-forge/linux-64::libns1-2.0.0-h7f98852_0
libsqlite          anaconda/cloud/conda-forge/linux-64::libsqlite-3.42.0-h2797004_0
libuuid            anaconda/cloud/conda-forge/linux-64::libuuid-2.38.1-h0b41bf4_0
libzlib            anaconda/cloud/conda-forge/linux-64::libzlib-1.2.13-h166bdaf_4
ncurses            anaconda/cloud/conda-forge/linux-64::ncurses-6.3-h27087fc_1
openssl            anaconda/cloud/conda-forge/linux-64::openssl-3.1.1-hd590300_1
pip               anaconda/cloud/conda-forge/noarch::pip-23.1.2-pyhd8ed1ab_0
python             anaconda/cloud/conda-forge/linux-64::python-3.8.16-he550d4f_1_cpython
readline           anaconda/cloud/conda-forge/linux-64::readline-8.2-h8228510_1
setuptools         anaconda/cloud/conda-forge/noarch::setuptools-67.7.2-pyhd8ed1ab_0
tk                 anaconda/cloud/conda-forge/linux-64::tk-8.6.12-h27826a3_0
wheel              anaconda/cloud/conda-forge/noarch::wheel-0.40.0-pyhd8ed1ab_0
xz                 anaconda/cloud/conda-forge/linux-64::xz-5.2.6-h166bdaf_0

Proceed ([y]/n)? y
```

图 3.1.1.2 选择 y 回车


```
The following NEW packages will be INSTALLED:

_libgcc_mutex             anaconda/cloud/conda-forge/linux-64::_libgcc_mutex-0.1-conda_forge
_openmp_mutex             anaconda/cloud/conda-forge/linux-64::_openmp_mutex-4.5-2_gnu
bzip2                     anaconda/cloud/conda-forge/linux-64::bzip2-1.0.8-hd590300_5
ca-certificates           anaconda/cloud/conda-forge/linux-64::ca-certificates-2024.2.2-hbcca054_0
ld_impl_linux-64          anaconda/cloud/conda-forge/linux-64::ld_impl_linux-64-2.40-h55db66e_0
libffi                    anaconda/cloud/conda-forge/linux-64::libffi-3.4.2-h7f98852_5
libgcc-ng                 anaconda/cloud/conda-forge/linux-64::libgcc-ng-13.2.0-h77fa898_7
libgomp                   anaconda/cloud/conda-forge/linux-64::libgomp-13.2.0-h77fa898_7
libnspr                    anaconda/cloud/conda-forge/linux-64::libnspr-2.0.1-hd590300_0
libsqlite                 anaconda/cloud/conda-forge/linux-64::libsqlite-3.45.3-h2797004_0
libuuid                   anaconda/cloud/conda-forge/linux-64::libuuid-2.38.1-h0b41bf4_0
libxcrypt                 anaconda/cloud/conda-forge/linux-64::libxcrypt-4.4.36-hd590300_1
libzlib                   anaconda/cloud/conda-forge/linux-64::libzlib-1.2.13-hd590300_5
ncurses                   anaconda/cloud/conda-forge/linux-64::ncurses-6.4.20240210-h59595ed_0
openssl                   anaconda/cloud/conda-forge/linux-64::openssl-3.3.0-hd590300_0
pip                       anaconda/cloud/conda-forge/noarch::pip-24.0-pyhd8ed1ab_0
python                   anaconda/cloud/conda-forge/linux-64::python-3.8.19-hd12c33a_0_cpython
readline                  anaconda/cloud/conda-forge/linux-64::readline-8.2-h8228510_1
setuptools                anaconda/cloud/conda-forge/noarch::setuptools-69.5.1-pyhd8ed1ab_0
tk                        anaconda/cloud/conda-forge/linux-64::tk-8.6.13-noxft_h4845f30_101
wheel                     anaconda/cloud/conda-forge/noarch::wheel-0.43.0-pyhd8ed1ab_1
xz                        anaconda/cloud/conda-forge/linux-64::xz-5.2.6-h16bdafe_0

Proceed ([y]/n)? y

Downloading and Extracting Packages

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#   $ conda activate python3.8-tk2-2.0
#
# To deactivate an active environment, use
#
#   $ conda deactivate
```

图 3.1.1.3 conda 环境新建完成

至此 conda 环境新建完成，可以通过在命令行终端输入以下命令进入 conda 环境。

```
conda activate python3.8-tk-2.0
```

在 conda 环境下通过以下命令退出该 conda 环境。

```
conda deactivate
```

3.2 安装 rknn-toolkit2 工具

3.2.1 安装 rknn-toolkit2 依赖库

进入到 rknn-toolkit2-2.0.0-beta0/rknn-toolkit2/packages 文件夹下面，打开终端执行以下命令激活 conda 环境。

```
conda activate python3.8-tk2-2.0
```

终端命令行前面出现 (python3.8-tk2-2.0) 即进入前面新建的 py3.8 的 conda 环境成功。

```
dominick@ubuntu:~/software/rknn-toolkit2-2.0.0-beta0/rknn-toolkit2/packages$ conda activate python3.8-tk2-2.0
(python3.8-tk2-2.0) dominick@ubuntu:~/software/rknn-toolkit2-2.0.0-beta0/rknn-toolkit2/packages$
```

图 3.2.1.1 进入 conda 环境

在安装 rknn-toolkit2 之前需要先安装相关依赖库，执行以下命令之前需要先检查自己是否进入到 rknn-toolkit2/packages 文件夹了，没有进入需要先进入 packages 文件夹。

```
pip install -r requirements_cp38-2.0.0b0.txt -i https://mirror.baidu.com/pypi/simple
```



```

(donnick@ubuntu:~/software/rknn-toolkit2-2.0.0-beta0/rknn-toolkit2/packages$ conda activate python3.8-tk2-2.0
(python3.8-tk2-2.0) donnick@ubuntu:~/software/rknn-toolkit2-2.0.0-beta0/rknn-toolkit2/packages$ pip install -r requirements_cp38-2.0.0b0.txt -i https://mirror.baid
Looking in indexes: https://mirror.baid
Collecting protobuf==3.20.3 (from -r requirements_cp38-2.0.0b0.txt (line 4))
  Downloading https://mirror.baid
 64.whl (1.0 MB)
Collecting psutil==5.9.0 (from -r requirements_cp38-2.0.0b0.txt (line 7))
  Downloading https://mirror.baid
 64.whl (288 kB)
Collecting ruamel.yaml==0.17.4 (from -r requirements_cp38-2.0.0b0.txt (line 9))
  Downloading https://mirror.baid
 117 kB)
Collecting scipy==1.5.4 (from -r requirements_cp38-2.0.0b0.txt (line 9))
  Downloading https://mirror.baid
 64.whl (34.5 MB)
Collecting tqdm==4.64.0 (from -r requirements_cp38-2.0.0b0.txt (line 10))
  Downloading https://mirror.baid
 78 kB)
Collecting opencv-python==4.5.5.64 (from -r requirements_cp38-2.0.0b0.txt (line 11))
  Downloading https://mirror.baid
 62 MB)
Collecting fast-histogram==0.11 (from -r requirements_cp38-2.0.0b0.txt (line 12))
  Downloading https://mirror.baid
 57 kB)
Collecting onnx==1.14.1 (from -r requirements_cp38-2.0.0b0.txt (line 15))
  Downloading https://mirror.baid
 14.6 MB)

```

图 3.2.1.2 安装 rknn-toolkit2 所需依赖库

这里参数后面的 `-i https://mirror.baidu.com/pypi/simple` 意思是指定源为 `pip` 镜像源为 <https://mirror.baidu.com/pypi/simple>,

该镜像源若无法使用, 可换为华为源, 代码如下

`pip install -r requirements_cp38-2.0.0b0.txt -i https://repo.huaweicloud.com/repository/pypi/simple/`

!!! 华为源会报错, 需要你手动将其中一个库降级!!!

(具体哪个库我忘了, 记得读报错信息)

当出现如下所示 `Successfully installed` 即安装成功相关依赖库。

```

Installing collected packages: tf-estimator-nightly, tensorboard-plugin-wit, mmhath, libclang, keras, flatbuffers, zipp, wrapt, urllib3, typing-extensions, tqdm, termcolor, tensor
low-io-gcs-fsfilesystem, tensorboard-data-server, sympy, six, ruamel.yaml.clib, pyasn1, psutil, protobuf, packaging, oauthlib, nvidia-nccl-cu12, nvidia-nvjitlink-cu12, nvidia-nccl-cu
12, nvidia-curand-cu12, nvidia-cufft-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, numpy, networkx, MarkupSafe, idna, humanfr
riendly, grpcio, gast, fsspec, filelock, charset-normalizer, certifi, cachetools, absl-py, werkzeug, triton, scipy, ruamel.yaml, rsa, requests, pyasn1-modules, opt-einsum, opencv-pyt
hon, onnx, onnxoptimizer, nvidia-cusolver-cu12, markdown, google-auth, torch, google-auth-oauthlib, tensorboard, tensorflow
Successfully installed MarkupSafe-2.1.5 absl-py-2.1.0 astunparse-1.6.3 cachetools-5.3.3 certifi-2024.2.2 charset-normalizer-3.3.2 coloredlogs-15.0.1 fast-histogram-0.13 filelock-3.
14.0 flatbuffers-24.3.25 fsspec-2024.3.1 gast-0.5.4 google-auth-2.29.0 google-auth-oauthlib-0.4.6 google-pasta-0.2.0 grpcio-1.63.0 h5py-3.11.0 humanfriendly-10.0 idna-3.7 importlib
metadata-7.1.0 Jinja2-3.1.4 keras-2.8.0 keras-preprocessing-1.2 libclang-18.1.1 markdown-3.6 mmhath-1.3.0 networkx-3.1 numpy-1.24.4 nvidia-cublas-cu12-12.1.3.1 nvidia-cuda-cupti
-cu12-12.1.105 nvidia-cuda-nvrtc-cu12-12.1.105 nvidia-cuda-runtime-cu12-12.1.105 nvidia-cudnn-cu12-8.9.2.26 nvidia-cufft-cu12-11.0.2.54 nvidia-curand-cu12-10.3.2.106 nvidia-cusolve
r-cu12-11.4.5.107 nvidia-cusparse-cu12-12.1.0.106 nvidia-nccl-cu12-12.1.1 nvidia-nvjitlink-cu12-12.4.127 nvidia-nvtx-cu12-12.1.105 oauthlib-3.2.2 onnx-1.14.1 onnxoptimizer-0.2.7 on
nxruntime-1.16.0 opencv-python-4.9.0.80 opt-einsum-3.3.0 packaging-24.0 protobuf-3.20.3 psutil-5.9.8 pyasn1-0.6.0 pyasn1-modules-0.4.0 requests-2.31.0 requests-oauthlib-2.0.0 rsa-4
.8 ruamel.yaml-0.18.6 ruamel.yaml.clib-0.2.8 scipy-1.10.1 six-1.16.0 sympy-1.12 tensorboard-2.8.0 tensorboard-data-server-0.6.1 tensorboard-plugin-wit-1.8.1 tensorflow-2.8.0 tensor
flow-io-gcs-fsfilesystem-0.34.0 termcolor-2.4.0 tf-estimator-nightly-2.8.0.dev202312109 torch-2.1.0 tqdm-4.66.4 triton-2.1.0 typing-extensions-4.11.0 urllib3-2.2.1 werkzeug-3.0.3 wr
apt-1.16.0 zipp-3.18.1

```

图 3.2.1.4 rknn-toolkit2 所需依赖库安装完成

3.3.2 安装 rknn-toolkit2 工具

依旧在 `conda` 环境 `py3.8` 下进到 `rknn-toolkit2/packages` 目录, 执行以下命令。

```
pip install rknn_toolkit2-2.0.0b0+9bab5682-cp38-cp38-linux_x86_64.whl
```

安装完成后会在最后一行出现红框所示 `Successfully installed rknn-toolkit2-2.0.0+9bab5682` 即安装成功。

```

Requirement already satisfied: rsa==4.7.1 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from google-auth<3,=>1.6.3->tensorboard<2.9,=>2.8->tenso
rflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (4.8)
Requirement already satisfied: requests-oauthlib==0.7.0 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from google-auth-oauthlib<0.5,=>0.4.1->tenso
rboard<2.9,=>2.8->tensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (2.0.0)
Requirement already satisfied: importlib-metadata==4.4 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from markdown==2.6.8->tensorboard<2.9,=>2.8->
tensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (7.1.0)
Requirement already satisfied: charset-normalizer<4,=>2 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from requests<3,=>2.21.0->tensorboard<2.9,=>
2.8->tensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (3.3.2)
Requirement already satisfied: idna<4,=>2.5 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from requests<3,=>2.21.0->tensorboard<2.9,=>2.8->tenso
rflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (3.7)
Requirement already satisfied: urllib3<3,=>1.21.1 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from requests<3,=>2.21.0->tensorboard<2.9,=>2.8->t
ensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (2.2.1)
Requirement already satisfied: certifi==2017.4.17 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from requests<3,=>2.21.0->tensorboard<2.9,=>2.8->t
ensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (2024.2.2)
Requirement already satisfied: zipp==0.5 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from importlib-metadata==4.4->markdown==2.6.8->tensorboard<
2.9,=>2.8->tensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (3.18.1)
Requirement already satisfied: pyasn1<0.7.0,=>0.4.0 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from pyasn1-modules==0.2.1->google-auth<3,=>1.6.
3->tensorboard<2.9,=>2.8->tensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (0.6.0)
Requirement already satisfied: oauthlib==0.7.0 in /home/donnick/anaconda3/envs/python3.8-tk2-2.0/lib/python3.8/site-packages (from requests-oauthlib==0.7.0->google-auth-oauthlib<0
.5,=>0.4.1->tensorboard<2.9,=>2.8->tensorflow==2.8.0->rknn-toolkit2==2.0.0b0+9bab5682) (3.2.2)
Installing collected packages: rknn-toolkit2
Successfully installed rknn-toolkit2-2.0.0b0+9bab5682

```



```
adb push rkllm_qwen_1.8.zip /userdata/aidemo
```

在开发板串口终端进入到 /userdata/aidemo 目录，执行以下命令。

(llm_start.sh 是启动脚本，依旧是需要自己写，这里不详细展开)

```
cd /userdata/aidemo unzip rkllm_qwen_1.8.zip
chmod +x llm_start.sh
./llm_start.sh
```

加载大模型会比较慢，需要稍等一段时间，成功加载如下图。

```
rkllm init start
rkllm init success

*****可输入以下问题对应序号获取回答/或自定义输入*****

[0] 把下面的现代文翻译成文言文：到了春风和煦，阳光明媚的时候，湖面平静，没有惊涛骇浪，天色湖光相连，一片碧绿，广阔无际；沙洲上的鸥鸟，时而飞翔，时而停歇，美丽的鱼游来游去，岸上与小洲上的花草，青翠欲滴。
[1] 以咏梅为题目，帮我写一首古诗，要求包含梅花、白雪等元素。
[2] 上联：江边惯看千帆过
[3] 把这句话翻译成中文：Knowledge can be acquired from many sources. These include books, teachers and practical experience, and each has its own advantages. The knowledge we gain from books and formal education enables us to learn about things that we have no opportunity to experience in daily life. We can also develop our analytical skills and learn how to view and interpret the world around us in different ways. Furthermore, we can learn from the past by reading books. In this way, we won't repeat the mistakes of others and can build on their achievements.
[4] 把这句话翻译成英文：RK3588是新一代高端处理器，具有高算力、低功耗、超强多媒体、丰富数据接口等特点

*****

user: 
```

图 1.2.2 成功加载 qwen 的 rkllm 模型

可以输入序号 0 回车，通过预设的问题获得回答，也可以自行输入中文问题或者英文问题提问后回车可获得回答。

```
user: 0
把下面的现代文翻译成文言文：到了春风和煦，阳光明媚的时候，湖面平静，没有惊涛骇浪，天色湖光相连，一片碧绿，广阔无际；沙洲上的鸥鸟，时而飞翔，时而停歇，美丽的鱼游来游去，岸上与小洲上的花草，青翠欲滴。
robot: 至春和煦之时，湖面平静，无波涛汹涌，天色湖光相接，一片碧绿，广阔无垠；沙洲之鸥鸟，时而翱翔，时而栖息，美丽之鱼游来游去，岸上与洲渚之花草，青翠欲滴。
```

图 1.2.2 成功加载 qwen 的 rkllm 模型

以下为 windows 部分的模型部署

由于代码全都是我手搓的, 所以全都不详细展开

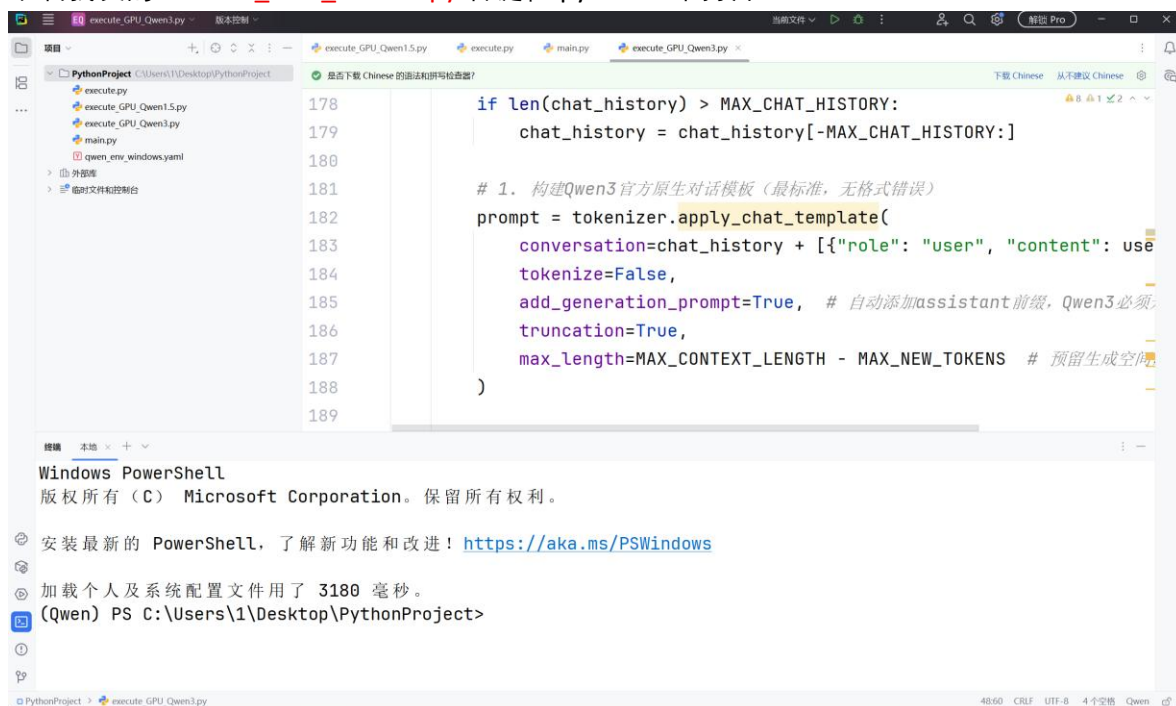
只概述大体流程

安装 pycharm 开发环境

进入官网 [PyCharm](https://www.jetbrains.com/pycharm/), 您需要的唯一 Python IDE 安装 pycharm

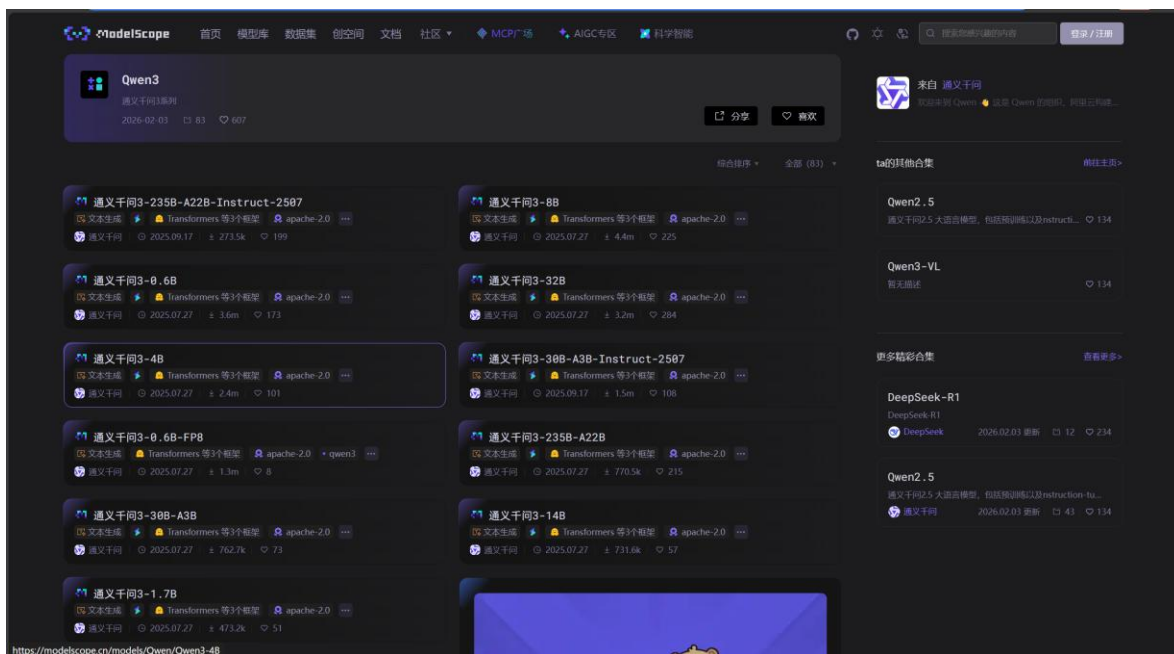
能一键补全你缺的所有环境

下载我发的 `execute_GPU_Qwen3.py` 右键在 pycharm 中打开

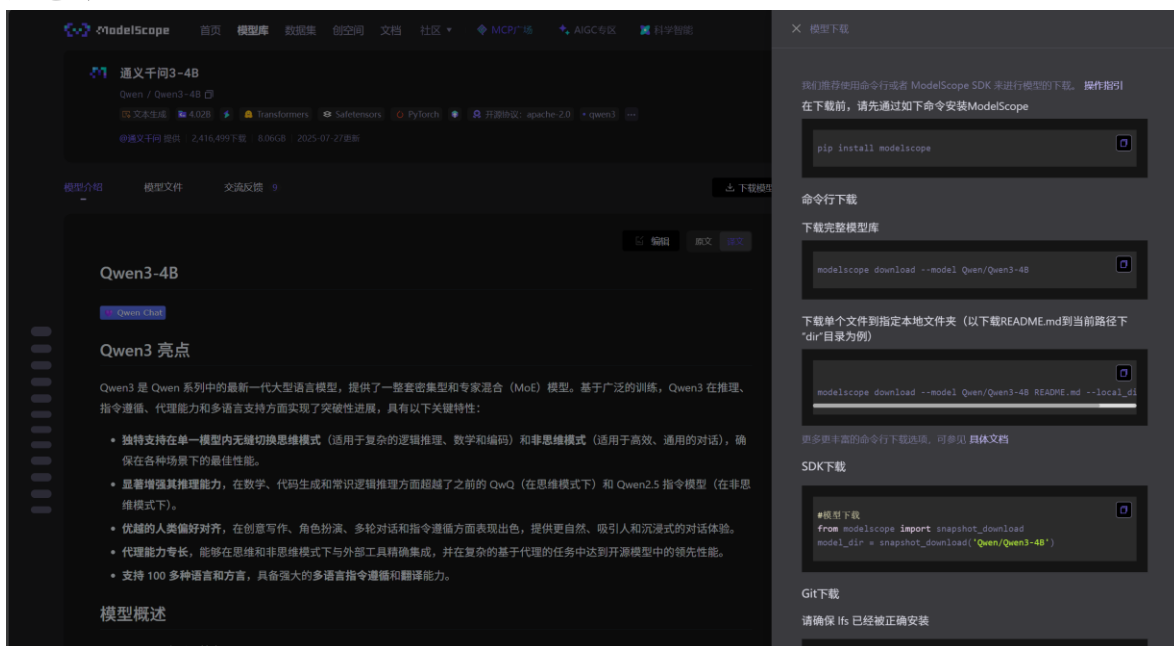


点击左下第四个, 进入终端

在通义千问官网找个你喜欢的模型 [Qwen3 合集详情-来自 Qwen·魔搭社区](#)



点击进入



点下载模型

在终端中依次执行

```
pip install modelscope
```

```
modelscope download --model Qwen/Qwen3-4B
```



```
# 设置随机种子（注释则生成结果随机，保留可复现）
set_seed(42)

# ===== 自定义配置（仅改模型路径！ RTX3060 专属适配） =====
MODEL_CACHE_PATH = r"D:\Qwen_Model\models\Qwen\Qwen3-1___7B" # 你的本地模
MODEL_SERIES = "Qwen3"
MODEL_VERSION = "8B"
```

我的代码用的是改过环境变量的绝对路径

记得改成你自己的相对路径

模型下载默认存放地址#

模型会下载到 ~/.cache/modelscope/hub 默认路径下。

如果需要修改 cache 目录，可以手动设置环境变量：MODELSCOPE_CACHE，完成设置后，模型将下载到该环境变量指定的目录中。